



ОБЛАСТНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ПРОФЕССИОНАЛЬНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
«ТОМСКИЙ ЭКОНОМИКО-ПРОМЫШЛЕННЫЙ КОЛЛЕДЖ»

К ЗАЩИТЕ ДОПУЩЕН

Директор ОГБПОУ «ТЭПК»

_____ / О.Н. Пояркова

« ____ » _____ 2025 г.

**ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА СЕРВИСА ИГРОВЫХ
СОХРАНЕНИЙ «PLAYSAVEBACK»**

ДИПЛОМНЫЙ ПРОЕКТ

Специальность 09.02.07 Информационные системы и программирование

Выполнил:

Студент IV курса группы 1912с

_____ / Заикин В.А.

« ____ » _____ 2025 г.

Проверил:

Руководитель ДП

_____ / Уляхин В.А.

« ____ » _____ 2025 г.

Томск 2025

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ.....	3
ГЛАВА 1. ТЕХНИЧЕСКОЕ ЗАДАНИЕ.....	4
Глоссарий	4
Общее описание	4
Требования к функциональности.....	6
Ролевая модель.....	6
Диаграмма прецедентов.....	7
Диаграммы последовательностей.....	8
Технологический стек.....	17
Хранимые данные	17
ГЛАВА 2. ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА БАЗЫ ДАННЫХ.....	18
Концептуальная модель данных	18
Логическая модель данных	19
Физическая модель данных	19
ГЛАВА 3. ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА API СЕРВИСА.....	21
Требования к REST API.....	21
Маршрутизация	36
Миграции	38
Заполнители	41
Модели.....	43
Сервисы.....	48
Контроллеры	54
ГЛАВА 4. ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ДЕСКТОПНОГО ПРИЛОЖЕНИЯ...68	68
Проектирование интерфейса десктопного приложения	68
Структура директорий проекта.....	74
Навигация и уведомления.....	75
Разработка страниц	83
ГЛАВА 5. ТЕСТИРОВАНИЕ	92
Тестирование серверной части	92
Тестирование десктопного приложения.....	100
ГЛАВА 6. РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ ИНФОРМАЦИОННОГО ПРОДУКТА.104	104
ЗАКЛЮЧЕНИЕ	109
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ.....	110

ВВЕДЕНИЕ

Современные игроки сталкиваются с проблемой хранения и управления своими игровыми сохранениями. Многие игры не предоставляют удобных средств для резервного копирования, версионности или переноса сохранений между устройствами. Это особенно актуально для пользователей, которые переустанавливают операционную систему, используют несколько игровых платформ или хотят вернуться к определённому состоянию игры. Существующие решения, такие как Steam Cloud, не всегда охватывают все игры и предоставляют ограниченные возможности управления резервными копиями, а также требуют наличие игры в их сервисе.

Разрабатываемая система решает эту проблему, предоставляя удобный инструмент для хранения, управления и синхронизации игровых сохранений в облаке. Пользователи смогут загружать и восстанавливать сохранения, отслеживать изменения, а также делиться своими данными с другими игроками.

Таким образом, система повышает удобство использования сохранений, минимизирует риск потери прогресса и предоставляет новые возможности для взаимодействия пользователей с их игровыми данными.

Цель дипломного проекта — спроектировать и разработать сервис игровых сохранений «PlaySaveBack». Для реализации поставленной цели были выделены следующие задачи:

1. Спроектировать и реализовать базу данных для сервиса с использованием СУБД SQLite.
2. Разработать API сервиса на основе фреймворка Laravel 12.
3. Создать десктопное приложение для клиента с использованием платформы WinUI 3.
4. Реализовать автоматизированное тестирование API с помощью Postman и проверить его функциональность.
5. Разработать сценарии ручного тестирования десктопного приложения и провести тестирование вручную.
6. Подготовить руководство пользователя для работы с десктопным приложением.

ГЛАВА 1. ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Глоссарий

Понятие	Определение понятия
API	(интерфейс программирования приложений) – набор определенных правил и соглашений, который используется для взаимодействия между различными программными компонентами. API определяет, как различные части программного обеспечения могут взаимодействовать друг с другом, обычно путем обмена данными и выполнения определенных операций.
Postman	Инструмент для тестирования и разработки API. Он предоставляет удобный интерфейс для отправки HTTP запросов к API, а также для анализа ответов.
Draw.io	Бесплатный онлайн и оффлайн инструмент для создания диаграмм и схем. Он предоставляет широкий набор инструментов и элементов, которые позволяют пользователям создавать различные типы диаграмм, такие как организационные диаграммы, блок-схемы, ER-диаграммы, сетевые диаграммы, диаграммы потока процессов и многое другое.
CRUD	Акроним, обозначающий четыре базовые функции, используемые при работе с данными: создание, чтение, модификация, удаление.
Laravel	Бесплатный, открытый фреймворк для создания веб-приложений на языке программирования PHP. Он предоставляет структуру и базовые инструменты для быстрого создания веб-приложений, используя принципы модели-представления-контроллер (MVC) и другие современные подходы к разработке.

Общее описание

Разрабатываемый сервис представляет собой систему управления сохранениями видеоигр с возможностью резервного копирования и восстановления данных. В основе проекта лежит клиентское приложение на WinUI 3, взаимодействующее с серверной частью, реализованной на Laravel. Система предназначена для геймеров, которые хотят сохранять, управлять и восстанавливать свои игровые данные без привязки к конкретному устройству.

Система поддерживает версиюность сохранений, интеграцию с облачными сервисами и возможность совместного использования сохранений с другими пользователями.

При отсутствии игры в базе данных пользователю предоставляется возможность добавить её в библиотеку в качестве сторонней. Это позволит сохранять и управлять файлами сохранений для данной игры. Однако оформление сохранений в виде публичных постов будет доступно только для игр, присутствующих в базе данных.

Функционал гостя охватывает регистрацию, авторизацию и восстановления аккаунта (см. рис. 1).

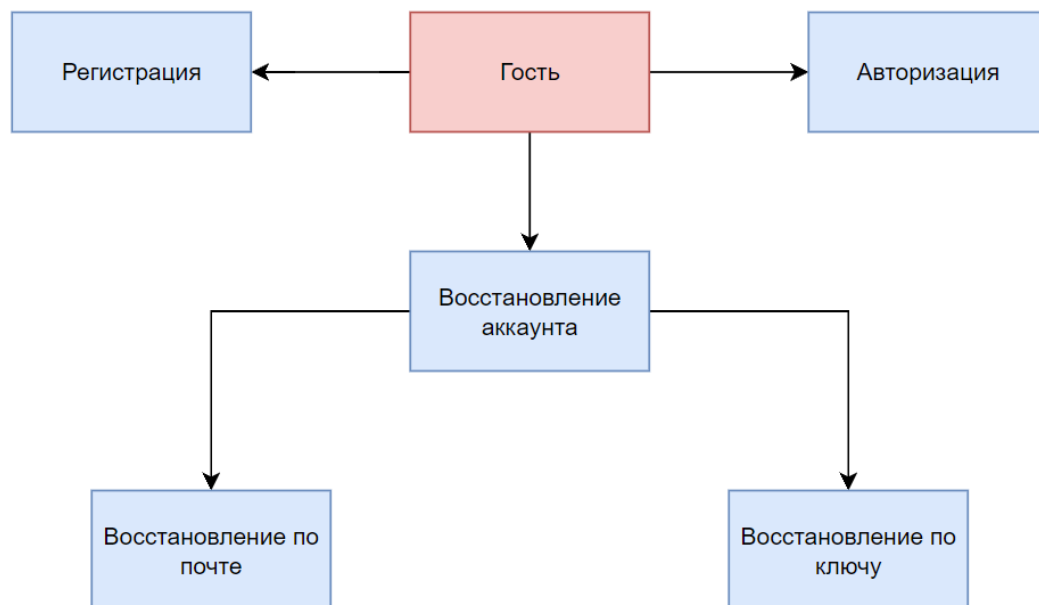


Рисунок 1 – Функционал гостя

Функционал пользователя охватывает управление библиотекой игр, работой с сохранениями и профилем, управление своими игровыми данными. Детали функционала пользователя представлены на рисунке 2.

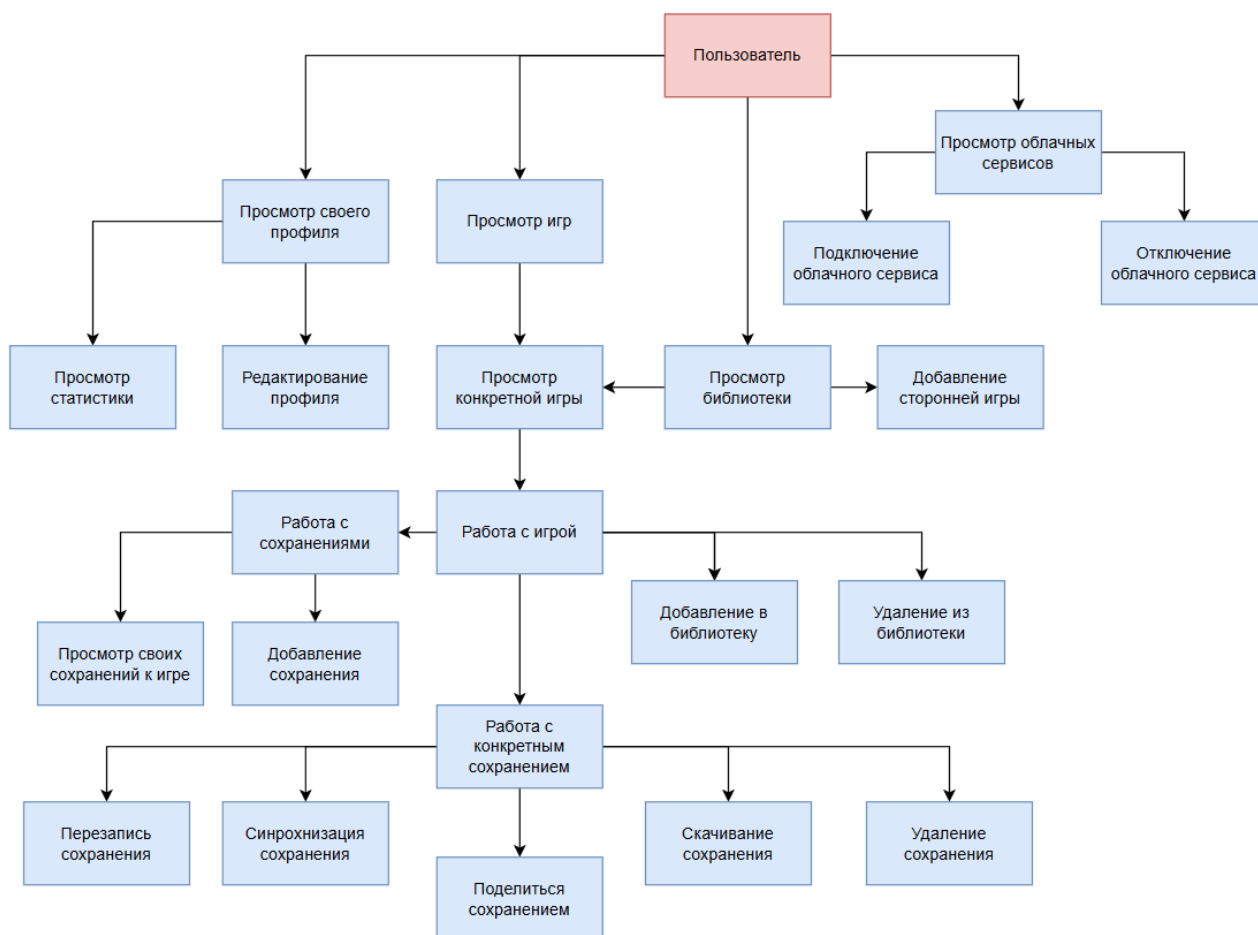


Рисунок 2 – Функционал пользователя

Требования к функциональности

Гости

1. Аутентификация и регистрация.
 - Аутентификация производится по токену, который генерируется при входе и отзывается при выходе.
 - Вход на другом устройстве не должен вызывать выход из аккаунта на всех устройствах одного пользователя.
2. Восстановление аккаунта.
 - Возможность восстановления аккаунта по особому ключу, выданного при регистрации.
 - Возможность восстановления аккаунта через электронную почту.

Пользователи

1. Управление своей библиотекой игр.
 - Добавление игры в библиотеку из списка доступных игр.
 - Добавление сторонней игры, если нужной игры нет в БД.
 - Удаление игры из библиотеки.
 - Добавление или удаление игры в избранное.
2. Управление сохранениями.
 - Просмотр списка всех сохранений.
 - Просмотр сохранений для конкретной игры.
 - Загрузка сохранения на облачное хранилище.
 - Восстановление сохранения.
 - Перезапись сохранения.
 - Добавление нового сохранения для игры.
 - Удаление сохранения.
 - Поделиться сохранением.
3. Работа с профилем
 - Просмотр своего профиля.
 - Обновление данных профиля.
 - Просмотр статистики.
4. Просмотр информации об играх.
 - Просмотр списка всех доступных игр.
 - Просмотр информации о конкретной игре.

Ролевая модель

Клиент — основной авторизованный пользователь сервиса, который загружает, организует и пользуется сохранениями.

Гость — неавторизованный пользователь, которому доступна только регистрация, вход и восстановление аккаунта.

Более структурированная и подробная информация представлена в таблице 1.

Таблица 1 – Ролевая модель системы

Раздел	Фичи	Пользователь	Гость
--------	------	--------------	-------

Аутентификация и регистрация	Вход	нет	да
	Регистрация	нет	да
	Восстановление аккаунта	нет	да
	Выход	да	нет
Профиль	Просмотр своего профиля	да	нет
	Обновление профиля	да	нет
Библиотека	Получить свою библиотеку	да	нет
	Добавить игру в библиотеку	да	нет
	Удалить игру из библиотеки	да	нет
	Добавить игру в избранное	да	нет
	Убрать игру из избранного	да	нет
	Обновление данных библиотеки	да	нет
Сохранения	Перезапись сохранения	да	нет
	Просмотр своих сохранений к игре	да	нет
	Синхронизация сохранения	да	нет
	Создание сохранения	да	нет
	Удаления сохранения	да	нет
	Восстановление сохранения	да	нет
	Скачивание сохранения	да	нет
Игры	Просмотр всех игр	да	нет
	Просмотр игры	да	нет
Сторонние игры	Добавление сторонней игры	да	нет
Облачные сервисы	Подключение облачного сервиса Google Drive	да	нет
	Отключение облачного сервиса Google Drive	да	нет

Диаграмма прецедентов

Диаграмма прецедентов используется для моделирования функциональных требований системы и взаимодействия между пользователями и системой. Она помогает определить основные сценарии использования системы и её ключевые функции. Состоит из следующих ключевых элементов:

- Акторы: роли пользователей или внешних систем, взаимодействующих с системой.
- Прецеденты: функциональные возможности или сценарии, которые система предоставляет.
- Связи: взаимодействия между акторами и прецедентами.
- Включение и расширение: отношения между прецедентами для повторного использования или добавления дополнительного поведения.

На диаграмме отражены ключевые функции, такие как управление сохранениями, библиотекой, работа со своим профилем и играми (см. рис. 3).

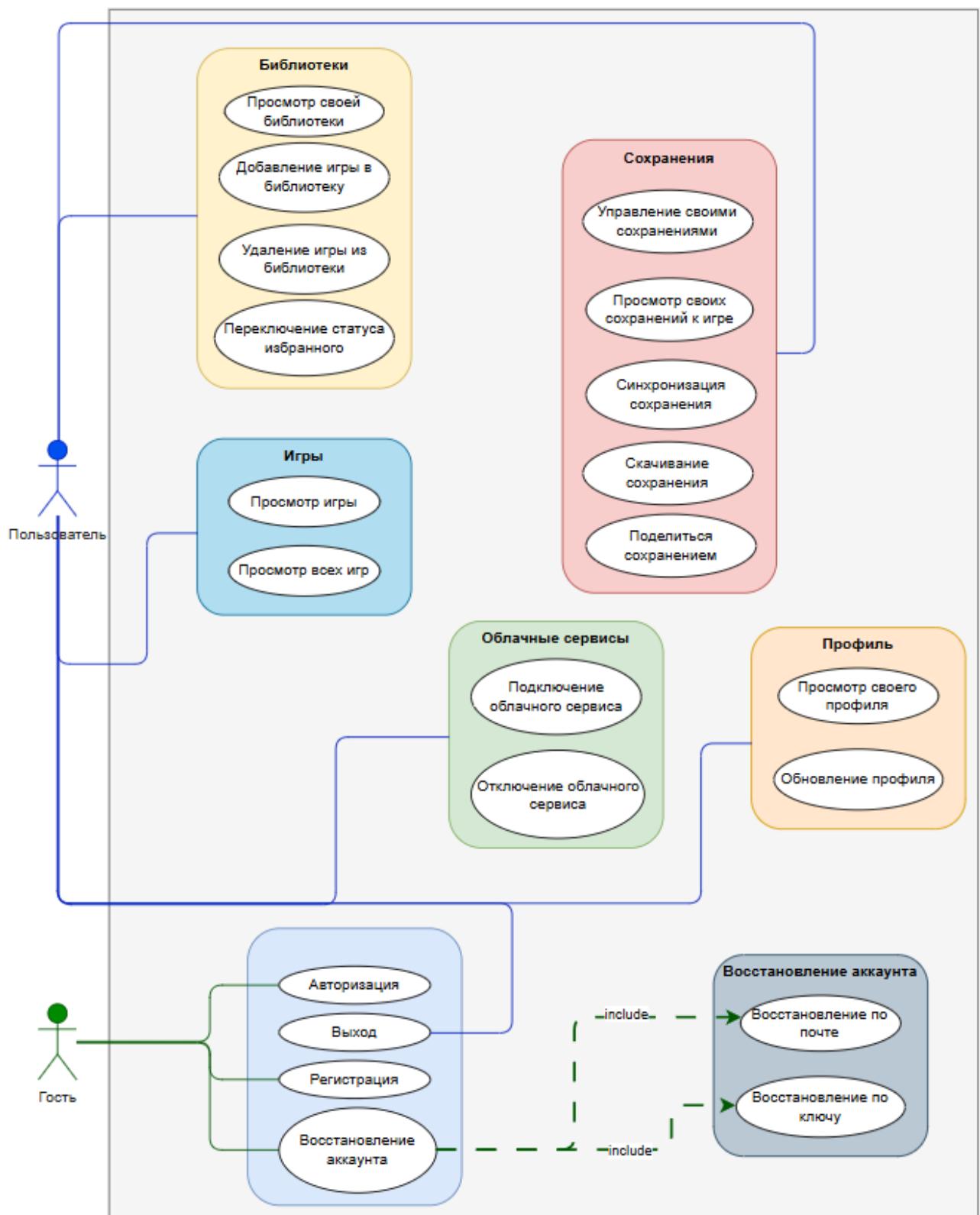


Рисунок 3 – Диаграмма прецедентов

Диаграммы последовательностей

Диаграмма последовательностей визуализирует взаимодействие объектов в системе в рамках конкретного сценария. Она отражает последовательность сообщений, передаваемых

между объектами, и их временную упорядоченность. Состоит из следующих ключевых элементов:

- Объекты и акторы: участники взаимодействия.
- Сообщения: вызовы методов или передача данных между объектами.
- Временная ось: вертикальная линия, показывающая порядок событий.
- Активация: период активности объекта при выполнении операции.
- Фреймы: обёртки над элементами для описания конкретного случая или цикла.

Общие диаграммы-дополнения

Общие диаграммы последовательности не являются самостоятельными и предназначены для дополнения к другим последовательностям для разгрузки повторяющихся элементов. Обычные диаграммы ссылаются на дополнительные посредством цветной активации по цвету выхода и припиской «(Общее: <название>)».

Валидация тела POST запроса осуществляется посредником, проверяя заданные правила на ключах (см. рис. 4). Имя ссылки: «валид».

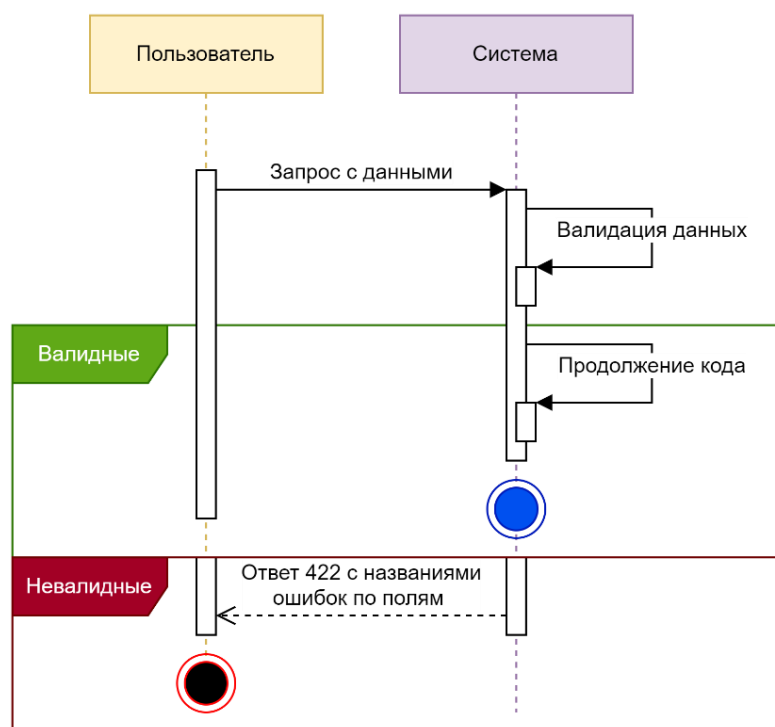


Рисунок 4 – Валидация тела запроса

Внесения в контекст кода модель пользователя и отброс неавторизированных осуществляется посредником, проверяя токен в заголовках запроса и запрашивая пользователя с этим ИД (см. рис. 5). Имя ссылки: «токен».

Внесения в контекст кода модель объекта по ИД и отброс если не модель не найдена осуществляется посредником, беря из маршрута ИД и запрашивая из БД её модель (см. рис. 6). Имя ссылки: «сущ».

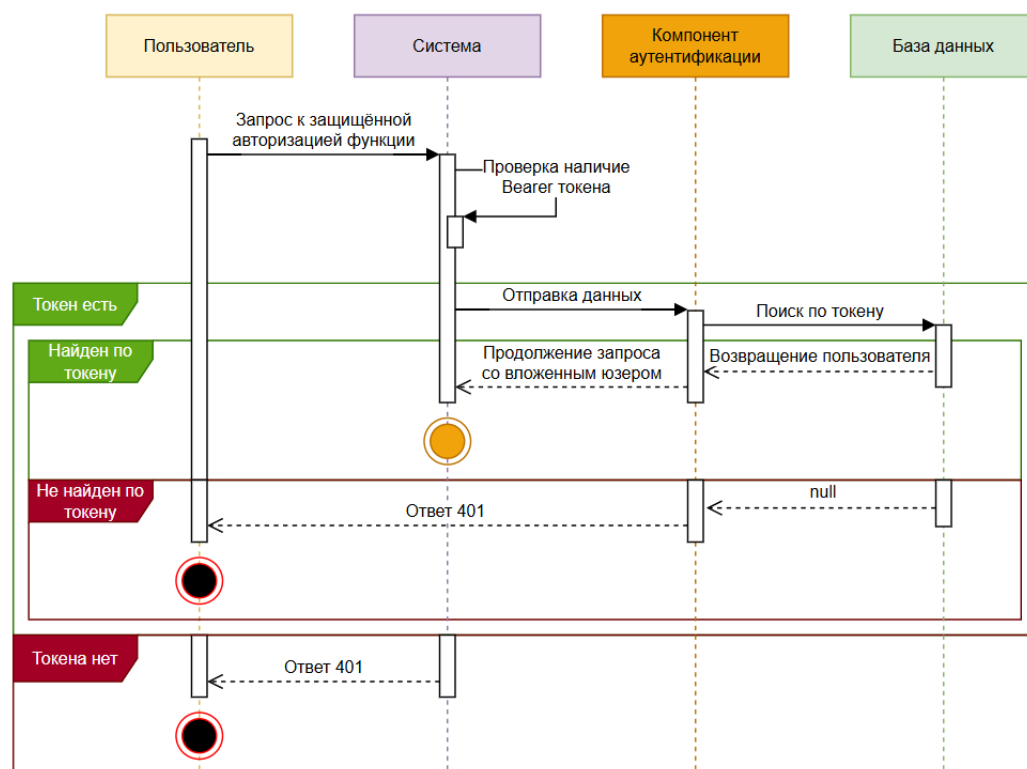


Рисунок 5 – Проверка токена и внесение в контекст пользователя

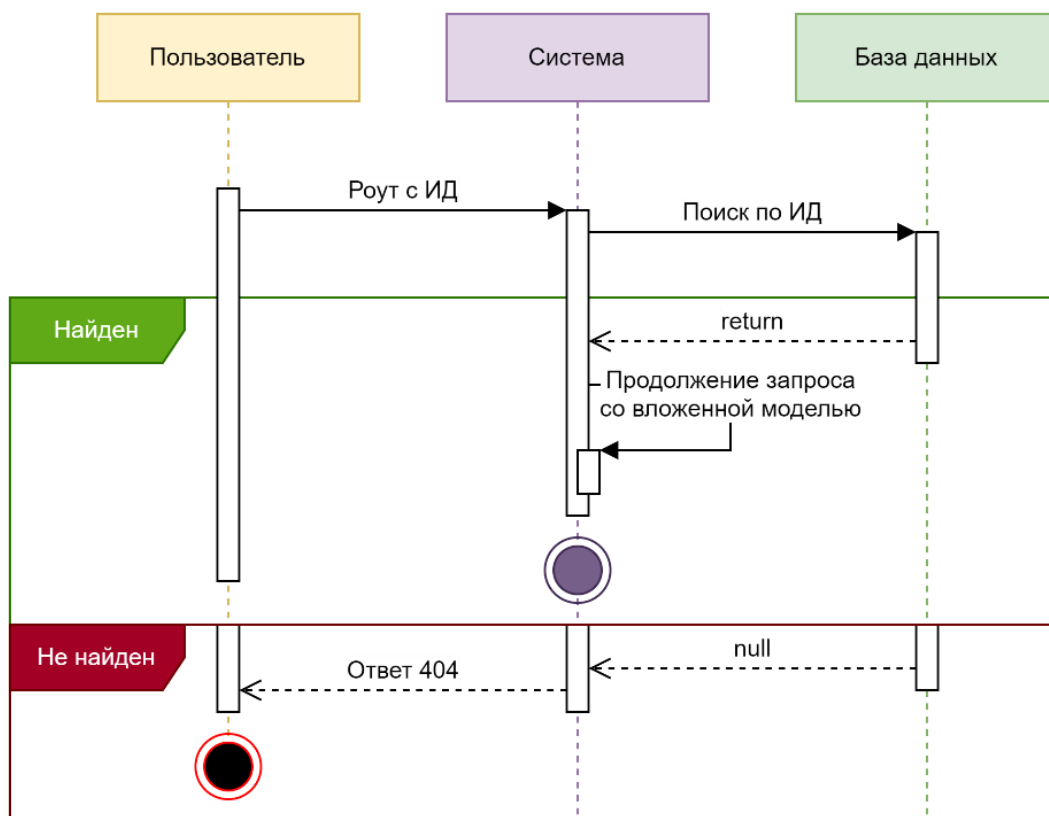


Рисунок 6 – Проверка существования объекта БД

Функции аутентификации

Регистрация новой учётной записи представлена на рисунке 7.

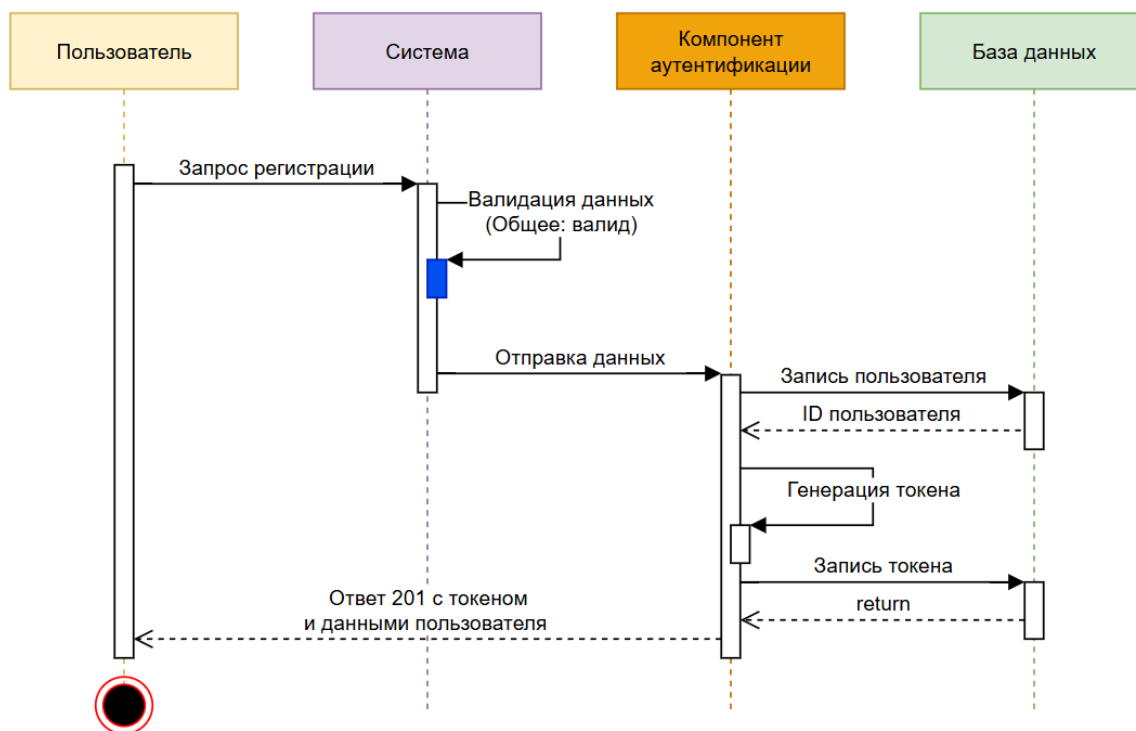


Рисунок 7 – Регистрация

Вход по реквизитам представлен на рисунке 8.

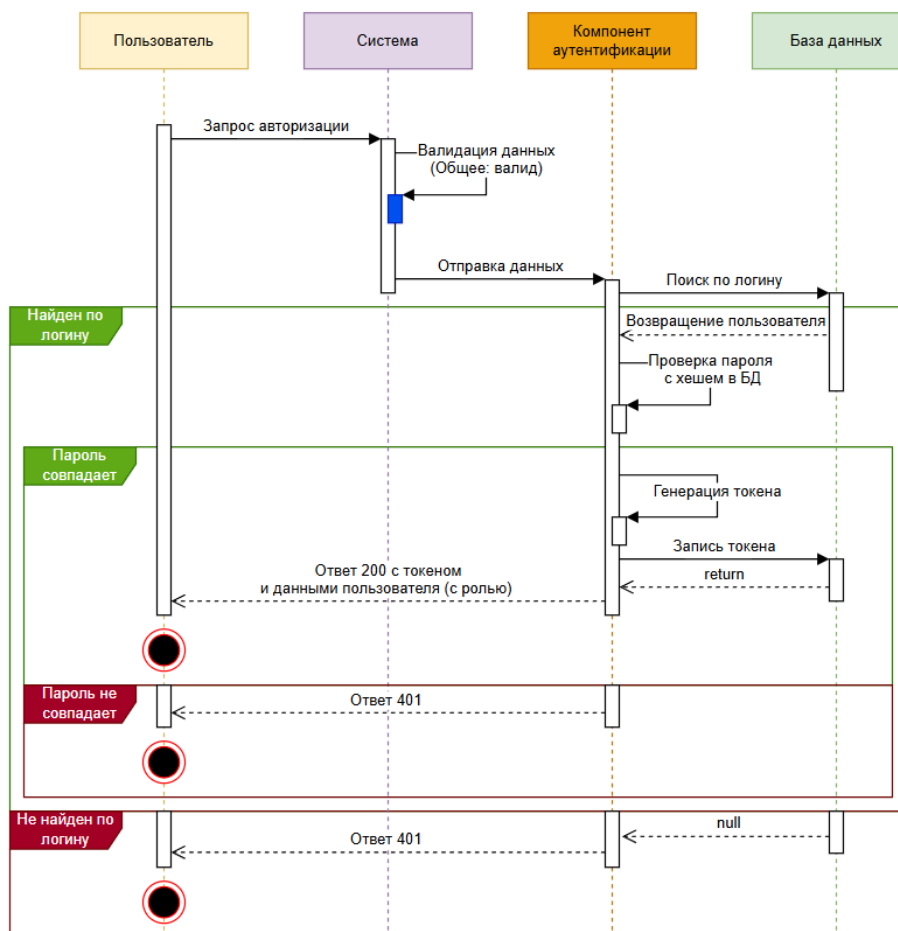


Рисунок 8 – Вход

Выход и удаление токена представлен на рисунке 9.

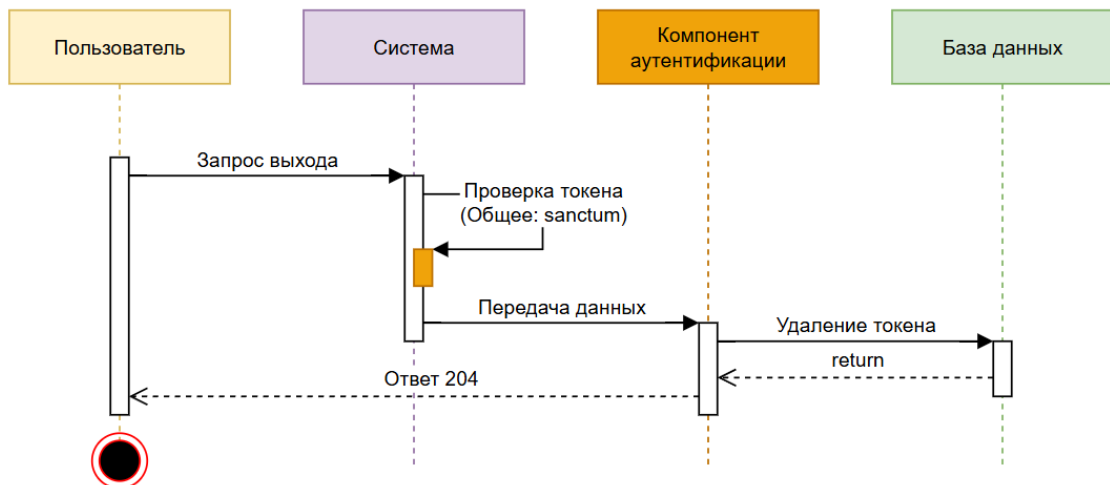


Рисунок 9 – Выход

Восстановление аккаунта

Восстановление пароля с помощью KEY представлен на рисунке 10.

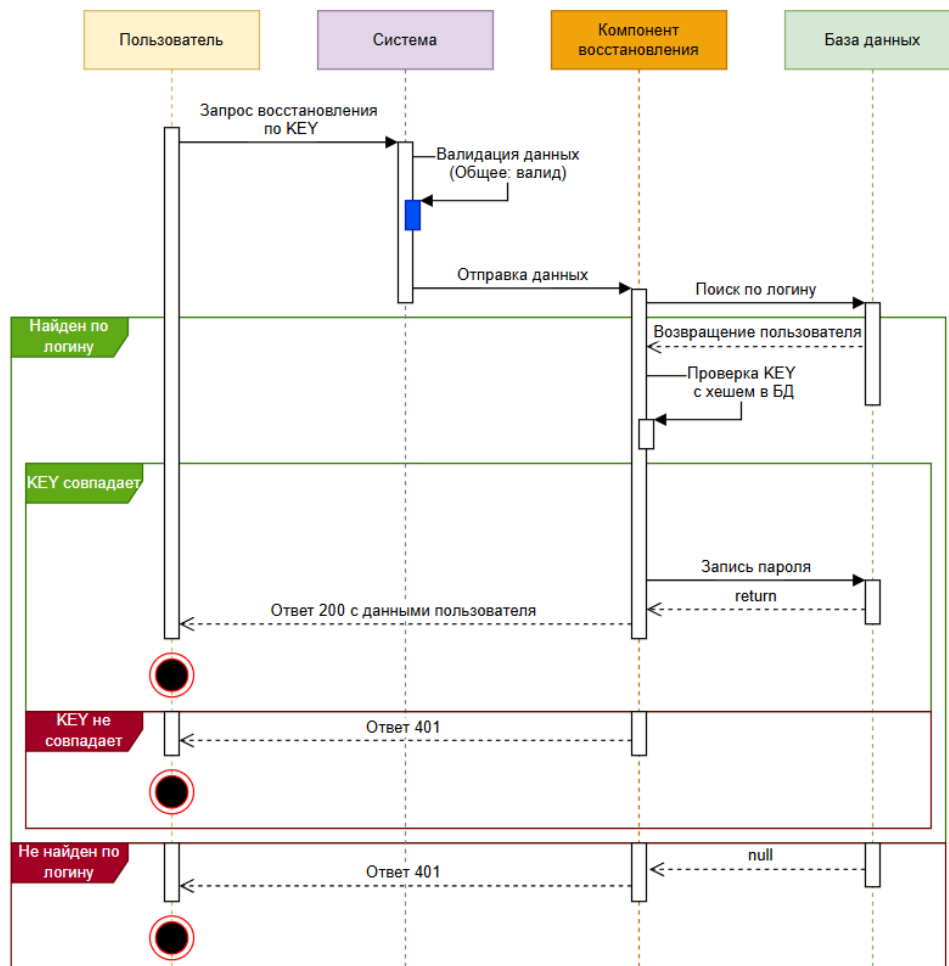


Рисунок 10 – Восстановление по KEY

Работа с профилем

Просмотр своего профиля представлен на рисунке 11.

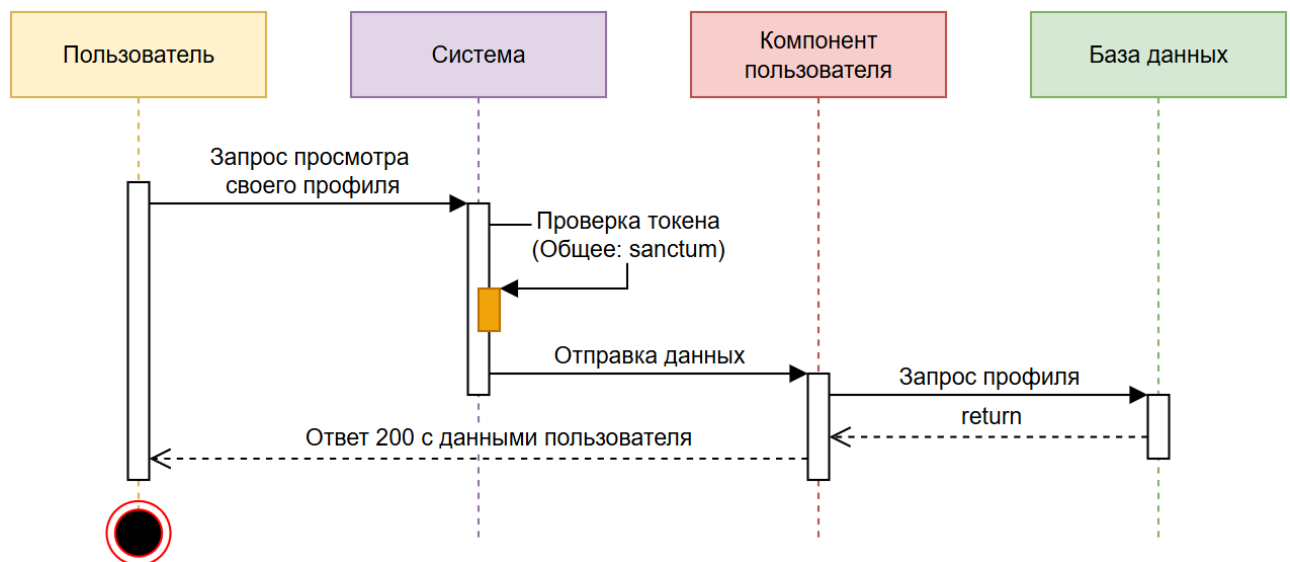


Рисунок 11 – Просмотр своего профиля

Обновление своего профиля представлен на рисунке 12.

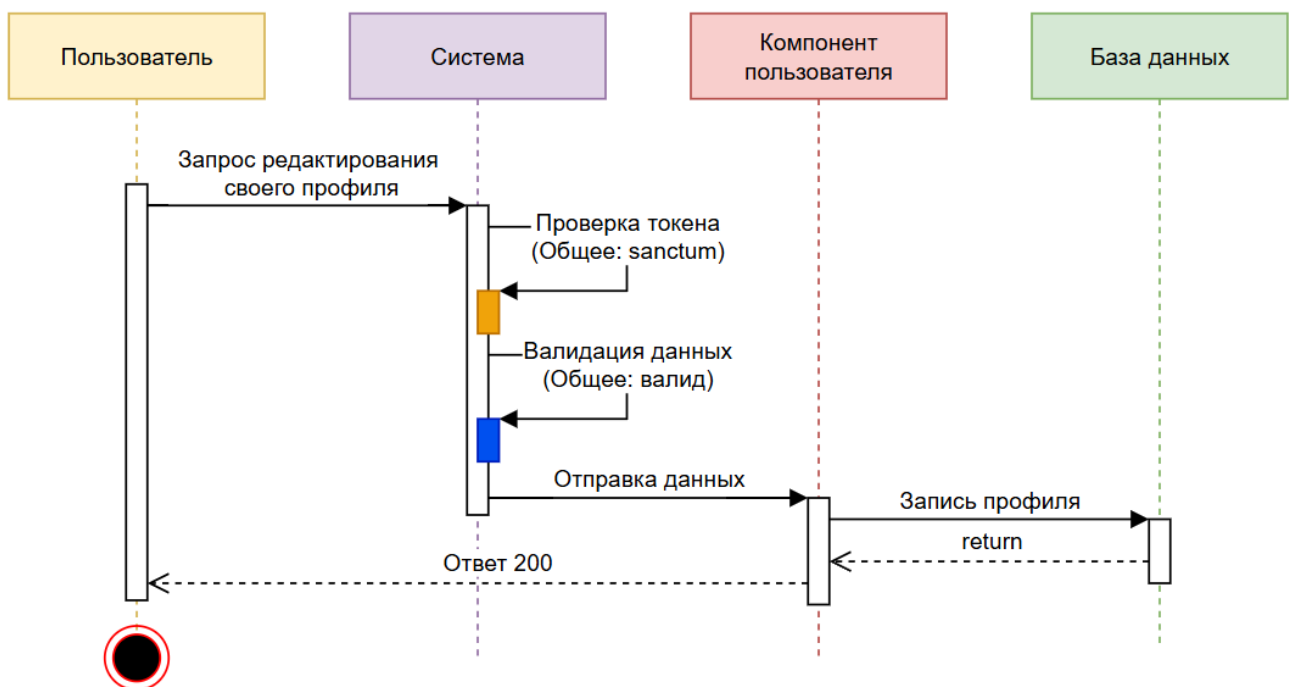


Рисунок 12 – Обновление своего профиля

Работа с библиотекой

Получение свой библиотеки представлен на рисунке 13.

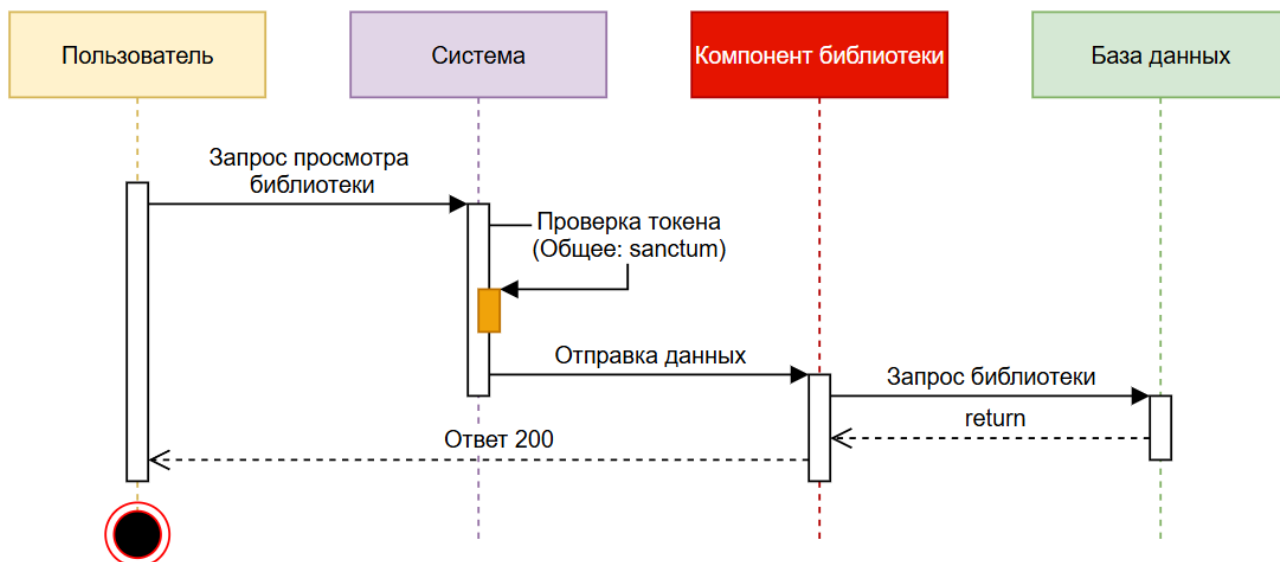


Рисунок 13 – Получение своей библиотеки

Добавление игры в библиотеку представлен на рисунке 14.

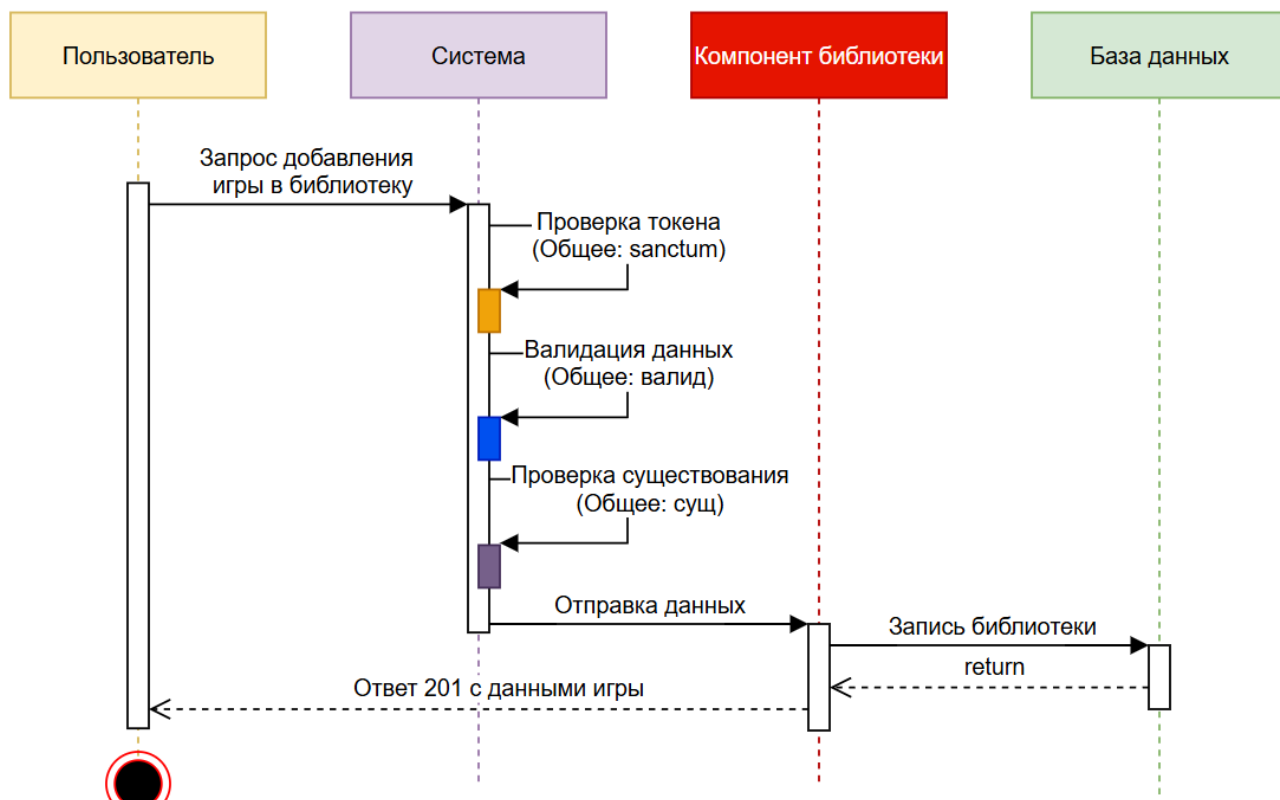


Рисунок 14 – Добавление игры в библиотеку

Удаление игры из библиотеки представлен на рисунке 15.

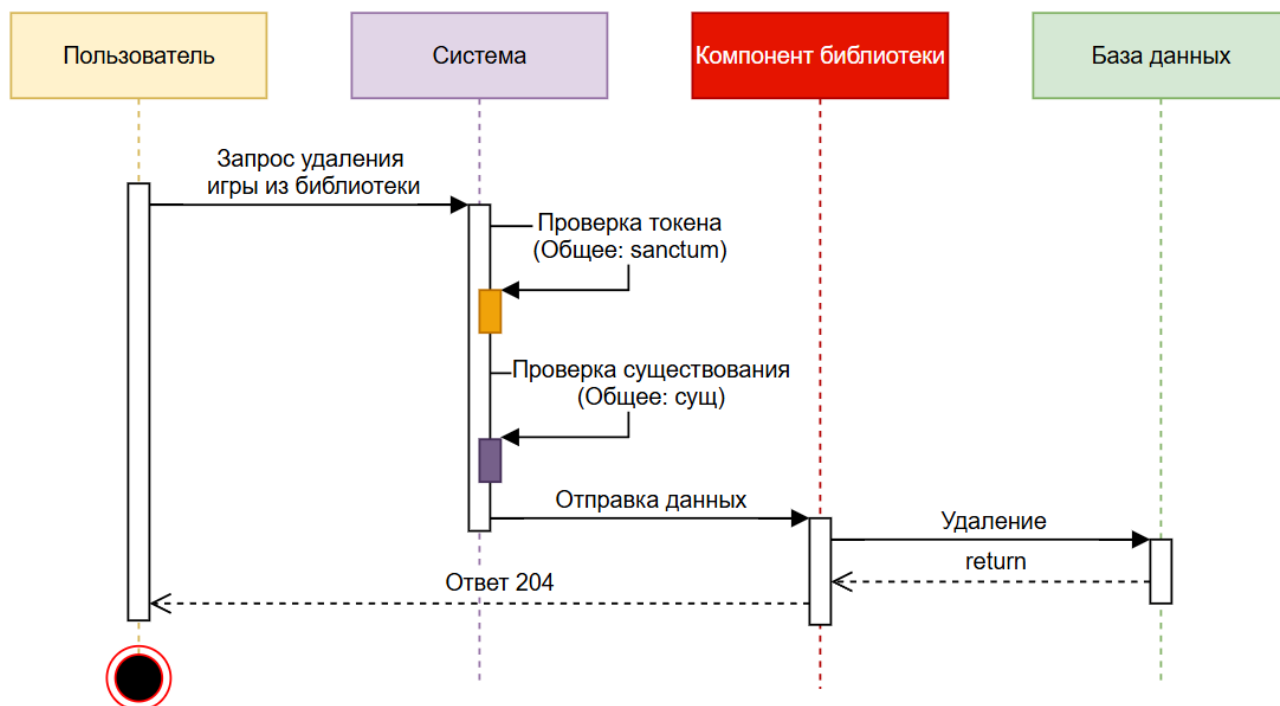


Рисунок 15 – Удаление игры из библиотеки

Обновление данных игры в библиотеке представлен на рисунке 16.

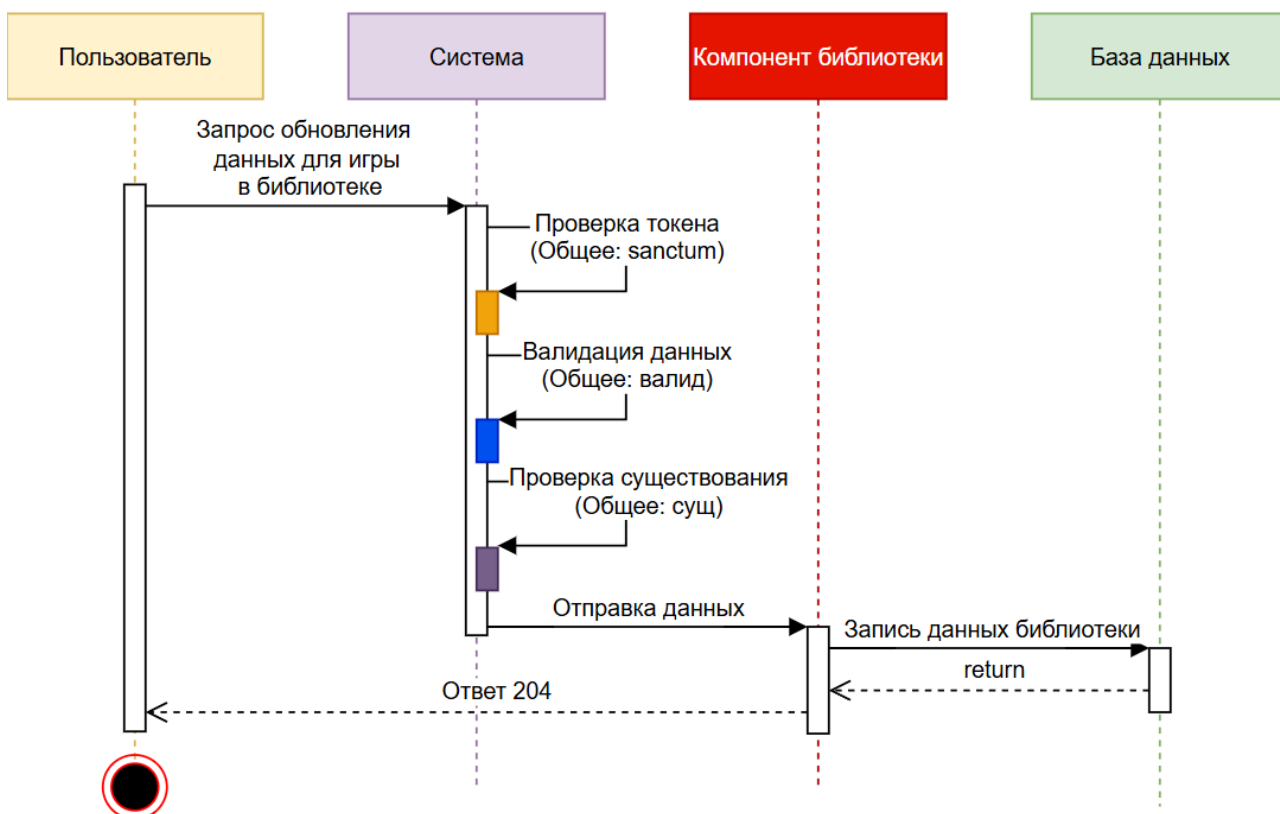


Рисунок 16 – Обновление данных игры в библиотеке

Переключение избранного статуса игры представлен на рисунке 17.

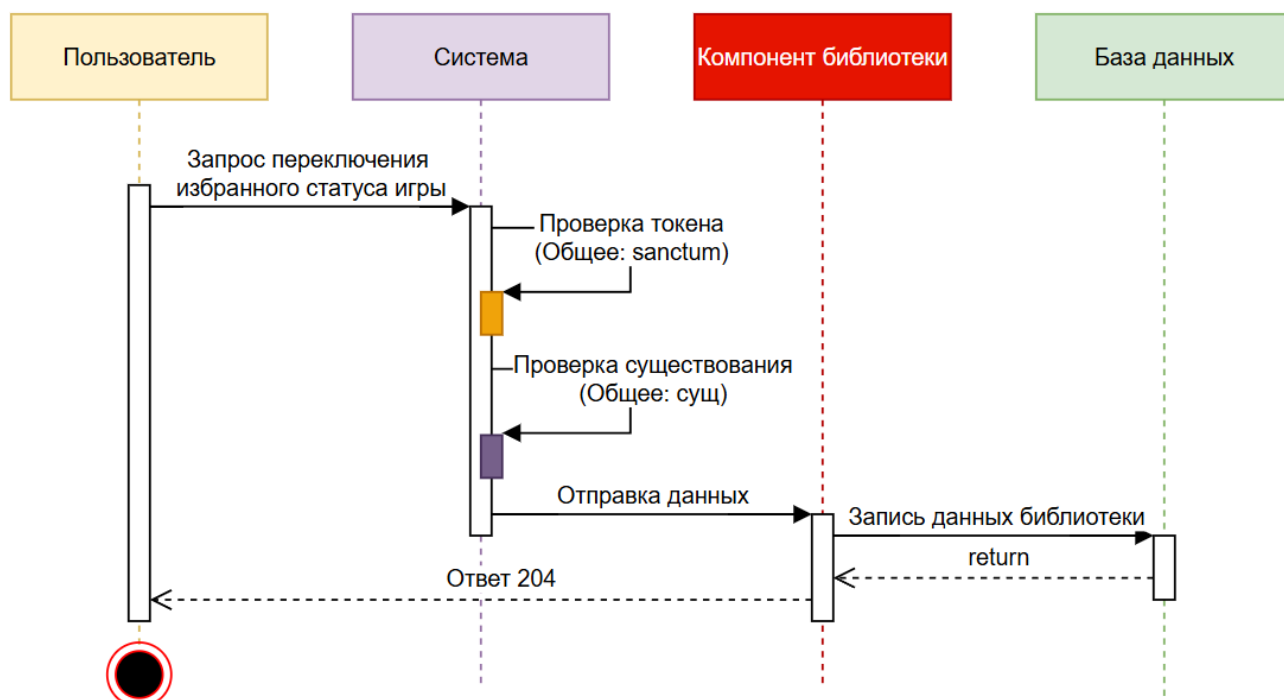


Рисунок 17 – Переключение избранного статуса игры

Технологический стек

Серверная часть

- Язык программирования: PHP 8.3.
- Фреймворк для разработки: Laravel 12.
- Система управление базами данными: SQLite.

Десктопное приложение

- Язык программирования и разметки: C#, XAML.
- Фреймворк для разработки: WinUI 3.

Хранимые данные

1. **Пользователи:** уникальный идентификатор пользователя, электронная почта, хеш пароля, никнейм, аватар, логин, заголовок профиля, ключ.
2. **Игры:** уникальный идентификатор игры, название игры, платформа, код игры.
3. **Сторонние игры:** уникальный идентификатор сторонней игры, название сторонней игры, идентификатор пользователя.
4. **Облачные сервисы:** уникальный идентификатор облачного сервиса, название облачного сервиса, описание облачного сервиса, иконка облачного сервиса.
5. **Облачные сервисы пользователей:** токен доступа, токен обновления, дата истечения токена, идентификатор облачного сервиса, идентификатор пользователя, статус, внешний идентификатор пользователя.
6. **Сохранения:** уникальный идентификатор сохранения, идентификатор файла сохранения, название файла сохранения, название версии сохранения, размер файла сохранения, описание сохранения, хэш файла сохранения, дата последней синхронизации сохранения, идентификатор пользователя, идентификатор игры, идентификатор сторонней игры, идентификатор облачного сервиса пользователя.
7. **Библиотеки:** уникальный идентификатор библиотеки, идентификатор пользователя, идентификатор игры, время последнего запуска игры, время, проведённое в игре, флаг избранного, идентификатор сторонней игры.
8. **Пути:** уникальный идентификатор пути, путь к папке сохранений, идентификатор игры.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА БАЗЫ ДАННЫХ

Концептуальная модель данных

Концептуальная модель — это формализованное описание предметной области, не связанное с какими-либо компьютерными средствами. Сущности (таблицы) обозначаются прямоугольниками; Атрибуты (поля таблиц) — овалы, которые могут быть объединены в скруглённые прямоугольники; Связи с обозначением мощности — стрелками (в направлении, где мощность равна многим — двойная стрелка, а со стороны, где она равна единице — одинарная).

Анализируя предметную область, можно выделить следующие сущности с их атрибутами (см. рис. 18):

1. Сущность «**Пользователь**» с атрибутами «Логин», «Никнейм», «Хеш пароля», «Аватар», «Шапка», «Хеш ключа», «Почта».
2. Сущность «**Путь**» с атрибутом «Путь».
3. Сущность «**Игра**», с атрибутами «Платформа», «Название», «Код».
4. Сущность «**Сторонняя игра**» с атрибутом «Название».
5. Сущность «**Облачный сервис**» с атрибутами «Название», «Описание», «Иконка».
6. Сущность «**Привязанный облачный сервис**» с атрибутами «Дата окончания», «Токен обновления», «Токен доступа», «Статус», «Внешний идентификатор пользователя».
7. Сущность «**Библиотека**» с атрибутами «В избранном», «Сыграно», «Последний запуск».
8. Сущность «**Сохранение**», с атрибутами «ИД файла», «Размер», «Дата последней синхронизации», «Название файла», «Описание», «Хеш», «Версия».

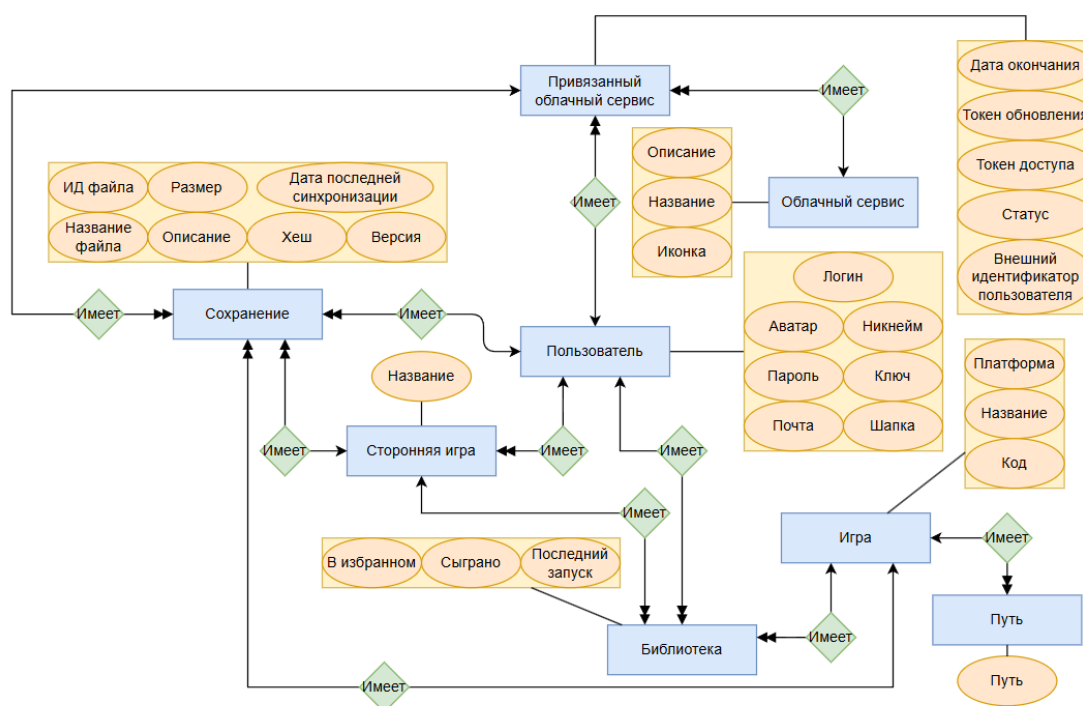


Рисунок 18 – Концептуальная модель данных

Логическая модель данных

Логическая модель базы данных — схема базы данных, выраженная в понятиях модели данных. Этим отличается от концептуальной модели, описывающей семантику предметной области без указания технологии (конкретных методов реализации), и от физической модели, которая описывает конкретные физические механизмы, применяемые для хранения данных в накопителях.

Используя нашу концептуальную модель данных и выполнив нормализацию данных до 3 нормальной формы включительно, построим логическую модель данных (см. рис. 19):

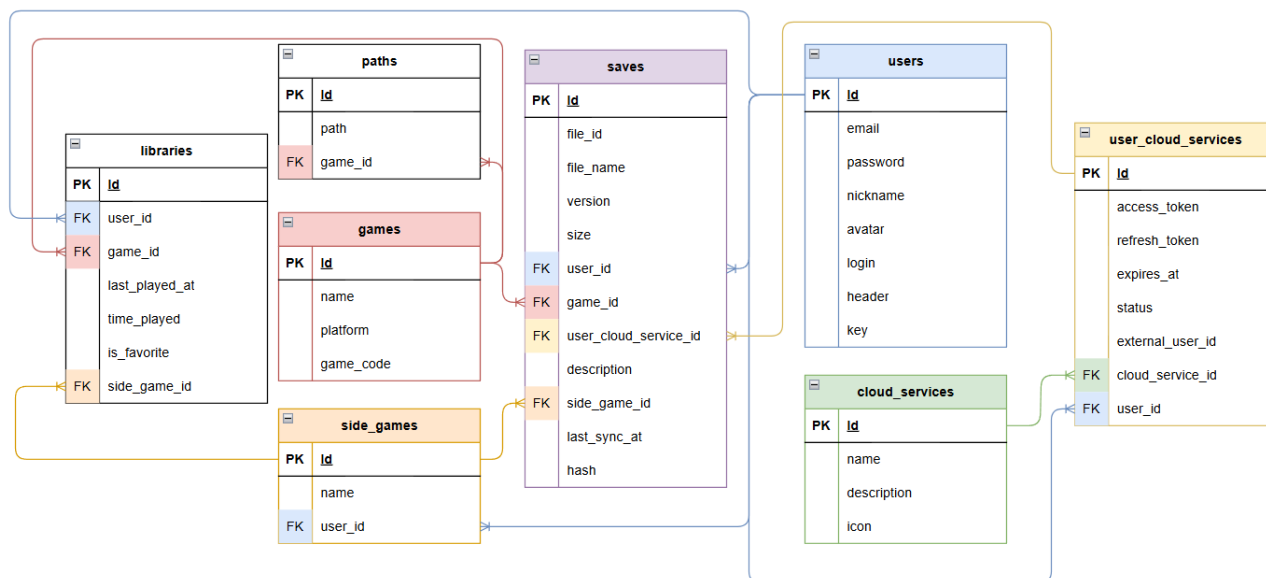


Рисунок 19 – Логическая модель данных

Физическая модель данных

Физическая модель данных — это модель данных, описанная с помощью средств конкретной системы управления базами данных (СУБД). Физическая модель данных строится на базе логической путем добавления особенностей конкретной СУБД. К таким особенностям могут относиться поддерживаемые СУБД типы данных, соглашения о присвоении имен таблицам, атрибутам и т.д. Физическая модель данных фактически является готовым заданием на создание БД, имея которое можно реализовать БД в выбранной СУБД.

Используя логическую модель, построим физическую модель данных для СУБД SQLite (см. рис. 20).

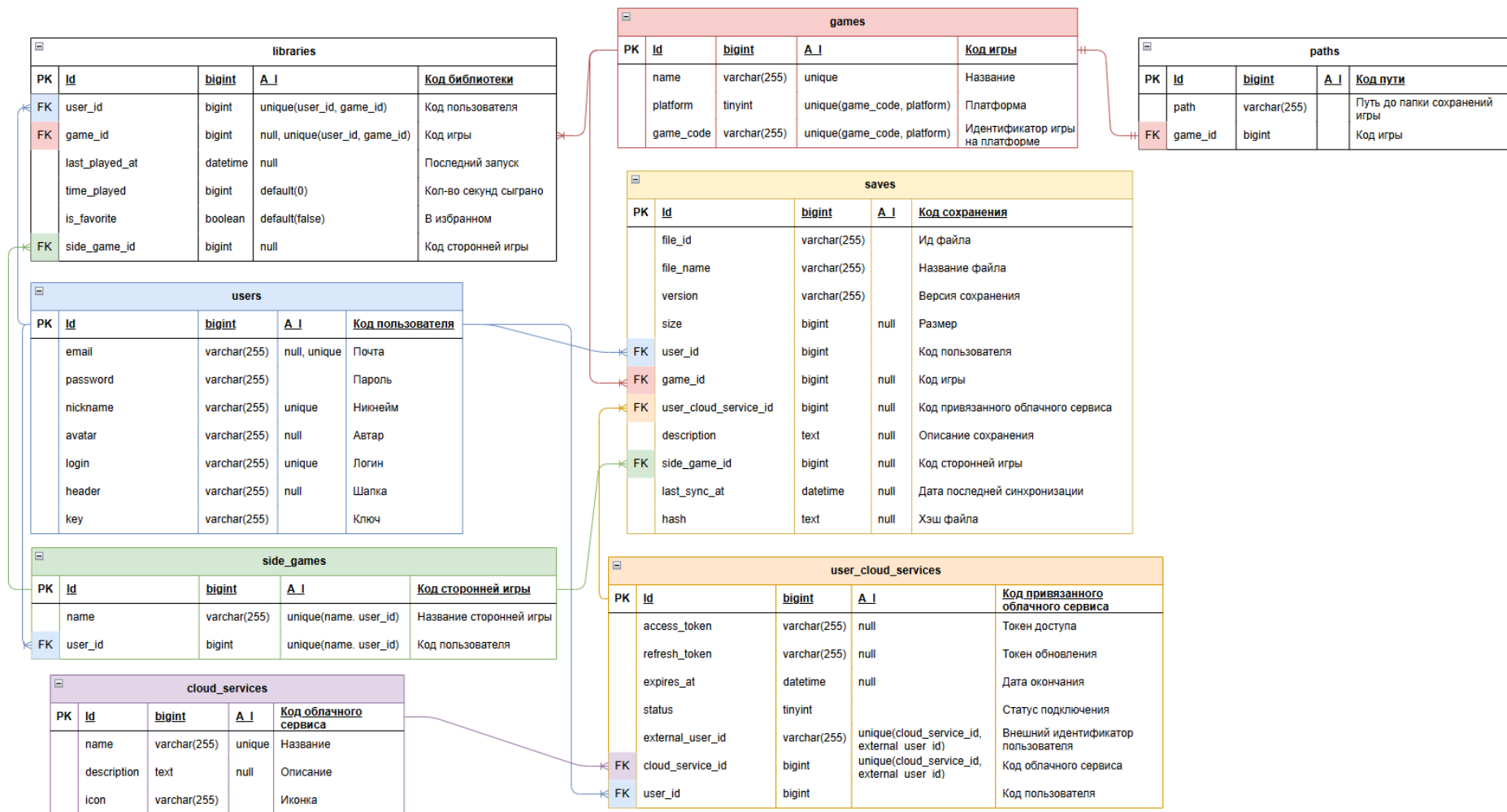


Рисунок 20 – Физическая модель данных

ГЛАВА 3. ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА API СЕРВИСА

Требования к REST API

Общие требования

Должно быть открытое API — правила CORS должны разрешать доступ с других доменов. Идентификацию пользователя для не гостевых функции должно быть организовано посредством Bearer Token.

При попытке доступа к защищенным авторизацией функциям системы во всех запросах необходимо возвращать ответ следующего вида:

Response
Status: 401 Unauthorized Content-Type: application/json Body: { "message": "Unauthorized" }

При попытке доступа авторизованным пользователем к функциям недоступным для своей роли во всех запросах необходимо возвращать ответ следующего вида:

Response
Status: 403 Forbidden Content-Type: application/json Body: { "message": "Forbidden" }

При попытке получить несуществующий ресурс возвращать ответ следующего вида:

Response
Status: 404 Not Found Content-Type: application/json Body: { "message": "Not Found" }

В случае ошибок связанных с валидацией данных в теле запроса необходимо возвращать следующий ответ:

Response
Status: 422 Unprocessable Content Content-Type: application/json Body: { "errors": [<ключ>: [<ошибка>, ...], <ключ>: [...], ...] }

Вместо поля <ключ> должно быть название ключа запроса, в котором произошла ошибка валидации, а вместо <ошибка> — описание допущенной ошибки в этом ключе. Ошибок в одном ключе может быть множество, как и в одном запросе — множество ключей с ошибками.

Аутентификация

РЕГИСТРАЦИЯ

Запрос позволяет пользователю зарегистрироваться, а также вернуть его токен и ключ.

Request	Response
URL: {api}/register Method: POST Content-Type: application/json Body: { "login": {login}, "password": {password}, "password_confirmation": {password_confirmation} }	Status: 201 Created Content-Type: application/json Body: { "user": <данные_пользователя>, "token": <токен>, "key": <ключ_пользователя> } Возможные статус-коды: 201, 422

Входные параметры:

№	Название	Тип	Обяз	Условия	Описание	Пример
1	api	string	да		URL до API	https://ex.com/api
2	login	string	да	мин: 4, макс: 64, уник	Логин	test
3	email	string	нет	email, макс: 255	Электронная почта	example@ex.ru
4	password	string	да	мин: 8, макс: 255	Пароль	Test123!
5	password_confirmation	string	да	мин: 8, макс: 255, совпадает с password	Подтверждение пароля	Test123!

АУТЕНТИФИКАЦИЯ

Запрос позволяет проверить подлинность пользователя и получить токен доступа.

Request	Response
URL: {api}/login Method: POST Content-Type: application/json Body: { "identifier": {identifier}, "password": {password} }	Status: 200 OK Content-Type: application/json Body: { "user": <модель_пользователя>, "token": <токен> } Возможные статус-коды: 200, 401, 422

Входные параметры:

№	Название	Тип	Обяз	Описание	Пример
1	identifier	string	да	Логин или электронная почта	mylogin или myemail@ex.tu
2	password	string	да	Пароль	Pass123!

ВЫХОД

Запрос предназначен для удаления текущего токена пользователя.

Request	Response
URL: {api}/logout Method: POST Authorization: Bearer {token}	Status: 204 No Content Возможные статус-коды: 204, 401

Входные параметры:

№	Название	Тип	Обяз	Описание	Пример
1	token	string	да	API токен	2 bRAnLwqE78gsX4Iwj2YjatONWQd1312cc5f

Восстановление аккаунта

ВОССТАНОВЛЕНИЕ С ПОМОЩЬЮ КЛЮЧА

Запрос предназначен для восстановления пароля пользователя с помощью ключа, выданного при регистрации.

Request	Response
URL: {api}/restore-from-key Method: POST Content-Type: application/json Body: { "login": {login}, "key": {key}, "new_password": {new_password}, "new_password_confirmation": {new_password_confirmation}, "logout": {logout} }	Status: 204 No Content Возможные статус-коды: 204, 401, 422

Входные параметры:

№	Название	Тип	Обяз	Условия	Описание	Пример
1	login	string	да		Логин	MyLogin
2	key	integer	да	мин: 6, макс: 6	Ключ	666666
3	new_password	string	да	мин: 6, макс: 255	Новый пароль	Test123!
4	new_password_confirmation	string	да	мин: 6, макс: 255, совпадает с new_password_confirmation	Подтверждение нового пароля	Test123!
5	logout	boolean	нет		Удаление всех токенов, 0 – нет 1 – да	1

ОТПРАВЛЕНИЕ КОДА НА ПОЧТУ

Запрос предназначен для отправления кода на почту для дальнейшего восстановления пароля пользователя через почту.

Request	Response
URL: {api}/mail/send	Status: 204 No Content

Method: POST Content-Type: application/json Body: { "email": {email} }	Возможные статус-коды: 204, 422, 429
---	--

Входные параметры:

№	Название	Тип	Обяз	Условия	Описание	Пример
1	email	string	да	пользователь с таким email существует	Электронная почта	email@email.ex

ПОДТВЕРЖДЕНИЕ КОДА

Запрос предназначен для подтверждения кода с получением токена для восстановления пароля пользователя через почту.

Request	Response
URL: {api}/mail/verify Method: POST Content-Type: application/json Body: { "email": {email}, "code": {code}, }	Status: 200 OK Content-Type: application/json Body: { "reset_token": "string" } Возможные статус-коды: 200, 403, 422

Входные параметры:

№	Название	Тип	Обяз	Условия	Описание	Пример
1	email	string	да	Пользователь с таким email существует	Электронная почта	email@email.ex
2	code	string	да	Код действителен	Код, присланный на электронную почту	UYI5XI

ВОССТАНОВЛЕНИЕ ЧЕРЕЗ ЭЛЕКТРОННУЮ ПОЧТУ

Запрос предназначен для восстановления пароля пользователя через электронную почту, используя полученный токен восстановления.

Request	Response
URL: {api}/mail/restore Method: POST Content-Type: application/json Body: { "email": {email}, "reset_token": {reset_token}, "new_password": {new_password}, "new_password_confirmation": {new_password_confirmation}, "logout": {logout} }	Status: 204 No Content Возможные статус-коды: 200, 403, 422

Входные параметры:

№	Название	Тип	Обяз	Условия	Описание	Пример
1	email	string	да	Пользователь с таким email существует	Электронная почта	exa@ex.ex
2	reset_token	string	да		Токен восстановления	fgxvdfgwr12Wdfd3

3	new_password	string	да	мин: 6, макс: 255	Новый пароль	Test123!
4	new_password_confirmation	string	да	мин: 6, макс: 255, совпадает с new_password_confirmation	Подтверждение нового пароля	Test123!
5	logout	boolean	нет		Удаление всех токенов, 0 – нет 1 – да	1

Работа с профилем

ПОЛУЧЕНИЕ СВОЕГО ПРОФИЛЯ

Запрос на получение данных о себе.

Request	Response
URL: {api}/profile Method: GET Authorization: Bearer {token}	Status: 200 OK Content-Type: application/json Body: { "id": 3, "nickname": "Test", "header": "urlToImg", "avatar": "urlToImg", "login": "Test123", "email": "email@ex.ru" } Возможные статус-коды: 200, 401

ОБНОВЛЕНИЕ СВОЕГО ПРОФИЛЯ

Запрос на изменение данных о себе.

Request	Response
URL: {api}/profile Method: POST Authorization: Bearer {token} Content-Type: application/json Body: { "nickname": {nickname}, "email": {email}, "current_password": {current_password}, "new_password": {new_password}, "new_password_confirmation": {new_password_confirmation}, "header": {header}, "header_file": {header_file}, "avatar": {avatar}, "avatar_file": {avatar_file} } 	Status: 200 OK Content-Type: application/json Body: { "id": 3, "nickname": "Test", "header": "urlToImg", "avatar": "urlToImg", "login": "Test123", "email": "email@ex.ru" } Возможные статус-коды: 200, 401, 422

Входные параметры:

№	Название	Тип	Обяз	Условия	Описание	Пример
1	nickname	string	нет	мин: 2, макс: 64, уник	Имя пользователя	Testello2

2	email	string	нет	email, макс: 255, уник (исключая текущего пользователя)	Email пользователя	user@example.com
3	current_password	string	да, при наличии и new_password	изменение пароля на новый	Текущий пароль	OldPass123
4	new_password	string	нет	мин: 6, confirmed	Новый пароль	NewPass123
5	new_password_confirmation	string	нет	мин: 6	Подтверждение нового пароля	NewPass123
6	header	string	нет	url, regex: /\.?(jpe?g png gif svg)(\?.*)?\$/i	Шапка	https://example.com/header.jpg
7	header_file	file	нет	image, mimes: jpeg, png, jpg, gif, svg, max: 2048,	Файл шапки изображения	-
8	avatar	string	нет	url, regex: /\.?(jpe?g png gif svg)(\?.*)?\$/i	Аватар	https://example.com/avatar.jpg
9	avatar_file	file	нет	image, mimes: jpeg, png, jpg, gif, svg, max: 2048	Файл аватара	-

ПОЛУЧЕНИЕ СТАТИСТИКИ

Запрос на получение своей статистики.

Request	Response
URL: {api}/profile/statistic Method: GET Authorization: Bearer {token}	Status: 200 OK Content-Type: application/json Body: { "totalPlayed": 483, "recentlyPlayed": [{ "game": { model }, "time_played": 7 }, {}] } Возможные статус-коды: 200, 401

ПОЛУЧЕНИЕ ОБЛАЧНЫХ СЕРВИСОВ

Запрос на получение доступных и подключенных облачных сервисов.

Request	Response
---------	----------

URL: {api}/profile/services Method: GET Authorization: Bearer {token}	Status: 200 OK Content-Type: application/json Body: [<pre>{ "id": 1, "name": "Google Drive", "icon": "url", "description": "Google Drive API", "is_connected": false }</pre>] Возможные статус-коды: 200, 401
--	---

Работа с библиотекой

ПОЛУЧЕНИЕ СВОЕЙ БИБЛИОТЕКИ

Запрос на получение доступных и подключенных облачных сервисов.

Request	Response
URL: {api}/library?limit={limit}?page={page} Method: GET Authorization: Bearer {token}	Status: 200 OK Content-Type: application/json Body: { "data": [{ "id": 9, "last_played_at": "2025-05-18 00:00:30", "time_played": 266, "is_favorite": false, "sideGame": { model } }] } Возможные статус-коды: 200, 401

Входные параметры:

№	Название	Тип	Обяз	Условия	Описание	Пример
1	limit	integer	нет	≥ 0	Количество элементов на странице (0 — отключает пагинацию, >0 — ограничивает вывод указанным числом записей)	0
2	page	integer	нет	≥ 0	Номер страницы	2

ДОБАВЛЕНИЕ ИГРЫ В БИБЛИОТЕКУ

Запрос для добавления игры в библиотеку.

Request	Response
URL: {api}/library/game/{gameID} Method: POST Authorization: Bearer {token}	Status: 201 Created Content-Type: application/json Body: { "id": 10, "side_game_id": null, "last_played_at": null, "time_played": 0, "is_favorite": false, }

	<pre>"game": {model} }</pre> Возможные статус-коды: 201, 401, 404
--	--

Входные параметры:

№	Название	Тип	Обяз	Условия	Описание	Пример
1	gameID	integer	да	> 0	Уникальный идентификатор игры в системе	2

ПЕРЕКЛЮЧЕНИЕ СТАТУСА ИЗБРАННОГО ДЛЯ ИГРЫ

Запрос для переключения статуса избранного для игры. Если игра не в избранном → добавляется в избранное. Если игра уже в избранном → удаляется из избранного.

Request	Response
URL: {api}/library/game/{gameID} Method: PATCH Authorization: Bearer {token}	Status: 204 No Content Возможные статус-коды: 204, 401, 404

ОБНОВЛЕНИЕ ДАННЫХ ИГРЫ В БИБЛИОТЕКЕ

Запрос для обновления данных игры в библиотеке.

Request	Response
URL: {api}/library/game/{gameID}/update Method: PATCH Authorization: Bearer {token}	Status: 204 No Content Возможные статус-коды: 204, 401, 404, 422

Входные параметры:

№	Название	Тип	Обяз	Условия	Описание	Пример
1	time_played	integer	нет	> 0	Сколько было сыграно в секундах	454536
2	last_played_at	date	нет	Формат Y-m-d H:i:s	Дата последнего запуска	2025-02-28 16:01:45

УДАЛЕНИЕ ИГРЫ ИЗ БИБЛИОТЕКИ

Запрос для удаления игры из библиотеки, сохранения при этом останутся в БД.

Request	Response
URL: {api}/library/game/{gameID} Method: DELETE Authorization: Bearer {token}	Status: 204 No Content Возможные статус-коды: 204, 401, 404

ПЕРЕКЛЮЧЕНИЕ СТАТУСА ИЗБРАННОГО ДЛЯ СТОРОННЕЙ ИГРЫ

Запрос для переключения статуса избранного для сторонней игры. Если игра не в избранном → добавляется в избранное. Если игра уже в избранном → удаляется из избранного.

Request	Response
URL: {api}/library/sidegame/{sidegameID} Method: PATCH Authorization: Bearer {token}	Status: 204 No Content Возможные статус-коды: 204, 401, 404

Входные параметры:

№	Название	Тип	Обяз	Условия	Описание	Пример
---	----------	-----	------	---------	----------	--------

1	sidegameID	integer	да	> 0	Уникальный идентификатор сторонней игры в системе	1
---	------------	---------	----	-----	--	---

ОБНОВЛЕНИЕ ДАННЫХ СТОРОННЕЙ ИГРЫ В БИБЛИОТЕКЕ

Запрос для обновления данных игры в библиотеке.

Request	Response
URL: {api}/library/sidegame/{sidegameID}/update Method: PATCH Authorization: Bearer {token}	Status: 204 No Content Возможные статус-коды: 204, 401, 404, 422

Входные параметры:

№	Название	Тип	Обяз	Условия	Описание	Пример
1	time_played	integer	нет	> 0	Сколько было сыграно в секундах	454536
2	last_played_at	date	нет	Формат Y-m-d H:i:s	Дата последнего запуска	2025-02-28 16:01:45

Работа с играми

ПОЛУЧЕНИЕ СПИСКА ИГР

Запрос на получение списка игр.

Request	Response
URL: {api}/games?name={name}?page={page} Method: GET Authorization: Bearer {token}	Status: 200 OK Content-Type: application/json Body: { "data": [{ <модель игры> },], "links" : {<ссылки_пагинации>}, "meta": {<мета_пагинации>} } Возможные статус-коды: 200, 401

Входные параметры:

№	Название	Тип	Обяз	Условия	Описание	Пример
1	name	string	нет	-	Поиск по частичному совпадению названия	test
2	page	integer	нет	≥ 0	Номер страницы	2

ПОЛУЧЕНИЕ ИГРЫ

Запрос возвращает полную информацию об игре. Дополнительно включает:

- Данные из библиотеки (если игра добавлена).
- Список сохранений (если имеются).

Request	Response
---------	----------

URL: {api}/games/{gameID} Method: GET Authorization: Bearer {token}	Status: 200 OK Content-Type: application/json Body: { "game": { <модель_игры> }, "library": null, "saves": null } Возможные статус-коды: 200, 401, 404
--	---

ПОЛУЧЕНИЕ ПУТИ ДО СОХРАНЕНИЙ ИГРЫ

Запрос возвращает путь до папки сохранений игры.

Request	Response
URL: {api}/games/{gameID}/path Method: GET Authorization: Bearer {token}	Status: 200 OK Content-Type: application/json Body: { "path": "%LOCALAPPDATA%\CD Projekt Red\Cyberpunk 2077" } Возможные статус-коды: 200, 401, 404

Работа с сторонними играми

ДОБАВЛЕНИЕ СТОРОННЕЙ ИГРЫ

Запрос на добавление сторонней игры в базу данных и сразу в библиотеку.

Request	Response
URL: {api}/sidegames Method: POST Authorization: Bearer {token} Content-Type: application/json Body: { "name": {name} }	Status: 201 Created Content-Type: application/json Body: { "id": 14, "game_id": null, "last_played_at": null, "time_played": 0, "is_favorite": false, "sideGame": { "id": 5, "name": "test5" } } Возможные статус-коды: 201, 401, 422

Входные параметры:

№	Название	Тип	Обяз	Условия	Описание	Пример
1	name	string	да	макс: 255	Название игры	test5

ПОЛУЧЕНИЕ СТОРОННЕЙ ИГРЫ

Запрос на получение информации о сторонней игре, включая данные библиотеки и сохранений.

Request	Response
---------	----------

URL: {api}/sidegames/{sidegameID} Method: GET Authorization: Bearer {token}	Status: 200 OK Content-Type: application/json Body: { "side_game": { "id": 2, "name": "test2" }, "library": { "id": 11, "game_id": null, "side_game_id": 2, "last_played_at": null, "time_played": 0, "is_favorite": false }, "saves": null } Возможные статус-коды: 200, 401, 404
--	---

Входные параметры:

№	Название	Тип	Обяз	Условия	Описание	Пример
1	sidegameID	integer	да	> 0	Уникальный идентификатор сторонней игры в системе	2

УДАЛЕНИЕ СТОРОННЕЙ ИГРЫ

Запрос на удаление сторонней игры. Удаляет все связанные данные в БД, включая сохранения.

Request	Response
URL: {api}/sidegames/{sidegameID} Method: DELETE Authorization: Bearer {token}	Status: 204 No Content Возможные статус-коды: 204, 401, 404

Работа с сохранениями

ПОЛУЧЕНИЕ СВОИХ СОХРАНЕНИЙ К ИГРЕ

Запрос на получение списка своих сохранений к сторонней игре.

Request	Response
URL: {api}/saves/game/{gameID}/my Method: GET Authorization: Bearer {token}	Status: 200 OK Content-Type: application/json Body: { "saves": [{ <saveModel> }, {}] } Возможные статус-коды: 200, 401, 404

Входные параметры:

№	Название	Тип	Обяз	Условия	Описание	Пример
1	gameID	integer	да	> 0	Уникальный идентификатор игры в системе	2

ПОЛУЧЕНИЕ СВОИХ СОХРАНЕНИЙ К СТОРОННЕЙ ИГРЕ

Запрос на получение списка своих сохранений к сторонней игре.

Request	Response
URL: {api}/saves/sidegame/{sidegameID}/my Method: GET Authorization: Bearer {token}	Status: 200 OK Content-Type: application/json Body: { "saves": [{ <saveModel> }, {},] } Возможные статус-коды: 200, 401, 404

Входные параметры:

№	Название	Тип	Обяз	Условия	Описание	Пример
1	sidegameID	integer	да	> 0	Уникальный идентификатор сторонней игры в системе	2

ПЕРЕЗАПИСЬ СОХРАНЕНИЯ

Запрос для редактирования данных о сохранении.

Request	Response
URL: {api}/saves/{saveId} Method: PATCH Authorization: Bearer {token}	Status: 200 OK Content-Type: application/json Body: { "saves": [{ <saveModel> }, {},] } Возможные статус-коды: 200, 401, 403, 404, 422

Входные параметры:

№	Название	Тип	Обяз	Условия	Описание	Пример
1	saveId	integer	да	> 0	Уникальный идентификатор сохранения в системе	1
2	version	string	нет	являться строкой	Новая версия сохранения	5.0
3	description	string	нет	являться строкой	Описание сохранения	Это описание
4	hash	string	нет	являться строкой	Хэш сохранения	34hj45bj234jh6b
5	last_sync_at	datetime	нет	являться датой	Дата последней синхронизации	2025-05-17 19:14:17

ЗАГРУЗКА СОХРАНЕНИЯ В GOOGLEDRIVE

Запрос предназначен для создания первичных данных о сохранении в БД с генерацией прямой ссылки для загрузки сохранения в GoogleDrive.

Request	Response
---------	----------

URL: {api}/save/google-drive/generate-upload-url Method: POST Authorization: Bearer {token} Content-Type: application/json Body: { "file_name": {file_name}, "file_size": {file_size}, "version": {version}, "description": {description}, "game_id": {game_id}, "side_game_id": {side_game_id}, }	Status: 200 OK Content-Type: application/json Body: { "upload_url": "url_from_google", "save_id": 31 } Возможные статус-коды: 200, 401, 422, 409
--	--

Входные параметры:

№	Название	Тип	Обяз	Условия	Описание	Пример
1	version	string	нет	являться строкой	Новая версия сохранения	5.0
2	description	string	нет	являться строкой	Описание сохранения	Это описание
3	hash	string	нет	являться строкой	Хэш сохранения	34hj45bj234jh6b
4	last_sync_at	datetime	нет	являться датой	Дата последней синхронизации	2025-05-17 19:14:17
5	game_id	integer	нет	> 0, обязательное, если в запросе нет side_game_id, game_id и side_game_id не могут быть одновременно в одном запросе	Идентификатор игры в системе	4
6	side_game_id	integer	нет	> 0, обязательное, если в запросе нет game_id, game_id и side_game_id не могут быть одновременно в одном запросе	Идентификатор сторонней игры в системе	3

ПОДТВЕРЖДЕНИЕ СИНХРОНИЗАЦИИ В GOOGLEDRIIVE

Запрос предназначен для подтверждения синхронизации сохранения в GoogleDrive. Он добавляет данные для первичного сохранения.

Request	Response
---------	----------

URL: {api}/saves/{saveId}/google-drive/confirm-upload Method: POST Authorization: Bearer {token} Content-Type: application/json Body: { "file_id": {file_id}, "file_hash": {file_hash}, }	Status: 200 OK Content-Type: application/json Body: { <save_model> } Возможные статус-коды: 200, 401, 403, 404, 422
---	--

Входные параметры:

№	Название	Тип	Обяз	Условия	Описание	Пример
1	saveId	integer	да	> 0	Уникальный идентификатор сохранения в системе	1
2	file_id	string	да	являться строкой	Идентификатор файла в Google Drive	1XK9k4jWH7rruW1
3	file_hash	string	да	являться строкой	Хэш файла	9bb115c5c0

СКАЧИВАНИЕ СОХРАНЕНИЯ

Запрос предназначен для скачивания сохранения из GoogleDrive.

Request	Response
URL: {api}/saves/{saveId}/google-drive/download Method: GET Authorization: Bearer {token}	Status: 200 OK Content-Type: application/json Body: { file } Возможные статус-коды: 200, 401, 403, 404

Входные параметры:

№	Название	Тип	Обяз	Условия	Описание	Пример
1	saveId	integer	да	> 0	Уникальный идентификатор сохранения в системе	1

ПОДЕЛИТЬСЯ СОХРАНЕНИЕМ

Запрос предназначен для создания ссылки к сохранению, который располагается в GoogleDrive.

Request	Response
---------	----------

URL: {api}/save/{saveId}/google-drive/share Method: GET Authorization: Bearer {token}	Status: 200 OK Content-Type: application/json Body: { "url": "url" } Возможные статус-коды: 200, 401, 403, 404
--	---

Входные параметры:

№	Название	Тип	Обяз	Условия	Описание	Пример
1	saveId	integer	да	> 0	Уникальный идентификатор сохранения в системе	1

УДАЛЕНИЕ СОХРАНЕНИЯ

Запрос предназначен для удаления сохранения из БД и GoogleDrive.

Request	Response
URL: {api}/save/{saveId}/google-drive/delete Method: DELETE Authorization: Bearer {token}	Status: 204 No Content Возможные статус-коды: 204, 401, 403, 404

Входные параметры:

№	Название	Тип	Обяз	Условия	Описание	Пример
1	saveId	integer	да	> 0	Уникальный идентификатор сохранения в системе	1

ПЕРЕЗАПИСЬ СОХРАНЕНИЯ В GOOGLEDRIIVE

Запрос предназначен для генерации прямой ссылки на замену существующего сохранения в GoogleDrive.

Request	Response
URL: {api}/save/{saveId}/google-drive/generate-overwrite-url Method: POST Authorization: Bearer {token} Content-Type: application/json Body: { "file_name": {file_name}, "file_size": {file_size}, }	Status: 200 OK Content-Type: application/json Body: { "url": "url" } Возможные статус-коды: 200, 401, 403, 404, 422

Входные параметры:

№	Название	Тип	Обяз	Условия	Описание	Пример
1	saveId	integer	да	> 0	Уникальный идентификатор сохранения в системе	1

2	file_name	string	да	являться строкой	Название файла	NameGame_Save
3	file_size	integer	да	> 0	Размер файла в байтах	3845792

Работа с GoogleDrive

АВТОРИЗАЦИЯ GOOGLEDRIIVE

Запрос предназначен для генерации ссылки на авторизацию в GoogleDrive.

Request	Response
URL: {api}/google-drive/auth-url Method: GET Authorization: Bearer {token}	Status: 200 OK Content-Type: application/json Body: { "success": true, "url": "big_url" } Возможные статус-коды: 200, 401

ОТКЛЮЧЕНИЕ GOOGLEDRIIVE

Запрос предназначен для деактивации текущего аккаунта google drive в системе (очищает токены доступа и делает статус подключенного облачного сервиса неактивным).

Request	Response
URL: {api}/google-drive/disconnect/{userCloudServiceID} Method: DELETE Authorization: Bearer {token}	Status: 204 No Content Возможные статус-коды: 204, 401, 403, 404

Входные параметры:

№	Название	Тип	Обяз	Условия	Описание	Пример
1	userCloudServiceId	integer	да	> 0	Уникальный идентификатор подключенного облачного сервиса в системе	1

Маршрутизация

В листинге 1 представлена API-маршрутизация, разработанная в соответствии с техническим заданием. Маршруты сгруппированы по функционалу: аутентификация, работа с профилем, почтой, библиотекой, сохранениями и интеграцией с Google Drive. Все маршруты определены в файле routes/api.php и автоматически получают префикс /api, так как вложены в группу маршрутов через RouteServiceProvider. Для защиты эндпоинтов используется посредник auth:sanctum, обеспечивающий аутентификацию через Laravel Sanctum.

Листинг 1 – Маршрутизация по API /routes/api.php

```
Route::controller(AuthController::class)->group(function () {
    // Регистрация
    Route::post('register', 'signUp');
    // Аутентификация
    Route::post('login', 'signIn');
    // Восстановление с помощью ключа
    Route::post('restore-from-key', 'restoreFromKey');
    // Выход
    Route::middleware('auth:sanctum')->get('logout', 'logout');
});
```

```

Route::controller(MailController::class)->group(function () {
    Route::prefix('mail')->group(function () {
        // Отправка кода на почту
        Route::post('send', 'sendResetCode');
        // Подтверждение кода
        Route::post('verify', 'verifyCode');
        // Восстановление через почту
        Route::post('restore', 'restoreFromMail');
    });
});

Route::middleware('auth:sanctum')->group(function () {
    Route::controller(UserController::class)->group(function () {
        Route::prefix('profile')->group(function () {
            // Получение статистики
            Route::get('statistic', 'getStatistic');
            // Получение своего профиля
            Route::get('', 'getProfile');
            // Получение облачных сервисов
            Route::get('services', 'getCloudServices');
            // Обновление своего профиля
            Route::post('', 'updateProfile');
        });
    });
});

Route::controller(LibraryController::class)->group(function () {
    Route::prefix('library')->group(function () {
        // Получение своей библиотеки
        Route::get('', 'getLibrary');
        Route::prefix('game/{game}')->group(function () {
            // Добавление игры в библиотеку
            Route::post('', 'addToLibrary');
            // Переключение статуса избранного
            Route::patch('', 'toggleFavorite');
            // Удаление игры из библиотеки
            Route::delete('', 'removeFromLibrary');
            // Изменить данные игры в библиотеке
            Route::patch('update', 'updateLibraryGame');
        });
        Route::prefix('sidegame/{sideGame}')->group(function () {
            // Добавить/убрать стороннюю игру в Избранное
            Route::patch('', 'toggleSideGameFavorite');
            // Изменить данные сторонней игры в библиотеке
            Route::patch('update', 'updateSideGameLibrary');
        });
    });
});

Route::controller(SaveController::class)->group(function () {
    Route::prefix('saves')->group(function () {
        // Получение своих сохранений к игре
        Route::get('game/{game}/my', 'getMySavesGame');
        // Получение своих сохранений к сторонней игре
        Route::get('sidegame/{sideGame}/my', 'getMySavesSideGame');
        // Загрузка сохранения в GoogleDrive
        Route::post('google-drive/generate-upload-url',
            [GoogleDriveController::class, 'generateUploadUrl']);
        Route::prefix('{save}')->group(function () {
            // Обновление данных сохранения
            Route::patch('', 'updateSave');
            Route::controller(GoogleDriveController::class)
                ->group(function () {
                    // Действия с Google Drive
                    Route::prefix('google-drive')->group(function () {
                        // Подтверждение загрузки сохранения
                        Route::post('confirm-upload', 'confirmUpload');
                        // Перезапись сохранения
                        Route::post('generate-overwrite-url', 'generateOverwriteUrl');
                        // Скачивание сохранения
                        Route::get('download', 'downloadFile');
                        // Поделиться сохранением

```

```

        Route::get('share', 'shareFile');
        // Удаление сохранения
        Route::delete('delete', 'deleteFile');
    });
});
});
});
Route::controller GameController::class->group(function () {
    Route::prefix('games')->group(function () {
        // Просмотр всех игр
        Route::get('', 'getGames');
        Route::prefix('{game}')->group(function () {
            // Просмотр игры
            Route::get('', 'getGame');
            // Просмотр пути до сохранений
            Route::get('path', 'getPath');
        });
    });
});
Route::controller SideGameController::class->group(function () {
    Route::prefix('sidegames')->group(function () {
        // Добавление сторонней игры
        Route::post('', 'addSideGame');
        Route::prefix('{sideGame}')->group(function () {
            // Просмотр сторонней игры
            Route::get('', 'getSideGame');
            // Удаление сторонней игры
            Route::delete('', 'removeSideGame');
        });
    });
});
Route::controller GoogleDriveController::class->group(function () {
    Route::prefix('google-drive')->group(function () {
        // Авторизация в GoogleDrive
        Route::get('auth-url', 'getAuthUrl');
        // Ответ от GoogleDrive
        Route::get('callback', 'callback')
            ->withoutMiddleware('auth:sanctum');
        // Отключение сервиса
        Route::delete('disconnect/{userCloudService}', 'disconnect');
    });
});
});
});

```

Миграции

Миграции — это система управления структурой базы данных через код. Фасад Schema обеспечивает независимую от базы данных поддержку для создания и управления таблицами во всех поддерживаемых Laravel системах баз данных. В обычной ситуации, этот фасад используется для создания и изменения таблиц / столбцов базы данных во время миграции.

Первым делом создаётся основная таблица для хранения данных пользователей. В листинге 2 представлен код создания таблицы users, которая содержит поля для аутентификации, профиля и системные метки. Уникальные ограничения наложены на login, email, nickname.

Листинг 2 – Создание таблицы пользователей

```

Schema::create('users', function (Blueprint $table) {
    $table->id();
    $table->string('login')->unique();
    $table->string('email')->nullable()->unique();
    $table->string('nickname')->nullable()->unique();

```

```

$table->string('avatar')->nullable();
$table->string('header')->nullable();
$table->string('password');
$table->string('key');
$table->timestamps();
});

```

В листинге 3 представлен код создания таблицы `side_games`, которая хранит сторонние игры, привязанные к пользователям. Таблица реализует связь «один-ко-многим» с таблицей `users` через внешний ключ `user_id` и предотвращает дублирование записей с помощью составного уникального ключа (`user_id`, `name`), гарантируя, что у одного пользователя не может быть нескольких игр с одинаковым названием.

Листинг 3 – Создание таблицы сторонних игр

```

Schema::create('side_games', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->foreignId('user_id')->constrained('users');
    $table->unique(['user_id', 'name']);
    $table->timestamps();
});

```

В листинге 4 представлен код создания таблицы `games`, предназначенной для хранения информации об играх. Таблица обеспечивает уникальность названия игры через ограничение `unique()` на поле `name`, а также исключает дублирование комбинации `game_code` и `platform` с помощью составного уникального ключа.

Листинг 4 – Создание таблицы игр

```

Schema::create('games', function (Blueprint $table) {
    $table->id();
    $table->string('name')->unique();
    $table->string('game_code');
    $table->tinyInteger('platform');
    $table->unique(['game_code', 'platform']);
    $table->timestamps();
});

```

В листинге 5 представлен код создания таблицы `cloud_services`, которая хранит данные облачных сервисов для хранения файлов. Таблица включает уникальное название сервиса (`name`), его описание (`description`) и путь к иконке (`icon`), обеспечивая уникальность каждого сервиса через ограничение `unique()` на поле `name`.

Листинг 5 - Создание таблицы облачных сервисов

```

Schema::create('cloud_services', function (Blueprint $table) {
    $table->id();
    $table->string('name')->unique();
    $table->text('description')->nullable();
    $table->string('icon');
    $table->timestamps();
});

```

В листинге 6 представлен код создания таблицы `user_cloud_services`, предназначенной для связи пользователей с облачными сервисами. Таблица хранит токены доступа (`access_token`, `refresh_token`), время истечения срока действия токена (`expires_at`), статус подключения (`status`) и идентификатор пользователя во внешней системе (`external_user_id`). Для предотвращения дублирования используется составной уникальный ключ (`cloud_service_id`, `external_user_id`), гарантирующий, что один пользователь не может быть привязан к одному сервису несколько раз. Связи с таблицами `cloud_services` и `users` реализованы через внешние ключи.

Листинг 6 - Создание привязанных облачных сервисов

```
Schema::create('user_cloud_services', function (Blueprint $table) {
    $table->id();
    $table->text('access_token')->nullable();
    $table->text('refresh_token')->nullable();
    $table->timestamp('expires_at')->nullable();
    $table->foreignId('cloud_service_id')
        ->constrained('cloud_services', 'id');
    $table->foreignId('user_id')->constrained();
    $table->tinyInteger('status');
    $table->string('external_user_id');
    $table->unique(['cloud_service_id', 'external_user_id']);
    $table->timestamps();
});
```

В листинге 7 представлен код создания таблицы `saves`, предназначенной для управления сохранениями игр. Таблица поддерживает привязку как к основным (`game_id`), так и к сторонним играм (`side_game_id`), а также к облачным сервисам (`user_cloud_service_id`). Для обеспечения контроля версионности используются две уникальные комбинации: (`user_id`, `game_id`, `version`) для основных игр и (`user_id`, `side_game_id`, `version`) для дополнительных, что исключает дублирование версий сохранений у одного пользователя. Таблица также хранит метаданные файлов: идентификатор (`file_id`), имя (`file_name`), размер (`size`), хеш-сумму (`hash`), описание (`description`) и время последней синхронизации (`last_sync_at`). Связи с таблицами `users`, `games`, `side_games` и `user_cloud_services` реализованы через внешние ключи, при этом удаление связанной дополнительной игры автоматически приводит к удалению сохранений благодаря `cascadeOnDelete()`.

Листинг 7 - Создание таблицы сохранений

```
Schema::create('saves', function (Blueprint $table) {
    $table->id();
    $table->string('file_id')->nullable();
    $table->string('file_name');
    $table->string('version');
    $table->unsignedBigInteger('size');
    $table->text('description')->nullable();
    $table->text('hash')->nullable();
    $table->dateTime('last_sync_at')->nullable();
    $table->foreignId('user_id')->constrained();
    $table->foreignId('game_id')->nullable()->constrained('games');
    $table->foreignId('side_game_id')->nullable()
        ->constrained('side_games', 'id')->cascadeOnDelete();
    $table->foreignId('user_cloud_service_id')->nullable()
        ->constrained('user_cloud_services', 'id');
    $table->unique(['user_id', 'game_id', 'version']);
    $table->unique(['user_id', 'side_game_id', 'version']);
    $table->timestamps();
});
```

В листинге 8 представлена стандартная таблица `Laravel personal_access_tokens` для API-аутентификации. Таблица предназначена для хранения персональных токенов доступа с возможностью ограничения прав через поле `abilities` и отслеживания их использования. Каждый токен включает уникальное значение (`token`), название (`name`), информацию о владельце через полиморфную связь (`tokenable`), время последнего использования (`last_used_at`) и срок действия (`expires_at`). Таблица обеспечивает безопасность за счет уникальности токенов и позволяет гибко управлять правами доступа через JSON-поле `abilities`.

Листинг 8 - Создание таблицы персональных токенов

```
Schema::create('personal_access_tokens', function (Blueprint $table) {
    $table->id();
    $table->morphs('tokenable');
```

```

$table->string('name');
$table->string('token', 64)->unique();
$table->text('abilities')->nullable();
$table->timestamp('last_used_at')->nullable();
$table->timestamp('expires_at')->nullable();
$table->timestamps();
});

```

В листинге 9 представлен код создания таблицы `libraries`, формирующей персональную библиотеку игр пользователя. Таблица обеспечивает хранение как основных (`game_id`), так и сторонних игр (`side_game_id`), предотвращая дублирование через уникальные составные ключи: (`user_id`, `game_id`) для основных игр и (`user_id`, `side_game_id`) для сторонних. В таблице учитывается игровая статистика: время последнего запуска (`last_played_at`), общее время в игре (`time_played`), а также флаг избранного (`is_favorite`). Связи с таблицами `users`, `games` и `side_games` реализованы через внешние ключи, при этом удаление связанной сторонней игры автоматически очищает соответствующие записи благодаря `cascadeOnDelete()`.

Листинг 9 - Создание таблицы библиотек

```

Schema::create('libraries', function (Blueprint $table) {
    $table->id();
    $table->foreignId('user_id')->constrained('users');
    $table->foreignId('game_id')->nullable()->constrained('games');
    $table->foreignId('side_game_id')->nullable()->constrained('side_games')->cascadeOnDelete();
    $table->unique(['user_id', 'game_id']);
    $table->unique(['user_id', 'side_game_id']);
    $table->dateTime('last_played_at')->nullable();
    $table->unsignedInteger('time_played')->nullable();
    $table->boolean('is_favorite');
    $table->timestamps();
});

```

В листинге 10 представлен код создания таблицы `paths`, предназначенной для хранения путей к папкам с сохранениями игр. Каждая запись содержит абсолютный путь (`path`) и жестко привязана к конкретной игре через внешний ключ `game_id`, что обеспечивает целостность данных и исключает возможность существования пути без связанной игры.

Листинг 10 - Создание таблицы путей

```

Schema::create('paths', function (Blueprint $table) {
    $table->id();
    $table->string('path');
    $table->foreignId('game_id')->constrained('games');
    $table->timestamps();
});

```

Заполнители

Заполнители (Seeders) в Laravel используются для заполнения базы данных начальными данными. Они позволяют автоматически добавлять тестовые или стандартные данные в таблицы, что упрощает процесс разработки и тестирования. Заполнители особенно полезны при работе с командами, так как обеспечивают согласованность данных на всех этапах разработки.

Фасад DB и модель Eloquent предоставляют удобные методы для вставки данных в таблицы. Заполнители обычно создаются в папке `database/seeders`, а основной класс для запуска всех заполнителей находится в файле `DatabaseSeeder.php`.

В листинге 11 представлен общий обязательный заполнитель, который находится в файле database/seeder/DatabaseSeeder.php. Этот файл используется для вызова других заполнителей и управления процессом заполнения базы данных.

Листинг 11 – Общий обязательный заполнитель database/seeder/DatabaseSeeder.php

```
class DatabaseSeeder extends Seeder
{
    public function run(): void
    {
        $this->call(CloudSeeder::class);
        $this->call(GameSeeder::class);
        $this->call(TestSeeder::class);
    }
}
```

В листинге 12 представлен заполнитель облачных сервисов, который нужен для корректной работы, так как он добавляет в базу данных облачные сервисы, которые будут в системе. В данном случае он добавляет лишь один Google Drive.

Листинг 12 – Заполнитель облачных сервисов CloudSeeder

```
class CloudSeeder extends Seeder
{
    public function run(): void
    {
        CloudService::query()->firstOrCreate([
            'name' => 'Google Drive',
            'description' => 'Google Drive API',
            'icon' => asset('assets/images/Google_Drive_icon.svg'),
        ]);
    }
}
```

В листинге 13 представлен заполнитель игр, предназначенный для загрузки данных из JSON-файла Steam в базу данных. Он фильтрует записи, исключая DLC, дополнения и неигровой контент по ключевым словам, разбивает обработку на чанки по 1000 записей для оптимизации производительности, проверяет уникальность игр по коду и названию, использует транзакции для обеспечения надежности, а также выводит прогресс выполнения. Это гарантирует целостность данных и логирование ошибок.

Листинг 13 – Заполнитель игр GameSeeder

```
class GameSeeder extends Seeder
{
    public function run(): void
    {
        $file = database_path("seeders/steam-games.json");
        $this->command->info("Начало обработки файла...");
        // Ключевые слова для исключения
        $excludeKeywords = ["DLC", "Expansion", "Pack", "Add-on", "Season Pass",
            "Soundtrack", "Soundtracks", "Player Profiles", "Film", "Demo"];
        $excludePattern = '/\b(' . implode('|', array_map('preg_quote', $excludeKeywords)) .
            ')\b/i';

        try {
            // Чтение содержимого файла
            $jsonContent = file_get_contents($file);
            $data = json_decode($jsonContent, true);
            if (json_last_error() !== JSON_ERROR_NONE) {
                $this->command->error("Ошибка при декодировании JSON: " . json_last_error_msg());
                return;
            }
            $apps = $data;
            $totalApps = count($apps);
            $this->command->info("Всего записей для обработки: {$totalApps}");
        }
```

```

$this->command->getOutput()->progressStart($totalApps);
// Обрабатываем данные порциями (chunks)
$chunkSize = 1000; // Размер порции
$chunks = array_chunk($apps, $chunkSize);
foreach ($chunks as $chunk) {
    DB::beginTransaction();
    try {
        foreach ($chunk as $appData) {
            $gameSteamId = $appData['id'];
            $gameName = $appData['name'];
            // Пропускаем записи с пустым именем
            if (empty($gameName)) {
                continue;
            }
            // Проверяем, содержит ли название ключевые слова для исключения
            if (preg_match($excludePattern, $gameName)) {
                continue;
            }
            // Проверяем, существует ли запись с таким game_code и platform
            $existingGame = Game::where('game_code', $gameSteamId)
                ->where('platform', 1)
                ->first();
            // Если запись не существует, создаем новую
            if (!$existingGame) {
                // Проверяем, существует ли запись с таким именем
                $existingName = Game::where('name', $gameName)->exists();
                if (!$existingName) {
                    Game::create([
                        'name' => $gameName,
                        'game_code' => $gameSteamId,
                        'platform' => GamePlatform::Steam,
                    ]);
                }
            }
            DB::commit();
        } catch (\Exception $e) {
            DB::rollBack();
            Log::error("Ошибка при обработке порции данных: " . $e->getMessage());
            $this->command->error("Ошибка при обработке порции данных: " . $e-
>getMessage());
        }
        $this->command->getOutput()->progressAdvance(count($chunk));
    }
    $this->command->getOutput()->progressFinish();
    $this->command->info("Обработка завершена!");
} catch (\Exception $e) {
    $this->command->error("Произошла ошибка: " . $e->getMessage());
    Log::error("Ошибка в GameSeeder: " . $e->getMessage());
}
}
}

```

Модели

Модели в Laravel используются для работы с базой данных через Eloquent ORM. Они позволяют взаимодействовать с таблицами, определять связи между данными и управлять бизнес-логикой. Каждая модель представляет собой класс, связанный с определенной таблицей, а свойства и методы модели помогают упрощать работу с данными. В `protected $fillable` – описывается какие есть поля у таблицы.

В листинге 14 представлена модель `CloudService`, которая представляет облачный сервис в системе, обеспечивая связь с привязанными облачными сервисами пользователей

через отношение «один ко многим». Она предназначена для хранения основных данных о облачном сервисе.

Листинг 14 – Модель CloudService

```
class CloudService extends Model
{
    protected $fillable = [
        'name',
        'icon',
        'description'
    ];
    public function userCloudServices(): HasMany
    {
        return $this->hasMany(UserCloudService::class);
    }
}
```

В листинге 15 представлена модель Game, которая описывает игру в системе. Модель включает связь с сохранениями через отношение «один ко многим», а также взаимодействие с пользователями по принципу «многие ко многим». Кроме того, модель обеспечивает связь с путями до сохранения через отношение «один к одному».

Листинг 15 – Модель Game

```
class Game extends Model
{
    protected $fillable = [
        'name',
        'platform',
        'game_code'
    ];

    public function saves(): HasMany
    {
        return $this->hasMany(Save::class);
    }
    public function user(): BelongsToMany
    {
        return $this->belongsToMany(User::class);
    }
    public function path(): HasOne
    {
        return $this->hasOne(Path::class);
    }
}
```

В листинге 16 представлена модель Library, которая описывает игровую библиотеку пользователя в системе. Модель хранит информацию о времени игры, последнем запуске и статусе избранного, а также устанавливает связи с пользователем, игрой и сторонней игрой через отношения «принадлежит». Модель включает стандартные значения по умолчанию и преобразование типов для корректной работы с булевыми данными.

Листинг 16 – Модель Library

```
class Library extends Model
{
    protected $fillable = [
        'user_id',
        'game_id',
        'last_played_at',
        'time_played',
        'is_favorite',
        'side_game_id'
    ];
    protected $attributes = [
        'is_favorite' => false,
    ];
}
```

```

        'time_played' => 0,
    ];
    protected function casts(): array
    {
        return [
            'is_favorite' => 'boolean'
        ];
    }

    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }
    public function game(): BelongsTo
    {
        return $this->belongsTo(Game::class);
    }
    public function sideGame(): BelongsTo
    {
        return $this->belongsTo(SideGame::class);
    }
}

```

В листинге 17 представлена модель Path, которая хранит данные о путях до папки сохранения, связанных с конкретной игрой. Модель обеспечивает связь с игрой через отношение «принадлежит».

Листинг 17 – Модель Path

```

class Path extends Model
{
    protected $table = 'paths';
    protected $fillable = [
        'game_id',
        'path'
    ];
    public function game(): BelongsTo
    {
        return $this->belongsTo(Game::class);
    }
}

```

В листинге 18 представлена модель Save, которая описывает систему хранения игровых сохранений в облачных сервисах. Модель содержит метаданные файлов сохранений, включая версию, размер, дату последней синхронизации. Модель устанавливает связи с пользователем, игрой, стороной игрой и облачным сервисом через отношения типа «принадлежит», обеспечивая целостность данных в системе. Скрытое поле `file_id` защищает внутренний идентификатор файла от публичного доступа.

Листинг 18 – Модель Save

```

class Save extends Model
{
    protected $hidden = ['file_id'];
    protected $fillable = [
        'file_id',
        'file_name',
        'version',
        'size',
        'description',
        'user_id',
        'game_id',
        'side_game_id',
        'user_cloud_service_id',
        'last_sync_at',
        'hash',
    ];
}

```

```

public function user(): BelongsTo
{
    return $this->belongsTo(User::class);
}
public function game(): BelongsTo
{
    return $this->belongsTo(Game::class);
}
public function sideGame(): BelongsTo
{
    return $this->belongsTo(SideGame::class);
}
public function userCloudService(): BelongsTo
{
    return $this->belongsTo(UserCloudService::class);
}
}

```

В листинге 19 представлена модель SideGame, которая описывает сторонние игры, добавляемые пользователями. Модель поддерживает связи с создателем через отношение «принадлежит», а также взаимодействует с игровыми библиотеками и сохранениями через отношения «один ко многим».

Листинг 19 – Модель SideGame

```

class SideGame extends Model
{
    protected $fillable = [
        'name',
        'user_id'
    ];
    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }
    public function libraries(): HasMany
    {
        return $this->hasMany(Library::class);
    }
    public function saves(): HasMany
    {
        return $this->hasMany(Save::class);
    }
}

```

В листинге 20 представлена модель User, которая описывает учетные записи пользователей системы с функционалом аутентификации и уведомлений. Модель включает защищенные поля для хранения конфиденциальных данных (пароль и ключ доступа) с автоматическим хешированием, а также поддерживает обработку изображений профиля. Модель устанавливает связи с игровыми сохранениями, облачными сервисами, библиотеками игр и сторонними играми через отношения «один ко многим», обеспечивая централизованное управление пользовательскими данными в системе.

Листинг 20 – Модель User

```

class User extends Authenticatable
{
    use HasFactory, Notifiable, HasApiTokens;
    protected $fillable = [
        'email',
        'password',
        'nickname',
        'avatar',
        'login',
        'header',
        'key',
    ];
}

```

```

];

protected $hidden = [
    'password',
    'remember_token',
    'key'
];

protected function casts(): array
{
    return [
        'password' => 'hashed',
        'key' => 'hashed'
    ];
}

public function getImage($value): ?string
{
    if ($value === null) {
        return null;
    }
    if(!Str::startsWith($value, 'http')){
        return asset(Storage::url($value));
    }
    return $value;
}

public function saves(): HasMany
{
    return $this->hasMany(Save::class);
}

public function userCloudService(): HasMany
{
    return $this->hasMany(UserCloudService::class);
}

public function libraries(): HasMany
{
    return $this->hasMany(Library::class);
}

public function sideGames(): HasMany
{
    return $this->hasMany(SideGame::class);
}
}

```

В листинге 21 представлена модель UserCloudService, которая описывает подключенные облачные сервисы пользователей. Модель хранит данные аутентификации (токены доступа, сроки действия) и статус подключения, устанавливая связи с пользователем и облачным сервисом через отношения «принадлежит». Модель автоматически устанавливает начальный статус подключения как неактивный и обеспечивает хранение игровых сохранений через отношение «один ко многим».

Листинг 21 – Модель UserCloudService

```

class UserCloudService extends Model
{
    protected $table = 'user_cloud_services';
    protected $fillable = [
        'access_token',
        'refresh_token',
        'expires_at',
        'cloud_service_id',
        'user_id',
        'status',
        'external_user_id',
    ];
    protected $casts = [
        'expires_at' => 'datetime',
    ];
    protected $attributes = [

```

```

        'status' => CloudStatus::Inactive,
    ];
    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }
    public function cloudService(): BelongsTo
    {
        return $this->belongsTo(CloudService::class);
    }
    public function saves(): HasMany
    {
        return $this->hasMany(Save::class);
    }
}

```

Сервисы

В Laravel сервисы представляют собой классы, инкапсулирующие бизнес-логику или интеграции со сторонними API. Они помогают разделять ответственность, упрощают тестирование и делают код более читаемым. Сервисы обычно размещаются в директории `app/Services` и могут внедряться в контроллеры, `jobs` или другие сервисы через `Dependency Injection`.

Листинг 22 демонстрирует класс `GoogleDriveService`, предназначенный для работы с Google Drive API. При инициализации сервис принимает объект `UserCloudService` (данные подключенного облачного сервиса пользователя) и настраивает клиент Google API с использованием токенов доступа. В конструкторе вызывается метод `initializeClient()`, который загружает `credentials` из `.env`, устанавливает токены и создает экземпляр сервиса Google Drive.

Листинг 22 – Конструктор и инициализация `GoogleDriveService`

```

class GoogleDriveService
{
    // Google API клиент
    private $client;
    // Сервис для работы с Google Drive
    private $driveService;
    // Данные пользовательского облачного сервиса
    private $userCloudService;
    // Кэш для хранения ID папок
    private array $folderCache;

    // Конструктор
    public function __construct($userCloudService)
    {
        $this->userCloudService = $userCloudService;
        $this->initializeClient();
    }

    // Инициализация клиента
    private function initializeClient(): void
    {
        // Настройка клиента Google API с использованием credentials из .env
        $this->client = new Client();
        $this->client->setClientId(env('GOOGLE_DRIVE_CLIENT_ID'));
        $this->client->setClientSecret(env('GOOGLE_DRIVE_CLIENT_SECRET'));
        $this->client->setAccessType('offline');
        $this->setAccessToken();
        $this->driveService = new Drive($this->client);
    }
}

```

В листинге 23 реализованы два метода для работы с OAuth-токенами Google API. Метод `setAccessToken()` расшифровывает сохранённые в базе данных `access_token` и `refresh_token`, устанавливает их в клиент Google API и проверяет срок действия `access_token` — если он истёк, вызывается `refreshToken()`. Метод `refreshToken()` обновляет `access_token` с помощью `refresh_token`, сохраняет новые данные (включая зашифрованный токен и время истечения) в базу данных и обновляет токен в клиенте. Если Google возвращает новый `refresh_token` (что происходит не всегда), он также сохраняется, иначе остаётся старый. Это обеспечивает бесперебойную аутентификацию при работе с Google Drive API.

Листинг 23 – Установка и обновление токена доступа в *GoogleDriveService*

```
// Установка токена доступа
private function setAccessToken(): void
{
    // Расшифровка токенов
    $accessToken = Crypt::decryptString($this->userCloudService->access_token);
    $refreshToken = Crypt::decryptString($this->userCloudService->refresh_token);

    // Установка токенов
    $this->client->setAccessToken([
        'access_token' => $accessToken,
        'refresh_token' => $refreshToken,
        'expires_in' => $this->userCloudService->expires_at->diffInSeconds(now()),
    ]);

    // Обновление токена при необходимости
    if ($this->client->isAccessTokenExpired()) {
        $this->refreshToken();
    }
}

// Обновление токена
private function refreshToken(): void
{
    // Обновление токена доступа с помощью refresh token
    $refreshToken = Crypt::decryptString($this->userCloudService->refresh_token);
    $newToken = $this->client->fetchAccessTokenWithRefreshToken($refreshToken);

    // Сохранение новых токенов
    $this->userCloudService->update([
        'access_token' => Crypt::encryptString($newToken['access_token']),
        'expires_at' => now()->addSeconds($newToken['expires_in']),
        'refresh_token' => isset($newToken['refresh_token'])
            ? Crypt::encryptString($newToken['refresh_token'])
            : $this->userCloudService->refresh_token,
    ]);

    $this->client->setAccessToken($newToken);
}
```

В листинге 24 реализована функция `generateResumableUploadUrl()`, которая формирует URL для докачки файлов в Google Drive: метод сначала создает структуру папок (если они отсутствуют) через `createFolderStructure()`, затем подготавливает метаданные файла (имя и родительскую папку) с помощью класса `DriveFile`. Далее инициализируется HTTP-клиент с авторизацией, после чего формируется запрос к Google Drive API с параметрами `uploadType=resumable` (для поддержки возобновляемой загрузки) и `supportsAllDrives=true` (для работы с Shared Drives). В случае успешного запроса из заголовка `Location` извлекается URL для загрузки, который возвращается вызывающему коду. Если URL не получен или возникает ошибка, выбрасывается исключение `ApiException` с детализацией проблемы, обеспечивая обработку сбоев на уровне интеграции с Google Drive.

Листинг 24 – Генерация URL для загрузки файлов в GoogleDriveService

```
// Генерация URL для загрузки файла с возможностью возобновления
public function generateResumableUploadUrl(string $fileName, string $folderPath): string
{
    try {
        $folderId = $this->createFolderStructure($folderPath);

        $fileMetadata = new DriveFile([
            'name' => $fileName,
            'parents' => [$folderId]
        ]);
        $httpClient = $this->driveService->getClient()->authorize();

        $uri = 'https://www.googleapis.com/upload/drive/v3/files?' . http_build_query([
            'uploadType' => 'resumable',
            'fields' => 'id',
            'supportsAllDrives' => 'true'
        ]);
        $request = new Request(
            'POST',
            $uri,
            [
                'Content-Type' => 'application/json',
                'X-Upload-Content-Type' => 'application/octet-stream'
            ],
            json_encode($fileMetadata)
        );
        $response = $httpClient->send($request);
        $location = $response->getHeaderLine('Location');
        if (empty($location)) {
            throw new ApiException('Google Drive did not return upload URL');
        }
        return $location;
    } catch (\Exception $e) {
        throw ApiException::fromException(
            $e,
            'Failed to generate upload URL: '
        );
    }
}
```

В листинге 25 реализована функция generateResumableOverwriteUrl(), которая генерирует URL для обновления существующего файла в Google Drive с поддержкой докачки: метод принимает ID файла и новое имя, создает метаданные с обновленным именем через класс DriveFile, формирует PATCH-запрос к Google Drive API с параметрами uploadType=resumable (для возобновляемой загрузки) и supportsAllDrives=true (для совместимости с Shared Drives), добавляя в заголовки авторизационный токен. После отправки запроса проверяется статус ответа и наличие заголовка Location, содержащего URL для загрузки. В случае ошибки выбрасывается исключение ApiException с детализацией, что обеспечивает надежную обработку сбоев при перезаписи файлов.

Листинг 25 – Перезапись существующего файла в GoogleDriveService

```
// Генерация URL для перезаписи существующего файла
public function generateResumableOverwriteUrl(string $fileId, string $fileName): string
{
    try {
        $client = $this->driveService->getClient();
        $httpClient = $client->authorize();

        // 1. Создаем метаданные файла
        $fileMetadata = new DriveFile([
            'name' => $fileName
        ]);
    }
}
```

```

        // 2. Формируем URL с параметрами
        $uri = 'https://www.googleapis.com/upload/drive/v3/files/' . urlencode($fileId) . '?' .
http_build_query([
            'uploadType' => 'resumable',
            'supportsAllDrives' => 'true'
        ]);

        // 3. Создаем PSR-7 запрос (PATCH для обновления)
        $request = new Request(
            'PATCH',
            $uri,
            [
                'Content-Type' => 'application/json',
                'X-Upload-Content-Type' => 'application/octet-stream',
                'Authorization' => 'Bearer ' . $client->getAccessToken()['access_token']
            ],
            json_encode($fileMetadata)
        );

        // 4. Отправляем запрос
        $response = $httpClient->send($request);

        // 5. Проверяем ответ
        if ($response->getStatusCode() != 200) {
            throw new ApiException('Invalid status code: ' . $response->getStatusCode());
        }

        $location = $response->getHeaderLine('Location');
        if (empty($location)) {
            throw new ApiException('Location header is missing');
        }
        return $location;
    } catch (\Exception $e) {
        throw ApiException::fromException(
            $e,
            'Failed to generate overwrite URL: '
        );
    }
}

```

В листинге 26 представлены методы для управления файлами в Google Drive:

1. `deleteFile()` удаляет файл по указанному `fileId`, используя API Google Drive, и обрабатывает возможные ошибки через исключение `ApiException`.
2. `shareFile()` предоставляет публичный доступ на чтение файла, создавая разрешение типа `anyone` с ролью `reader`, и возвращает публичную ссылку (`webViewLink`) для просмотра файла.
3. `downloadFile()` загружает содержимое файла по его ID, возвращая массив с данными: бинарное содержимое (`content`), MIME-тип и исходное имя файла, полученные через отдельные запросы к API. Все методы включают обработку исключений с преобразованием в `ApiException` для единообразного управления ошибками.

Листинг 26 – Управление файлами в `GoogleDriveService`

```

// Удаление файла по ID
public function deleteFile($fileId)
{
    try {
        $this->driveService->files->delete($fileId);
    } catch (\Exception $e) {
        throw ApiException::fromException($e, 'Failed to delete file: ');
    }
}

// Настройка доступа к файлу (публичный доступ для чтения)
public function shareFile($fileId)

```

```

{
    try {
        $permission = new Permission([
            'type' => 'anyone',
            'role' => 'reader',
        ]);

        $this->driveService->permissions->create($fileId, $permission);

        // Возвращает публичную ссылку на файл
        $file = $this->driveService->files->get($fileId, ['fields' => 'webViewLink']);

        return $file->getWebViewLink();
    } catch (\Exception $e) {
        throw ApiException::fromException(
            $e,
            'Failed to share file: '
        );
    }
}

// Скачивание файла
public function downloadFile($fileId): array
{
    try {
        $fileMetadata = $this->driveService->files->get($fileId, ['fields' => 'mimeType, name']);
        $fileContent = $this->driveService->files->get($fileId, ['alt' => 'media']);

        // Возвращает содержимое файла, MIME-тип и имя файла
        return [
            'content' => $fileContent->getBody(),
            'mimeType' => $fileMetadata->getMimeType(),
            'fileName' => $fileMetadata->getName(),
        ];
    } catch (\Exception $e) {
        throw ApiException::fromException(
            $e,
            'Failed to generate download file: '
        );
    }
}
}

```

В листинге 27 реализованы методы для работы с папками в Google Drive:

1. `getFileParentFolderId()` получает ID родительской папки файла через API Google Drive, возвращая первый элемент из списка `parents` (или `null`, если файл находится в корне).
2. `renameFolder()` переименовывает папку по её `folderId`, обновляя метаданные через `DriveFile` и возвращая объект с новым именем и ID.
3. `getAndRenameParentFolder()` комбинирует функциональность: сначала находит родительскую папку файла через `getFileParentFolderId()`, а затем переименовывает её с помощью `renameFolder()`, выбрасывая исключение `ApiException`, если файл находится в корне. Все методы поддерживают работу с Shared Drives (параметр `supportsAllDrives`) и обрабатывают ошибки через унифицированный `ApiException`.

Листинг 27 – Работа с папками в *GoogleDriveService*

```

// Получение ID родительской папки для файла
public function getFileParentFolderId(string $fileId): ?string
{
    try {
        $file = $this->driveService->files->get($fileId, [
            'fields' => 'parents',
            'supportsAllDrives' => true
        ]);
        if (empty($file->getParents())) {
            return null;
        }
    }
}

```

```

        // Возвращаем первую родительскую папку (файл может быть в нескольких папках)
        return $file->getParents()[0];
    } catch (\Exception $e) {
        throw ApiException::fromException(
            $e,
            'Failed to get parent folder: '
        );
    }
}

// Переименование папки
public function renameFolder(string $folderId, string $newName): DriveFile
{
    try {
        $folderMetadata = new DriveFile([
            'name' => $newName
        ]);

        return $this->driveService->files->update($folderId, $folderMetadata, [
            'fields' => 'id,name',
            'supportsAllDrives' => true
        ]);
    } catch (\Exception $e) {
        throw ApiException::fromException(
            $e,
            'Failed to rename folder: '
        );
    }
}

// Получение и переименование родительской папки файла
public function getAndRenameParentFolder(string $fileId, string $newFolderName): string
{
    $parentId = $this->getFileParentFolderId($fileId);
    if ($parentId === null) {
        throw new ApiException('The file is not in any folder (it is in the root)');
    }
    $this->renameFolder($parentId, $newFolderName);

    return $parentId;
}

```

В листинге 28 реализованы методы для рекурсивного создания папок в Google Drive с кэшированием результатов:

1. createFolderStructure() принимает путь вида folder1/folder2, разбивает его на части и последовательно создаёт или находит каждую папку, начиная с корня (root), используя кэш Laravel (cache()->rememberForever) для избежания повторных запросов к API.
2. getOrCreateFolder() проверяет существование папки по имени и родительскому ID через запрос к API (с экранированием спецсимволов), используя внутренний кэш (\$this->folderCache) для быстрого доступа. Если папка не найдена, создаёт её с указанным parentId и типом application/vnd.google-apps.folder, сохраняя результат в кэш. Оба метода поддерживают Shared Drives (supportsAllDrives) и оптимизируют работу с API за счёт двухуровневого кэширования (Laravel Cache + in-memory).

Листинг 28 – Создание папок в GoogleDriveService

```

// Создание структуры папок по заданному пути
private function createFolderStructure(string $path): string
{
    $cacheKey = 'gdrive_folder_' . md5($path);
    return cache()->rememberForever($cacheKey, function () use ($path) {
        $parts = explode('/', $path);
        $parentId = 'root';
        foreach ($parts as $folderName) {
            $parentId = $this->getOrCreateFolder($folderName, $parentId);
        }
        return $parentId;
    });
}

```

```

});
}
// Получение ID существующей папки или создание новой
public function getOrCreateFolder(string $folderName, string $parentId = 'root'): string
{
    $cacheKey = $parentId . ':' . $folderName;
    if (isset($this->folderCache[$cacheKey]))
        return $this->folderCache[$cacheKey];
    $escapedName = addslashes($folderName, "\\");
    $query = sprintf(
        "mimeType='application/vnd.google-apps.folder' and name='%s' and '%s' in parents and
        trashed=false",
        $escapedName,
        $parentId
    );
    $response = $this->driveService->files->listFiles([
        'q' => $query,
        'fields' => 'files(id)',
        'pageSize' => 1,
        'supportsAllDrives' => true,
    ]);
    if (count($response->getFiles()) > 0) {
        $folderId = $response->getFiles()[0]->getId();
        $this->folderCache[$cacheKey] = $folderId;
        return $folderId;
    }
    $folderMetadata = new DriveFile([
        'name' => $folderName,
        'mimeType' => 'application/vnd.google-apps.folder',
        'parents' => [$parentId],
    ]);
    $folder = $this->driveService->files->create($folderMetadata, [
        'fields' => 'id',
        'supportsAllDrives' => true,
    ]);
    $folderId = $folder->getId();
    $this->folderCache[$cacheKey] = $folderId;
    return $folderId;
}

```

Контроллеры

Вместо того чтобы определять всю логику обработки запросов как замыкания в файлах маршрутов, мы организуем это поведение с помощью классов «контроллеров». Контроллеры могут сгруппировать связанную логику обработки запросов в один класс.

Контроллеры в Laravel отвечают за обработку входящих запросов, выполнение бизнес-логики и возврат соответствующих ответов. Они часто используются для управления данными, взаимодействия с моделями и отображения информации через API или веб-страницы. Контроллеры могут включать методы для обработки различных HTTP-запросов, таких как GET, POST, PUT, DELETE.

В листинге 29 представлен контроллер аутентификации AuthController, реализующий базовые функции безопасности с использованием Laravel Sanctum:

1. Метод `signUp` обрабатывает регистрацию пользователя: генерирует 6-символьный резервный ключ, создает запись пользователя в БД и возвращает данные пользователя, JWT-токен и сгенерированный ключ с HTTP-статусом 201.
2. Метод `signIn` выполняет авторизацию: определяет тип идентификатора (email или login), проверяет учетные данные через `Auth::attempt`, генерирует новый токен доступа и возвращает данные пользователя вместе с токеном.

3. Метод `restoreFromKey` обеспечивает восстановление доступа через резервный ключ: проверяет его валидность через `Hash::check`, обновляет пароль пользователя и при необходимости завершает все активные сеансы.
4. Метод `logout` выполняет выход из системы: удаляет текущий токен доступа и возвращает пустой ответ с HTTP-статусом 204.

Листинг 29 – Контроллер аутентификации *AuthController*

```
class AuthController extends Controller
{
    // Регистрация
    public function signUp(SignUpRequest $request): JsonResponse
    {
        $key = Str::random(6);
        $user = User::query()->create([
            ...$request->validated(),
            'key' => $key,
            'nickname' => $request->input('login'),
        ]);
        $token = $user->createToken(Str::random(100))->plainTextToken;
        return response()->json(['user' => UserResource::make($user), 'token' => $token, 'key' =>
$key], 201);
    }

    // Авторизация
    public function signIn(SignInRequest $request): JsonResponse
    {
        $identifier = $request->input('identifier');
        $password = $request->input('password');
        $field = filter_var($identifier, FILTER_VALIDATE_EMAIL) ? 'email' : 'login';

        if (!Auth::attempt([$field => $identifier, 'password' => $password])) {
            throw new ApiException('Invalid credentials', 401);
        }
        $user = Auth::user();
        if (!$user instanceof User) {
            throw new UnauthorizedException();
        }

        $token = $user->createToken(Str::random(100))->plainTextToken;

        return response()->json([
            'user' => UserResource::make($user),
            'token' => $token,
        ]);
    }

    // Восстановление через ключ
    public function restoreFromKey(RestoreFromKeyRequest $request): JsonResponse
    {
        $user = User::query()->where('login', $request->login)->firstOrFail();
        if (!Hash::check($request->key, $user->key)) {
            throw new ApiException('Invalid credentials', 401);
        }
        $user->update(['password' => $request->new_password]);
        if($request->logout == true)
        {
            $user->tokens()->delete();
        }
        return response()->json(null, 204);
    }

    // Выход
    public function logout(Request $request): JsonResponse
    {
        $request->user()->currentAccessToken()->delete();
        return response()->json(null, 204);
    }
}
```

В листинге 30 представлен контроллер GameController, который обрабатывает запросы, связанные с играми в системе. Он предоставляет методы для поиска игр, получения информации о конкретной игре и её сохранениях, а также для работы с путями к файлам. Рассмотрим подробнее каждый из методов контроллера:

1. `getGames(Request $request)`: метод возвращает список игр с возможностью фильтрации по названию. Если передан параметр `name`, выполняется поиск игр, содержащих указанную строку. Результат возвращается в формате пагинации (по 20 записей на страницу) с использованием `GameResource` для преобразования данных.
2. `getGame(Game $game)`: метод возвращает детальную информацию об игре, включая данные из библиотеки пользователя (`Library`) и список сохранений (`saves`). Если игра не добавлена в библиотеку, поле `library` будет `null`. Аналогично, если сохранений нет, поле `saves` также вернётся как `null`.
3. `getPath(Game $game)`: метод возвращает путь к папке сохранений для указанной игры. Если путь не найден, генерируется исключение 404 Not Found. Результат содержит только поле `path` без дополнительных данных.

Листинг 30 – Контроллер игр GameController

```
class GameController extends Controller
{
    // Получение игр
    public function getGames(Request $request): JsonResponse
    {
        $name = $request->query('name');
        $games = Game::query()
            ->when($name, function ($query, $name) {
                return $query->where('name', 'like', '%' . $name . '%');
            })
            ->paginate(20)
            ->appends(['name' => $name]);
        return GameResource::collection($games)->response();
    }

    // Получение игры
    public function getGame(Game $game): JsonResponse
    {
        $user = auth()->user();
        $library = Library::query()->where('game_id', $game->id)->where('user_id', $user->id)->first();
        $saves = $user->saves()->where('game_id', $game->id)->get();
        return response()->json([
            'game' => GameResource::make($game),
            'library' => $library ? LibraryResource::make($library) : null,
            'saves' => $saves->isEmpty() ? null : SaveResource::collection($saves),
        ]);
    }

    // Получение пути к папке сохранений
    public function getPath(Game $game): JsonResponse
    {
        $path = $game->path()->firstOrFail();
        return response()->json($path->only('path'));
    }
}
```

В листинге 31 представлена функция `getSubFromIdToken(string $idToken)` в контроллере `GoogleDriveController`, которая представляет собой статический метод, который извлекает идентификатор пользователя (`sub`) из JWT-токена Google. Декодирует `payload` токена и возвращает `sub` или `null`, если токен невалиден.

Листинг 31 – Функция *getSubFromIdToken* в *GoogleDriveController*

```
class GoogleDriveController extends Controller
{
    public static function getSubFromIdToken(string $idToken): ?string
    {
        $parts = explode('.', $idToken);
        if (count($parts) < 2) {
            return null;
        }
        $payload = $parts[1];
        // Добавление padding для base64 (если не кратно 4)
        $payload .= str_repeat('=', 4 - strlen($payload) % 4);
        $json = base64_decode(strtr($payload, '-_', '+/'));
        if (!$json) {
            return null;
        }
        $data = json_decode($json, true);
        return $data['sub'] ?? null;
    }
}
```

В листинге 32 представлена функция *getAuthUrl(Request \$request)* в контроллере *GoogleDriveController*, которая генерирует URL для OAuth-авторизации в Google Drive. Устанавливает необходимые scope, создает случайный state (для защиты от CSRF) и сохраняет его в кэше. Возвращает URL для перенаправления пользователя на страницу авторизации Google.

Листинг 32 – Функция *getAuthUrl* в *GoogleDriveController*

```
public function getAuthUrl(Request $request): JsonResponse
{
    $user = $request->user();
    $client = new Client();
    $client->setClientId(env('GOOGLE_DRIVE_CLIENT_ID'));
    $client->setClientSecret(env('GOOGLE_DRIVE_CLIENT_SECRET'));
    $client->setRedirectUri(env('GOOGLE_DRIVE_REDIRECT_URI'));
    $client->addScope("openid");
    $client->addScope("https://www.googleapis.com/auth/drive.file");
    $client->setAccessType('offline');
    $client->setPrompt('consent');

    $state = bin2hex(random_bytes(16)) . '-' . $user->id;
    Cache::put("oauth_state:{$state}", $user->id, now()->addMinutes(10));
    $client->setState($state);

    return response()->json([
        'success' => true,
        'url' => $client->createAuthUrl()
    ]);
}
```

В листинге 33 представлена функция *callback(Request \$request)* в контроллере *GoogleDriveController*, которая обрабатывает ответ от Google OAuth после успешной авторизации. Получает access/refresh токены, сохраняет их в зашифрованном виде в БД и связывает с пользователем. Перенаправляет на страницу успешной аутентификации.

Листинг 33 – Функция *callback* в *GoogleDriveController*

```
public function callback(Request $request)
{
    $state = $request->state;
    $userId = Cache::pull("oauth_state:{$state}");

    $client = new Client();
    $client->setClientId(env('GOOGLE_DRIVE_CLIENT_ID'));
    $client->setClientSecret(env('GOOGLE_DRIVE_CLIENT_SECRET'));
    $client->setRedirectUri(env('GOOGLE_DRIVE_REDIRECT_URI'));
```

```

$token = $client->fetchAccessTokenWithAuthCode($request->get('code'));
$idToken = $token['id_token'] ?? null;
$externalUserId = $idToken ? GoogleDriveController::getSubFromIdToken($idToken) : null;

$cloudService = CloudService::query()->where('name', 'Google Drive')->first();

UserCloudService::query()->updateOrCreate(
    [
        'cloud_service_id' => $cloudService->id,
        'external_user_id' => $externalUserId,
    ],
    [
        'user_id' => $userId,
        'access_token' => Crypt::encryptString($token['access_token']),
        'refresh_token' => isset($token['refresh_token']) ?
Crypt::encryptString($token['refresh_token']) : null,
        'expires_at' => now()->addSeconds($token['expires_in']),
        'external_user_id' => $externalUserId,
        'status' => CloudStatus::Active
    ]
);

return redirect('auth/success');
}

```

В листинге 34 представлена функция generateUploadUrl(UploadSaveRequest \$request) в контроллере GoogleDriveController, которая создает URL для загрузки файла сохранения в Google Drive. Проверяет, существует ли уже сохранение с такой версией, и либо обновляет его, либо создает новую запись. Формирует путь к папке в формате PlaySaveBack/{game}/{version} и возвращает URL для резюмируемой загрузки.

Листинг 34 – Функция generateUploadUrl в GoogleDriveController

```

// Генерация URL для прямой загрузки файла в Google Drive
public function generateUploadUrl(UploadSaveRequest $request): JsonResponse
{
    $user = auth()->user();
    $game = $this->resolveGame($request);
    // Проверяем существующее сохранение
    $existingSave = $user->saves()
        ->where(function($q) use ($game) {
            $q->where('game_id', $game->id)
                ->orWhere('side_game_id', $game->id);
        })
        ->where('version', $request->version)
        ->first(); // Получаем запись, а не просто exists()
    if ($existingSave) {
        // Если у сохранения заполнены критические поля – конфликт
        if (
            !empty($existingSave->file_id) ||
            !empty($existingSave->hash) ||
            !empty($existingSave->last_sync_at) ||
            !empty($existingSave->user_cloud_service_id)
        ) {
            throw new ConflictException();
        }
        // Если поля пустые – обновляем запись и разрешаем загрузку
        $existingSave->update([
            'file_name' => $request->file_name,
            'size' => $request->file_size,
            'description' => $request->description,
        ]);
        $save = $existingSave;
    } else {
        // Создаем новую запись, если сохранения нет
        $saveData = [
            'version' => $request->version,

```

```

        'description' => $request->description,
        'user_id' => $user->id,
        'file_name' => $request->file_name,
        'size' => $request->file_size,
    ];
    $game instanceof Game
        ? $saveData['game_id'] = $game->id
        : $saveData['side_game_id'] = $game->id;
    $save = Save::create($saveData);
}
// Генерируем URL для загрузки
$cloudService = CloudService::where('name', 'Google Drive')->first();
$service = $user->userCloudService()
    ->where('cloud_service_id', $cloudService->id)
    ->where('status', CloudStatus::Active)
    ->firstOrFail();
$googleDriveService = new GoogleDriveService($service);
$folderPath = "PlaySaveBack/{ $game->name }/{ $request->version }";
$uploadUrl = $googleDriveService->generateResumableUploadUrl(
    fileName: $request->input('file_name'),
    folderPath: $folderPath
);
return response()->json([
    'upload_url' => $uploadUrl,
    'save_id' => $save->id,
]);
}

```

В листинге 35 представлена функция `confirmUpload(Save $save, ConfirmUploadSave $request)` в контроллере `GoogleDriveController`, которая подтверждает успешную загрузку файла. Обновляет запись сохранения, добавляя `file_id`, хеш файла и метку времени последней синхронизации. Возвращает обновленные данные сохранения.

Листинг 35 – Функция `confirmUpload` в `GoogleDriveController`

```

// Подтверждение успешной загрузки файла
public function confirmUpload(Save $save, ConfirmUploadSave $request): JsonResponse
{
    $user = auth()->user();
    if($save->user_id != $user->id)
        throw new ForbiddenException();
    $cloudService = CloudService::where('name', 'Google Drive')->first();
    $service = $user->userCloudService()
        ->where('cloud_service_id', $cloudService->id)
        ->where('status', CloudStatus::Active)
        ->firstOrFail();
    $save->update([
        'file_id' => $request->file_id,
        'hash' => $request->file_hash,
        'last_sync_at' => now(),
        'user_cloud_service_id' => $service->id
    ]);

    return response()->json(SaveResource::make($save));
}

```

В листинге 36 представлен вспомогательный метод для определения типа игры `resolveGame(Request $request)` в контроллере `GoogleDriveController`, который возвращает модель `Game` или `SideGame` в зависимости от переданных параметров.

Листинг 36 – Вспомогательный метод `resolveGame` в `GoogleDriveController`

```

// Узнать тип игры
private function resolveGame(Request $request)
{
    if ($request->has('side_game_id') && $request->side_game_id != null) {
        return SideGame::query()->findOrFail($request->side_game_id);
    }
}

```

```

    }
    return Game::query()->findOrFail($request->game_id);
}

```

В листинге 37 представлена функция `generateOverwriteUrl(Save $save, OverwriteSaveRequest $request)` в контроллере `GoogleDriveController`, которая генерирует URL для перезаписи существующего файла сохранения в Google Drive. Проверяет права пользователя и возвращает URL.

Листинг 37 – Функция `generateOverwriteUrl` в `GoogleDriveController`

```

// Генерация ссылки на перезапись сохранения
public function generateOverwriteUrl(Save $save, OverwriteSaveRequest $request): JsonResponse
{
    $user = auth()->user();
    if($save->user_id != $user->id)
        throw new ForbiddenException();
    $cloudService = CloudService::where('name', 'Google Drive')->first();
    $service = $user->userCloudService()
        ->where('cloud_service_id', $cloudService->id)
        ->where('status', CloudStatus::Active)
        ->firstOrFail();
    $googleDriveService = new GoogleDriveService($service);

    return response()->json([
        'upload_url' => $googleDriveService->generateResumableOverwriteUrl(
            $save->file_id,
            $request->input('file_name')
        ),
        'expires_at' => now()->addHours(1)->toIso8601String()
    ]);
}

```

В листинге 38 представлена функция `disconnect(UserCloudService $userCloudService)` в контроллере `GoogleDriveController`, которая отключает интеграцию с Google Drive для пользователя. Очищает токены и переводит статус подключения в `Inactive`. Возвращает статус 204 No Content.

Листинг 38 – Функция `disconnect` в `GoogleDriveController`

```

// Отключение GoogleDrive
public function disconnect(UserCloudService $userCloudService): JsonResponse
{
    $user = auth()->user();

    if ($userCloudService->user_id != $user->id)
        throw new ForbiddenException();

    $userCloudService->update([
        'access_token' => null,
        'refresh_token' => null,
        'expires_at' => null,
        'status' => CloudStatus::Inactive,
    ]);
    return response()->json(null, 204);
}

```

В листинге 39 представлены функции для работы с файлами в контроллере `GoogleDriveController`. Рассмотрим их подробнее:

1. `downloadFile(Save $save)`: скачивает файл сохранения из Google Drive. Проверяет права доступа, получает содержимое файла через API и возвращает его как ответ с заголовками для скачивания.
2. `shareFile(Save $save)`: открывает публичный доступ к файлу сохранения в Google Drive. Возвращает URL для просмотра файла в браузере.

3. `deleteFile(Save $save)`: удаляет файл сохранения из Google Drive и соответствующую запись из БД. Возвращает статус 204 No Content.

Листинг 39 – Функции для работы с файлами в *GoogleDriveController*

```
// Скачивание сохранения
public function downloadFile(Save $save)
{
    $user = auth()->user();
    if($save->user_id != $user->id)
        throw new ForbiddenException();
    $fileId = $save->file_id;
    $cloudService = CloudService::query()->where('name', 'Google Drive')->first();
    $service = $user->userCloudService()
        ->where('cloud_service_id', $cloudService->id)
        ->where('status', CloudStatus::Active)
        ->firstOrFail();

    $googleDriveService = new GoogleDriveService($service);
    $fileData = $googleDriveService->downloadFile($fileId);

    return response($fileData['content'], 200, [
        'Content-Type' => $fileData['mimeType'],
        'Content-Disposition' => 'attachment; filename="' . $fileData['fileName'] . '"',
    ]);
}

// Поделиться сохранением
public function shareFile(Save $save): JsonResponse
{
    $user = auth()->user();
    if($save->user_id != $user->id)
        throw new ForbiddenException();
    $fileId = $save->file_id;
    $cloudService = CloudService::query()->where('name', 'Google Drive')->first();
    $service = $user->userCloudService()
        ->where('cloud_service_id', $cloudService->id)
        ->where('status', CloudStatus::Active)
        ->firstOrFail();

    $googleDriveService = new GoogleDriveService($service);
    $url = $googleDriveService->shareFile($fileId);

    return response()->json(['url' => $url]);
}

// Удаление сохранения
public function deleteFile(Save $save): JsonResponse
{
    $user = auth()->user();
    if($save->user_id != $user->id)
        throw new ForbiddenException();
    $fileId = $save->file_id;
    $cloudService = CloudService::query()->where('name', 'Google Drive')->first();
    $service = $user->userCloudService()
        ->where('cloud_service_id', $cloudService->id)
        ->where('status', CloudStatus::Active)
        ->firstOrFail();

    $googleDriveService = new GoogleDriveService($service);
    $googleDriveService->deleteFile($fileId);

    $save->delete();

    return response()->json(null, 204);
}
```

В листинге 40 представлен контроллер *LibraryController*, который управляет библиотекой пользователя. Он предоставляет функционал для работы с коллекцией игр пользователя, включая добавление/удаление игр, управление избранным и обновление метаданных. Рассмотрим методы подробнее:

1. `getLibrary(Request $request)`: основной метод для получения содержимого библиотеки пользователя. Поддерживает два режима работы: с пагинацией (по умолчанию 30 записей на страницу) и полную загрузку всех записей (при параметре `limit=0`) Загружает связанные модели игр (`game`, `sideGame`) и сортирует по времени игры (`time_played`). Возвращает данные в формате JSON через `LibraryResource`.
2. `addToLibrary(Game $game)`: добавляет игру в библиотеку пользователя. Если запись уже существует, возвращает существующую. Создает новую запись в таблице `libraries` с базовыми параметрами. Возвращает созданную запись со статусом 201 (Created).
3. `toggleFavorite(Game $game)`: переключает статус «Избранное» для игры в библиотеке. Инвертирует текущее значение поля `is_favorite`. Возвращает пустой ответ со статусом 204 (No Content).
4. `toggleSideGameFavorite(SideGame $sideGame)`: аналогичный метод для сторонних игр. Переключает статус избранного для `sideGame`. Также возвращает статус 204.
5. `updateLibraryGame(Game $game, EditLibraryGameRequest $request)`: обновляет метаданные игры в библиотеке (например, время игры или последнюю дату запуска). Использует валидацию через `EditLibraryGameRequest`. Возвращает статус 204 после успешного обновления.
6. `updateSideGameLibrary(SideGame $sideGame, EditLibraryGameRequest $request)`: аналогичный метод обновления для сторонних игр. Применяет те же правила валидации и возвращает тот же статус.
7. `removeFromLibrary(Game $game)`: удаляет игру из библиотеки пользователя. Находит запись по связи `user-game` и полностью удаляет ее из таблицы. Возвращает статус 204.

Листинг 40 – Контроллер библиотеки *LibraryController*

```
class LibraryController extends Controller
{
    // Получение библиотеки
    public function getLibrary(Request $request): JsonResponse
    {
        $user = auth()->user();
        // Проверяем, есть ли параметр limit в запросе и равен ли он 0
        if ($request->has('limit') && $request->input('limit') == 0) {
            // Если limit равен 0, загружаем все записи без пагинации
            $libraries = $user->libraries()
                ->with(['game', 'sideGame'])->orderByDesc('time_played')->get();
        } else {
            // Иначе используем пагинацию с указанным лимитом или по умолчанию 30
            $perPage = $request->input('limit', 30);
            $libraries = $user->libraries()->with(['game', 'sideGame'])->simplePaginate($perPage);
        }
        return LibraryResource::collection($libraries)->response();
    }

    // Добавление игры в библиотеку
    public function addToLibrary(Game $game): JsonResponse
    {
        $user = auth()->user();
        $library = Library::query()->firstOrCreate([
            'user_id' => $user->id,
            'game_id' => $game->id,
        ]);
        $library->load('game');
        return response()->json(LibraryResource::make($library), 201);
    }

    // Изменение статуса избранного для игры
    public function toggleFavorite(Game $game): JsonResponse
    {
        $user = auth()->user();
        $library = $user->libraries()->where('game_id', $game->id)->firstOrFail();
    }
}
```

```

        $library->update(['is_favorite' => !$library->is_favorite]);
        return response()->json(null, 204);
    }
    // Изменение статуса избранного для сторонней игры
    public function toggleSideGameFavorite(SideGame $sideGame): JsonResponse
    {
    }
    // Изменение данных библиотеки игры
    public function updateLibraryGame(Game $game, EditLibraryGameRequest $request): JsonResponse
    {
        $user = auth()->user();
        $library = $user->libraries()->where('game_id', $game->id)->firstOrFail();
        $library->update($request->validated());
        return response()->json(null, 204);
    }
    // Изменение данных библиотеки сторонней игры
    public function updateSideGameLibrary(SideGame $sideGame, EditLibraryGameRequest $request):
    JsonResponse
    {
    }
    // Удаление игры из библиотеки
    public function removeFromLibrary(Game $game): JsonResponse
    {
        $user = auth()->user();
        $library = $user->libraries()->where('game_id', $game->id)->firstOrFail();
        $library->delete();
        return response()->json(null, 204);
    }
}

```

В листинге 41 представлен контроллер MailController, отвечающий за функционал восстановления пароля через электронную почту. Контроллер реализует трехэтапный процесс: отправку кода подтверждения, его верификацию и непосредственную смену пароля. Рассмотрим методы подробнее:

1. `sendResetCode()`: отправляет 6-значный код на почту, ограничивая частоту запросов (1 раз в минуту), сохраняет код в кэш на 5 минут и возвращает статус 204.
2. `verifyCode()`: проверяет введенный код с сохраненным в кэше, при успехе генерирует одноразовый токен для смены пароля и возвращает его в ответе.
3. `restoreFromMail()`: проверяет токен, обновляет пароль пользователя, при необходимости выходит со всех устройств, очищает кэш и возвращает статус 204.

Листинг 41 – Контроллер восстановления аккаунта через почту MailController

```

class MailController extends Controller
{
    // Отправка письма с кодом
    public function sendResetCode(SendResetCodeRequest $request): JsonResponse
    {
        $email = $request->email;
        // Проверяем лимит (1 запрос в минуту)
        if (RateLimiter::tooManyAttempts('reset-code:' . $email, 1)) {
            $seconds = RateLimiter::availableIn('reset-code:' . $email);
            throw new ApiException("Retry possible in $seconds seconds", 429);
        }
        // Увеличиваем счётчик запросов (лимит: 1 запрос в 60 сек.)
        RateLimiter::hit('reset-code:' . $email, 60);
        $code = Str::random(6);
        Cache::put('password_reset_' . $email, $code, now()->addMinutes(5));
        Mail::to($email)->send(new PasswordResetCodeMail($code));
        return response()->json(null, 204);
    }
    // Подтверждение кода
    public function verifyCode(VerifyCodeRequest $request): JsonResponse
    {
        $cachedCode = Cache::get('password_reset_' . $request->email);
        if (!$cachedCode || $cachedCode !== $request->code) {
            throw new ApiException("Invalid or expired code", 403);
        }
    }
}

```

```

    }
    // Генерируем одноразовый токен для смены пароля
    $resetToken = Str::random(32);
    Cache::put('password_reset_token_' . $request->email, $resetToken, now()->addMinutes(5));
    return response()->json(['reset_token' => $resetToken]);
}
// Восстановление через почту
public function restoreFromMail(RestoreFromMailRequest $request): JsonResponse
{
    // Проверяем токен
    $savedToken = Cache::get('password_reset_token_' . $request->email);
    if (!$savedToken || $savedToken !== $request->reset_token) {
        throw new ApiException("Invalid or expired token", 403);
    }
    // Обновляем пароль
    $user = User::where('email', $request->email)->firstOrFail();
    $user->update(['password' => $request->new_password]);
    if($request->logout == true)
        $user->tokens()->delete();
    // Очищаем кеш
    Cache::forget('password_reset_' . $request->email);
    Cache::forget('password_reset_token_' . $request->email);
    return response()->json(null, 204);
}
}

```

В листинге 42 представлен контроллер SaveController, который управляет операциями с игровыми сохранениями пользователя. Рассмотрим подробнее каждый метод:

1. `getMySavesGame()`: возвращает все сохранения пользователя для указанной игры, включая только те, что связаны с активными облачными сервисами, с дополнительной информацией о сервисе хранения.
2. `getMySavesSideGame()`: аналогично получает сохранения, но для сторонних игр (модификаций), также проверяя активность облачного сервиса перед возвратом данных.
3. `updateSave()`: обновляет метаданные сохранения и, если оно хранится в Google Drive, переименовывает родительскую папку согласно новой версии, затем возвращает обновленные данные сохранения.

Листинг 42 – Контроллер сохранений SaveController

```

class SaveController extends Controller
{
    // Получение сохранений к игре
    public function getMySavesGame(Game $game)
    {
        $user = auth()->user();
        $saves = $user->saves()
            ->where('game_id', $game->id)
            ->whereHas('userCloudService', function ($query) {
                $query->where('status', CloudStatus::Active);
            })
            ->with('userCloudService')->get();
        return response()->json(['saves' => SaveResource::collection($saves)]);
    }
    // Получение сохранений к сторонней игре
    public function getMySavesSideGame(SideGame $sideGame)
    {
        $user = auth()->user();
        $saves = $user->saves()
            ->where('side_game_id', $sideGame->id)
            ->whereHas('userCloudService', function ($query) {
                $query->where('status', CloudStatus::Active);
            })
            ->with('userCloudService')->get();
        return response()->json(['saves' => SaveResource::collection($saves)]);
    }
}

```

```

}
// Перезапись сохранения
public function updateSave(Save $save, UpdateSaveRequest $request)
{
    $user = auth()->user();
    $cloudService = CloudService::query()
        ->where('name', 'Google Drive')->first();
    if($save->userCloudService()
        ->where('cloud_service_id', $cloudService->id)
        ->where('status', CloudStatus::Active)->exists())
    {
        $fileId = $save->file_id;
        $service = $user->userCloudService()
            ->where('cloud_service_id', $cloudService->id)
            ->where('status', CloudStatus::Active)->first();
        $googleDriveService = new GoogleDriveService($service);
        $googleDriveService->getAndRenameParentFolder($fileId, $request->version);
    }
    $save->update($request->validated());
    return response()->json(SaveResource::make($save));
}
}

```

В листинге 43 представлен контроллер SideGameController, который управляет созданием, получением и удалением сторонних игр. Рассмотрим подробнее функции контроллера:

1. addSideGame(): создает новую стороннюю игру и автоматически добавляет её в библиотеку пользователя. Проверяет уникальность названия, связывает с текущим пользователем и возвращает данные созданной записи со статусом 201.
2. getSideGame(): возвращает полную информацию о сторонней игре - основные данные, статус в библиотеке и список связанных сохранений. Если сохранений нет, возвращает null для соответствующего поля.
3. removeSideGame(): удаляет стороннюю игру пользователя вместе со всеми зависимостями. Проверяет принадлежность игры перед удалением и возвращает пустой ответ со статусом 204.

Листинг 43 – Контроллер сторонних игр SideGameController

```

class SideGameController extends Controller
{
    // Добавление сторонней игры
    public function addSideGame(SideGameRequest $request): JsonResponse
    {
        $user = auth()->user();
        $sideGame = $user->sideGames()->firstOrCreate($request->validated());
        $library = Library::query()->firstOrCreate([
            'user_id' => $user->id,
            'side_game_id' => $sideGame->id,
        ]->load('sideGame'));
        return response()->json(LibraryResource::make($library), 201);
    }

    // Получение сторонней игры
    public function getSideGame(SideGame $sideGame): JsonResponse
    {
        $user = auth()->user();
        $library = $user->libraries()->where('side_game_id', $sideGame->id)->firstOrFail();
        $saves = $user->saves()->where('side_game_id', $sideGame->id)->get();
        return response()->json([
            'side_game' => SideGameResource::make($sideGame),
            'library' => $library ? LibraryResource::make($library) : null,
            'saves' => $saves->isEmpty() ? null : SaveResource::collection($saves),
        ]);
    }

    // Удаление сторонней игры
}

```

```

public function removeSideGame(SideGame $sideGame): JsonResponse
{
    $user = auth()->user();
    $user->sideGames()->findOrFail($sideGame->id)->delete();
    return response()->json(null, 204);
}
}

```

В листинге 44 представлен контроллер UserController, который управляет профилем пользователя и связанными данными. Рассмотрим функции подробнее:

1. `getProfile()`: возвращает полную информацию о текущем аутентифицированном пользователе в формате JSON, используя `UserResource` для преобразования данных.
2. `getCloudServices()`: возвращает список всех доступных облачных сервисов для интеграции, отформатированный через `CloudServiceResource`. Не требует аутентификации.
3. `getStatistic()`: формирует игровую статистику пользователя - общее время игры и список последних 6 игр (основных или модификаций) с временем игры, отсортированный по дате последнего запуска.

Листинг 44 – Контроллер пользователя UserController

```

class UserController extends Controller
{
    // Получение своего профиля
    public function getProfile(): JsonResponse
    {
        $user = auth()->user();
        return response()->json(UserResource::make($user));
    }
    // Получение облачных сервисов
    public function getCloudServices(): JsonResponse
    {
        $services = CloudService::all();
        return response()->json(CloudServiceResource::collection($services));
    }
    // Получение статистики
    public function getStatistic(): JsonResponse
    {
        $user = auth()->user();
        $totalPlayed = $user->libraries()->get()->sum('time_played');

        $recentlyPlayed = $user->libraries()
            ->get()
            ->sortByDesc('last_played_at')
            ->take(6)
            ->map(function ($item) {
                $gameData = $item->game_id
                    ? ['game' => GameResource::make($item->game)]
                    : ['sideGame' => SideGameResource::make($item->sideGame)];

                return array_merge($gameData, [
                    'time_played' => $item->time_played
                ]);
            })
            ->values();
        return response()->json([
            'totalPlayed' => $totalPlayed,
            'recentlyPlayed' => $recentlyPlayed,
        ]);
    }
}

```

В листинге 45 представлена функция `updateProfile()` в `UserController`, которая обновляет данные профиля с комплексной валидацией. Обработывает смену пароля (с проверкой

текущего), загрузку аватара и хедера с уникальными именами файлов, автоматически удаляя старые изображения. Возвращает обновлённый профиль.

Листинг 45 – Функция updateProfile в UserController

```
// Обновление профиля
public function updateProfile(UpdateProfileRequest $request): JsonResponse
{
    $user = auth()->user();
    if (!$user instanceof User) {
        throw new \RuntimeException('Authenticated user is not an instance of User model.');
```

ГЛАВА 4. ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ДЕСКТОПНОГО ПРИЛОЖЕНИЯ

Проектирование интерфейса десктопного приложения

В рамках данного проекта используются схематические макеты (wireframe), определяющие расположение элементов интерфейса и логику пользовательского взаимодействия. Дизайн и стилизация не проработаны на данном этапе.

Окно входа

Окно входа представляет собой стартовый интерфейс системы, который является первым и основным пунктом взаимодействия пользователя с приложением — оно автоматически открывается при первоначальном запуске программы или в случае, когда пользователь явно завершил предыдущую сессию, нажав кнопку "Выйти". Главная функциональная нагрузка данного окна заключается в обеспечении безопасной и удобной процедуры аутентификации, когда пользователь подтверждает свою личность путём ввода заранее созданных учётных данных, а именно — уникального логина или привязанного email-адреса в сочетании с секретным паролем (см. рис. 21). Помимо базового сценария входа, интерфейс предусматривает вспомогательные опции, призванные охватить все возможные пользовательские ситуации: для тех, кто утратил доступ к своему аккаунту, предусмотрен механизм восстановления пароля на выбор пользователя (см. рис. 22), а для новых пользователей, только знакомящихся с приложением, доступна интуитивно понятная возможность мгновенного перехода к процессу регистрации, где можно создать совершенно новый аккаунт и получить полный доступ ко всем функциям системы.

Вход в аккаунт

Логин или email
Введите ваш логин или email

Пароль
Введите ваш пароль

Забыли пароль?

Войти

или

Создать аккаунт

Рисунок 21 – Окно входа

Вход в аккаунт

Логин или email
Введите ваш логин или email

Пароль
Введите ваш пароль

Забыли пароль?

Восстановить с помощью ключа

Восстановить через email

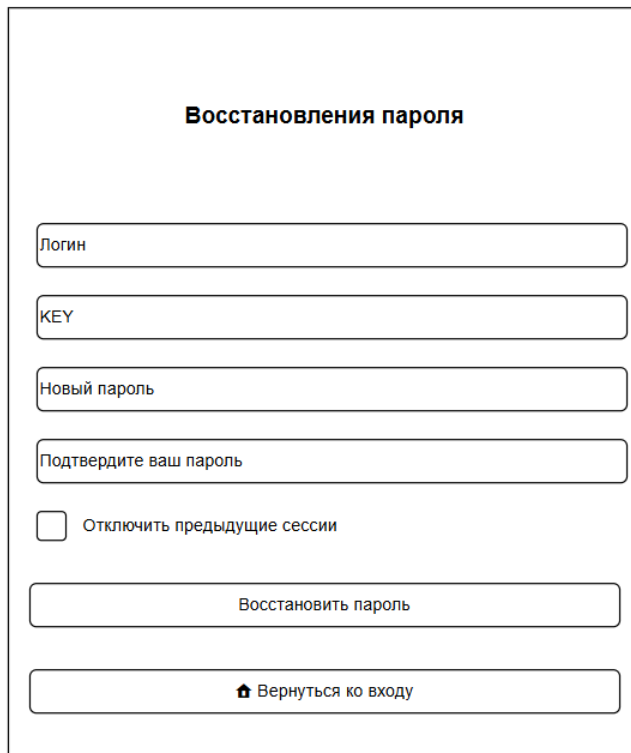
или

Создать аккаунт

Рисунок 22 – Выбор восстановления пароля

Страница восстановления пароля по ключу

Окно восстановления пароля по ключу является частью системы безопасности приложения и позволяет пользователю сбросить утраченный или забытый пароль (см. рис. 23). На этот экран можно попасть из окна входа, кликнув на ссылку «*Забыли пароль?*» под полем ввода пароля. После успешного восстановления пользователь перенаправляется обратно на экран входа, где может авторизоваться с новыми данными.



Восстановления пароля

Логин

KEY

Новый пароль

Подтвердите ваш пароль

☐ Отключить предыдущие сессии

Восстановить пароль

🏠 Вернуться ко входу

Рисунок 23 – Восстановление пароля по ключу

Окно регистрации

Окно регистрации нового аккаунта выполняет две основные функции: сбор необходимых данных для идентификации пользователя (логин, email и пароль) и обеспечение безопасности будущего аккаунта через многоуровневую проверку вводимой информации (см. рис. 24).

После успешного заполнения формы система генерирует уникальный KEY для восстановления доступа - критически важный элемент защиты, который пользователь должен сохранить в надёжном месте. Особенностью процесса является двухэтапный подход: сначала стандартная регистрация с базовыми данными, а затем - обязательное подтверждение сохранения резервного KEY (см. рис. 25), что значительно снижает риски безвозвратной потери доступа к аккаунту.

Создать аккаунт

Логин

Придумайте логин

Email

Введите ваш email

Пароль

Введите ваш пароль

Подтверждение пароля

Повторите пароль

Зарегистрироваться

Уже есть аккаунт?

Войти

Рисунок 24 – Окно регистрации

Создать аккаунт

Логин

Придумайте логин

Ваш KEY для восстановления

Это ваш KEY для восстановления. Сохраните его в надежном месте. Он понадобится для восстановления доступа к вашему аккаунту.

1 2 3 4 5 6

☐ Я сохранил свой KEY для восстановления

Подтвердить

Закрыть

Уже есть аккаунт?

Войти

Рисунок 25 – Подтверждение KEY

Главное окно приложения

Главное окно является центральным узлом взаимодействия пользователя с приложением после успешной авторизации. Оно предоставляет доступ ко всем ключевым функциям через интуитивную навигационную панель и рабочую область для отображения контента (см. рис. 26).

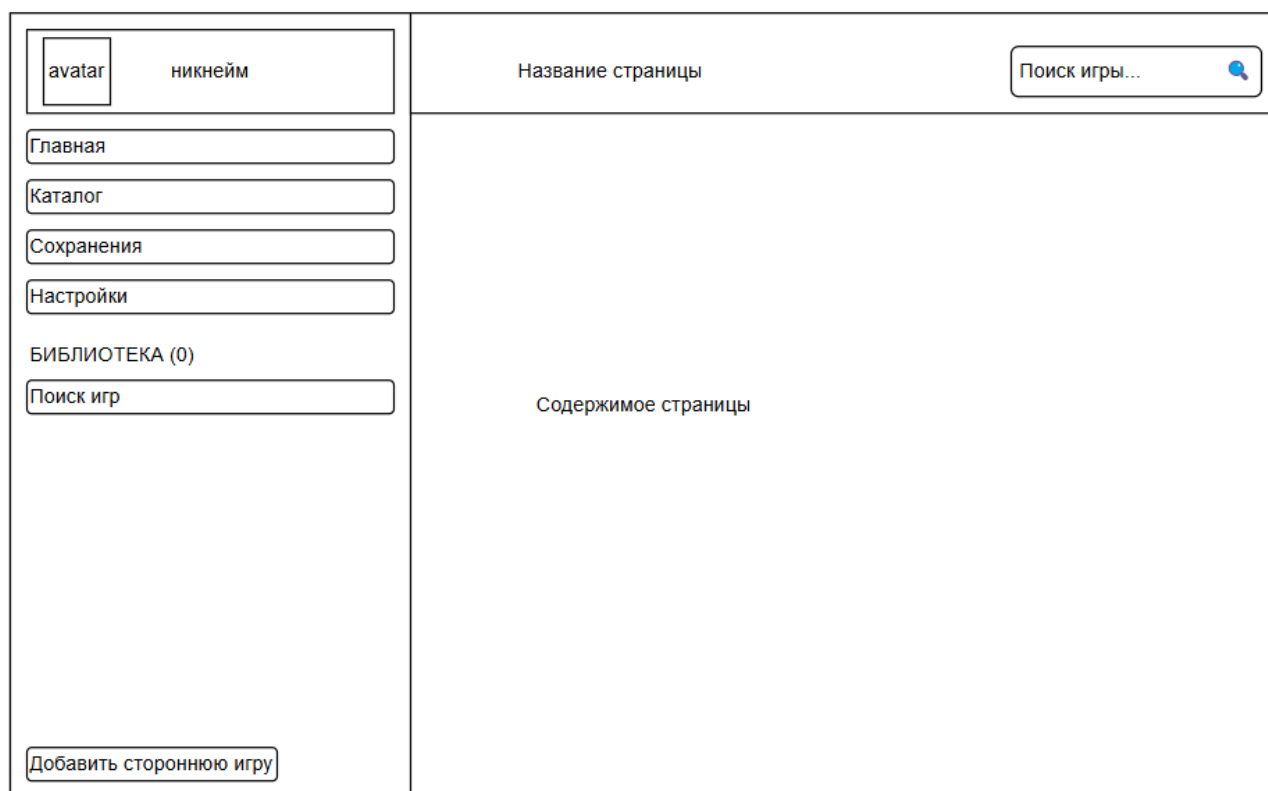


Рисунок 26 – Главное окно приложения

Страница каталога

Страница каталога игр является центральным разделом приложения, предназначенным для просмотра, поиска и выбора игр (см. рис. 27). Оно открывается при переходе из главного меню по соответствующей вкладке и предоставляет пользователю удобный инструментарий для навигации по обширному ассортименту контента.

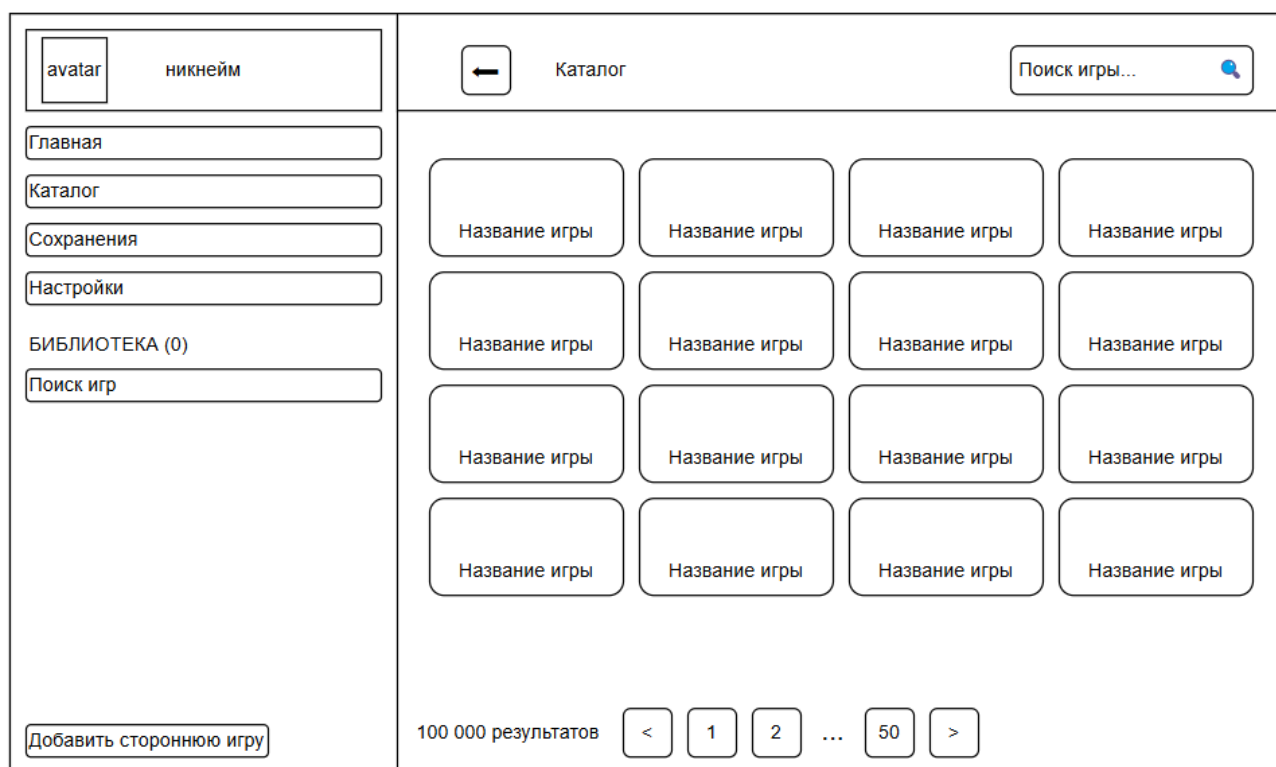


Рисунок 27 – Страница каталога

Страница игры

Страница игры открывается при переходе из раздела *Каталог* по нажатию карточки с игрой (см. рис. 27) и служит для отображения полной информации о конкретном продукте. Это ключевая точка взаимодействия пользователя с контентом перед запуском или добавлением в библиотеку (см. рис. 28).



Рисунок 28 – Страница игры

Страница игры после добавления в библиотеку становится личной игровой панелью пользователя, предоставляя доступ к управлению игрой, отслеживанию статистики и работе с сохранениями, последующее отображение страницы игры при переходе из библиотеки или каталога изображено на рисунке 29.

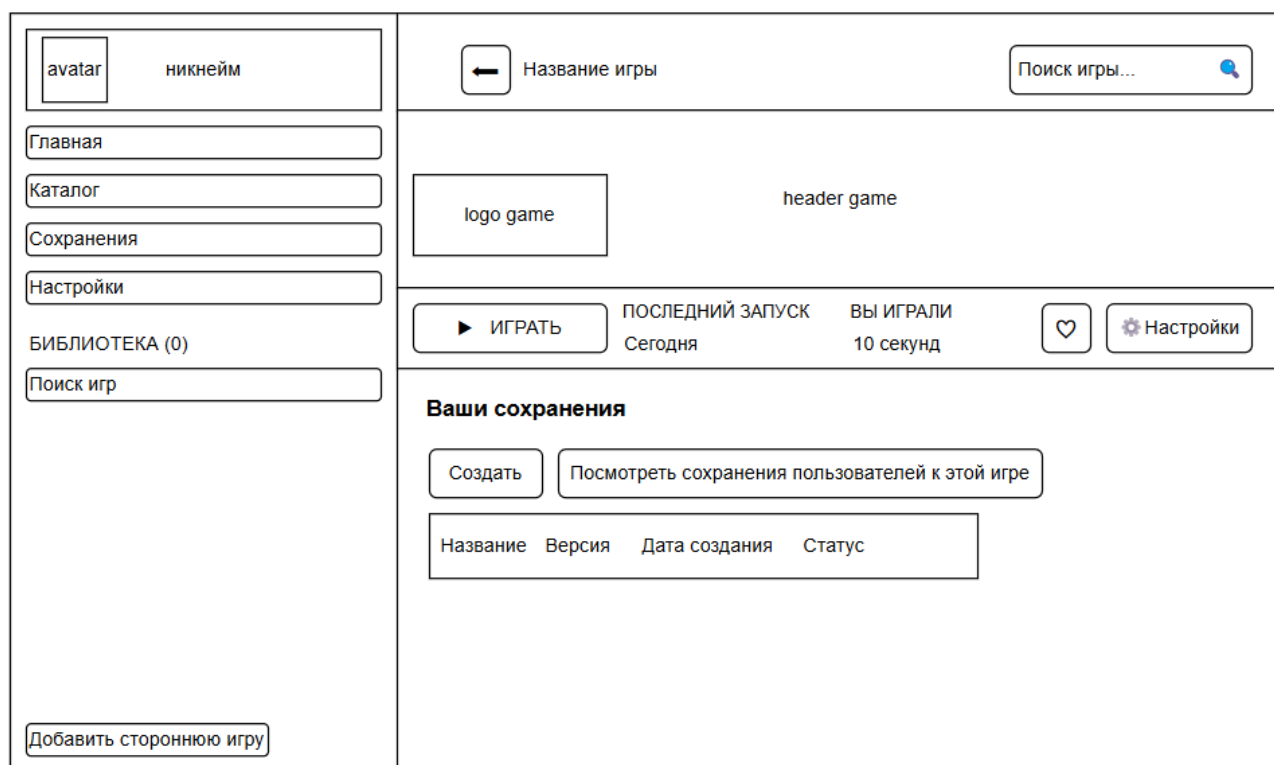


Рисунок 29 – Страница игры после добавления в библиотеку

Структура директорий проекта

Десктопное приложение «PlaySaveBack» построено на платформе WINUI 3 с применением архитектурного паттерна MVVM (Model-View-ViewModel).

Архитектура MVVM

1. View отвечает только за отображение данных.
2. ViewModel содержит бизнес-логику и взаимодействует с API.
3. Model представляет структуру данных, получаемых с сервера.

Структура проекта включает в себя следующие основные каталоги и файлы:

- Api/ – содержит модели запросов и ответов API.
 - Request/ – модели данных для отправки запросов к API.
 - Response/ – модели данных, получаемых в ответ от API.
 - URLs.cs – класс содержащий базовый URL и URL к API.
- Converters/ – содержит различные конвертеры данных.
- Helpers/ – вспомогательные классы.
 - FindSaves.cs – класс, который ищет папку с сохранениями, основываясь на расположение исполняемого файла.
 - IconFromExe.cs – класс, который извлекает иконку игры из исполняемого файла для дальнейшего использования.
 - ZipHelper.cs – вспомогательный класс для работы с zip файлами.
- Models/ – модели данных, используемые в приложении.
- Services/ – сервисы для вызова из любой части приложения.
 - AuthNavigationService.cs – сервис, в котором описана навигация в рамках аутентификации.
 - DialogService.cs – сервис, используемый для вызова диалогового окна.
 - ErrorHandlerService.cs – сервис для парсинга ошибок из API.
 - GameLaunchService.cs – сервис для запуска и закрытия игры.
 - LibraryService.cs – сервис, связанный с библиотекой пользователя.
 - NavigationService.cs – общий сервис навигации, позволяющий перемещаться между окнами приложения.
 - NotifivationService.cs – сервис уведомления, который показывает ошибку или успех
- Styles/ – стили используемые в приложении.
- Utils/ – утилитарные классы, в которых хранятся локальные данные.
 - Game – классы, связанные с играми.
 - BaseDataManager.cs – базовый дата-менеджер, от которого будут наследоваться остальные менеджеры.
 - GameDataManager.cs – менеджер, хранящий данные об игре.
 - LastPlayedGameManager.cs – менеджер, хранящий последнюю запущенную игру.
 - LibraryDataManager.cs – менеджер, отвечающий за библиотеки.
 - PathDataManager.cs – менеджер, отвечающий за различные пути.
 - SavesDataManage.cs – менеджер, отвечающий за сохранения.
- ViewModels/ – классы ViewModel, отвечающие за логику представлений.
- Views/ – XAML-страницы и пользовательские элементы управления.

Навигация и уведомления

Каждая страница имеет уникальный тег и тип, что позволяет однозначно идентифицировать её в системе. Навигация реализована через `NavigationService`, который располагается в `Services/NavigationService.cs`, он управляет переходами, историей и параметрами (см. листинг 46).

Листинг 46 – Сервис навигации `NavigationService`

```
public class NavigationService
{
    // Основные UI-компоненты для навигации
    private readonly Frame _frame;
    private readonly NavigationView _navView;
    private readonly TextBlock _headerText;
    // Словарь для связи тегов страниц с их типами
    private readonly Dictionary<string, Type> _pages = new()
    {
        { "HomePage", typeof(HomePage) },
        { "CatalogPage", typeof(CatalogPage) },
        { "SettingsPage", typeof(SettingsPage) },
        { "ProfilePage", typeof(ProfilePage) },
    };
    // Словарь заголовков для страниц
    private readonly Dictionary<string, string> _pageTitles = new()
    {
        { "HomePage", "Главная" },
        { "CatalogPage", "Каталог" },
        { "SettingsPage", "Настройки" },
        { "ProfilePage", "Профиль" },
    };
    // Конструктор с инъекцией зависимостей
    public NavigationService(Frame frame, NavigationView navView, TextBlock headerText)
    {
        _frame = frame;
        _navView = navView;
        _headerText = headerText;

        // Подписка на события навигации
        _frame.Navigated += OnNavigated;
        _navView.SelectionChanged += OnNavigationViewSelectionChanged;
    }
    // Основной метод навигации по тегу страницы
    public void Navigate(string pageTag)
    {
        // Проверка входных данных
        if (string.IsNullOrEmpty(pageTag))
        {
            Debug.WriteLine("Page tag is null or empty.");
            return;
        }

        try
        {
            // Специальная обработка страниц игр
            if (pageTag.StartsWith("Game_") || pageTag.StartsWith("SideGame_"))
            {
                // Извлекаем ID и название игры из тега
                ulong gameId = ExtractGameId(pageTag);
                string gameName = ExtractGameName(pageTag);
                string type = pageTag.StartsWith("Game_") ? "Game" : "SideGame";

                // Оптимизация: не перезагружаем страницу, если уже на ней
                if (_frame.Content is GamePage currentGamePage &&
                    currentGamePage.GameViewModel?.GameId == gameId)
                {
                    _headerText.Text = gameName;
                    return;
                }

                // Подготовка параметров навигации
                var parameters = new GameNavigationParameters
                {
                    Type = type,
```

```

        gameId = gameId
    };

    // Выполняем навигацию
    _frame.Navigate(typeof(GamePage), parameters);
    _headerText.Text = gameId;
    return;
}

// Обработка обычных страниц
if (_pages.TryGetValue(pageTag, out var pageType))
{
    _frame.Navigate(pageType);
    // Устанавливаем заголовок из словаря или используем тег как fallback
    _headerText.Text = _pageTitles.TryGetValue(pageTag, out var title)
        ? title
        : pageTag;
}
else
{
    Debug.WriteLine($"Page type not found for tag: {pageTag}");
    _frame.Content = new TextBlock { Text = "Page not found" };
}
}
catch (Exception ex)
{
    Debug.WriteLine($"Navigation error: {ex.Message}");
    _frame.Content = new TextBlock { Text = "Navigation error" };
}
}

```

В листинге 47 описан MainWindow.xaml, который определяет основную структуру главного окна приложения, включая систему навигации и глобальные уведомления. В файле реализован:

1. Навигационный каркас с NavigationView в левой панели (содержит пункты меню «Главная», «Каталог», «Настройки» и профиль пользователя с аватаркой).
2. Контентная область (Frame) для отображения страниц.
3. Глобальный InfoBar для показа уведомлений в правом нижнем углу.
4. Шапка с кнопкой "Назад" и поиском.
5. Визуализация профиля (аватар + никнейм на фоне изображения).
6. Кнопка добавления сторонней игры в подвале меню.

Разметка использует DataBinding для динамического обновления данных (аватар, имя пользователя) и анимации (плавное появление уведомлений). Навигация работает по тегам (Tag), которые обрабатываются NavigationService.

Листинг 47 – Главное окно приложение MainWindow.xaml

```

<Grid x:Name="RootGrid">
    <NavigationView
        x:Name="NavView"
        IsBackEnabled="True"
        AlwaysShowHeader="True"
        PaneDisplayMode="Left"
        SelectionFollowsFocus="Disabled"
        IsTabStop="False"
        IsBackButtonVisible="Collapsed"
        IsSettingsVisible="False"
        IsTitleBarAutoPaddingEnabled="False"
        IsPaneToggleButtonVisible="False"
        IsPaneOpen="True"
    >
        <!-- Заголовок с кнопкой "Назад" -->
        <NavigationView.Header>
            <Grid Height="72" ColumnDefinitions="50, Auto,*"
                VerticalAlignment="Center"
                ColumnSpacing="10"
                Padding="24,0"
                Margin="-56,0,0,0">
                <Button x:Name="BackButton"

```

```

        Content="#xE72B;"
        FontFamily="Segoe MDL2 Assets"
        FontSize="16"
        Click="BackButton_Click"
        Grid.Column="0"/>
<TextBlock x:Name="HeaderText"
    VerticalAlignment="Center"
    Style="{StaticResource SubtitleTextBlockStyle}"
    Grid.Column="1"/>
<!-- Поиск -->
<AutoSuggestBox Width="300"
    Height="40"
    Grid.Column="2"
    HorizontalAlignment="Right"
    VerticalAlignment="Center"
    QuerySubmitted="AutoSuggestBox_QuerySubmitted"
    Text="{x:Bind CatalogViewModel.Name, Mode=TwoWay}"
    QueryIcon="Find"
    PlaceholderText="Поиск игр..."
    CornerRadius="10"/>
</Grid>
</NavigationView.Header>
<NavigationView.MenuItems>
    <NavigationViewItem Margin="0, 10, 0, 10" Height="100" Tag="ProfilePage">
        <NavigationViewItem.Content>
            <Grid VerticalAlignment="Stretch" HorizontalAlignment="Stretch" Grid.Row="0">
                <!-- Фоновое изображение (растянуто на весь Grid) -->
                <Image
                    Source="{x:Bind ProfileViewModel.User.Header, Mode=OneWay}"
                    Stretch="Fill"
                    HorizontalAlignment="Stretch"
                    VerticalAlignment="Stretch"/>
                <!-- Контент поверх фона -->
                <StackPanel
                    Orientation="Horizontal"
                    VerticalAlignment="Center"
                    Spacing="12">
                    <!-- Аватарка -->
                    <Image
                        VerticalAlignment="Center"
                        Stretch="Fill"
                        Height="95"
                        Width="95"
                        Source="{x:Bind ProfileViewModel.User.Avatar, Mode=OneWay}"/>
                    <!-- Никнейм -->
                    <TextBlock
                        Text="{x:Bind ProfileViewModel.User.Nickname, Mode=OneWay}"
                        FontWeight="SemiBold"
                        FontSize="28"
                        VerticalAlignment="Center"/>
                </StackPanel>
            </Grid>
        </NavigationViewItem.Content>
    </NavigationViewItem>
    <NavigationViewItem Content="Главная" Tag="HomePage" Icon="Home" IsSelected="True"/>
    <NavigationViewItem Content="Каталог" Tag="CatalogPage">
        <NavigationViewItem.Icon>
            <FontIcon Glyph="#xF0E2;"/>
        </NavigationViewItem.Icon>
    </NavigationViewItem>
    <NavigationViewItem Content="Настройки" Tag="SettingsPage" Icon="Setting"/>
</NavigationView.MenuItems>
<NavigationView.PaneFooter>
    <Button Margin="10,0,0,0" Command="{x:Bind ProfileViewModel.AddSideGameCommand,
Mode=OneWay}" >
        <StackPanel Orientation="Horizontal" Spacing="6">
            <FontIcon Glyph="#xE710;"/>
            <TextBlock Text="Добавить стороннюю игру"/>
        </StackPanel>
    </Button>
</NavigationView.PaneFooter>
<Frame x:Name="ContentFrame" />
</NavigationView>
<!-- InfoBar в правом нижнем углу -->
<InfoBar x:Name="GlobalInfoBar"
    HorizontalAlignment="Right"
    VerticalAlignment="Bottom"
    Margin="0,0,20,20"

```

```

        IsOpen="False"
        Title="Уведомление"
        IsClosable="True"
        CornerRadius="4"
        Width="400"
        Opacity="0.95"
        RequestedTheme="Dark">
    <InfoBar.Transitions>
    <TransitionCollection>
    <EntranceThemeTransition FromHorizontalOffset="0" FromVerticalOffset="20"/>
    </TransitionCollection>
    </InfoBar.Transitions>
    </InfoBar>
</Grid>

```

На рисунке 30 изображен интерфейс навигации приложения с пустой библиотекой.

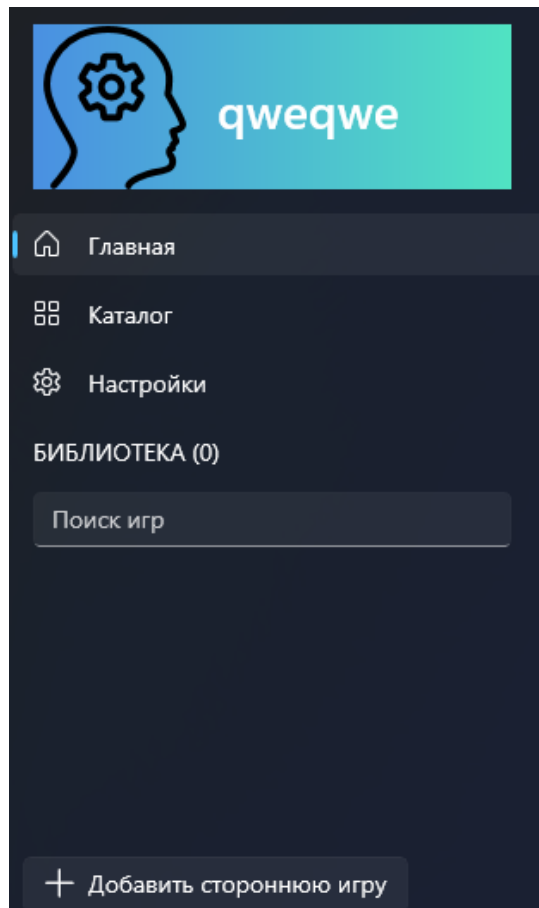


Рисунок 30 – Интерфейс навигации с пустой библиотекой

В листинге 48 представлен сервис библиотек `LibraryService`, который отвечает за динамическое обновление навигационного меню библиотеки игр. При добавлении игры в библиотеку сервис автоматически создает соответствующий пункт в меню, группируя игры по категориям:

1. Установленные игры – игры с обнаруженным исполняемым файлом.
2. Избранное – отмеченные пользователем игры.
3. Сторонние игры – пользовательские модификации.
4. Без категории – остальные игры.

Ключевые функции:

- Динамическое обновление: при изменении коллекции Libraries автоматически перестраивает меню, сохраняя состояние сворачивания категорий.
- Поиск игр: фильтрация через AutoSuggestBox по названию (включая сторонние игры).
- Визуализация:
 - Отображает количество игр в заголовке «БИБЛИОТЕКА (N)».
 - Использует иконки исполняемых файлов (если путь доступен) или стандартные иконки.
 - Поддерживает сворачивание/разворачивание категорий.
- Навигация: пункты меню содержат теги формата Game_<ID> или SideGame_<ID>, которые обрабатываются NavigationService для открытия страницы игры.

Интеграция:

- Связан с ProfileViewModel для доступа к данным библиотеки.
- Использует NavigationService для синхронизации выделения в меню.
- Добавляет поисковую панель и заголовок в NavigationView.

Листинг 48 – Сервис библиотек LibraryService

```
public class LibraryService
{
    private readonly NavigationView _navView;
    private readonly ProfileViewModel _profileViewModel;
    private readonly NavigationService _navigationService;
    private readonly AutoSuggestBox _searchBox;
    private readonly TextBlock _gamesCountTextBlock; // Для отображения количества игр
    private readonly NavigationViewItem _libraryHeader; // Заголовок "БИБЛИОТЕКА"

    // Словарь для хранения состояния сворачивания категорий
    private readonly Dictionary<string, bool> _categoryCollapseState = [];

    public LibraryService(NavigationView navView, ProfileViewModel profileViewModel,
        NavigationService navigationService)
    {
        _navView = navView;
        _profileViewModel = profileViewModel;
        _navigationService = navigationService;

        // Инициализация AutoSuggestBox
        _searchBox = new AutoSuggestBox
        {
            PlaceholderText = "Поиск игр",
            HorizontalAlignment = Microsoft.UI.Xaml.HorizontalAlignment.Stretch,
        };

        // Подписка на событие изменения текста
        _searchBox.TextChanged += OnSearchTextChanged;

        // Создаем заголовок "БИБЛИОТЕКА" с количеством игр
        _libraryHeader = new NavigationViewItem
        {
            Content = "БИБЛИОТЕКА",
            SelectsOnInvoked = false // Отключаем выбор элемента
        };

        // Создаем TextBlock для отображения количества игр
        _gamesCountTextBlock = new TextBlock
        {
            VerticalAlignment = Microsoft.UI.Xaml.VerticalAlignment.Center,
            FontSize = 14
        };

        // Добавляем AutoSuggestBox и заголовок в NavigationView
        var searchItem = new NavigationViewItem
        {
```

```

        Content = _searchBox,
        SelectsOnInvoked = false // Отключаем выбор элемента
    };

    // Добавляем элементы в MenuItems
    _navView.MenuItems.Add(_libraryHeader);
    _navView.MenuItems.Add(searchItem);

    // Подписка на обновления библиотеки
    _profileViewModel.Libraries.CollectionChanged += (s, e) => UpdateLibraryMenu();
}

private void OnSearchTextChanged(AutoSuggestBox sender, AutoSuggestBoxTextChangedEventArgs args)
{
    if (args.Reason == AutoSuggestionBoxTextChangeReason.UserInput)
    {
        // Фильтрация игр по введенному тексту
        var searchText = sender.Text?.ToLower() ?? string.Empty;

        // Если поле поиска пустое, возвращаем все игры
        if (string.IsNullOrEmpty(searchText))
        {
            UpdateLibraryMenu([.. _profileViewModel.Libraries]);
            return;
        }

        // Фильтруем игры по названию (включая сторонние игры)
        var filteredGames = _profileViewModel.Libraries
            .Where(game => game.Game != null && game.Game.Name.Contains(searchText,
                System.StringComparison.CurrentCultureIgnoreCase) ||
                game.SideGame != null && game.SideGame.Name.Contains(searchText,
                System.StringComparison.CurrentCultureIgnoreCase))
            .ToList();

        // Обновление меню с учетом фильтрации
        UpdateLibraryMenu(filteredGames);
    }
}

private bool _isUpdatingLibrary = false;

public void UpdateLibraryMenu(List<Library> filteredGames = null) {}
}

```

На рисунке 31 изображена навигация с наличием игр в библиотеке.

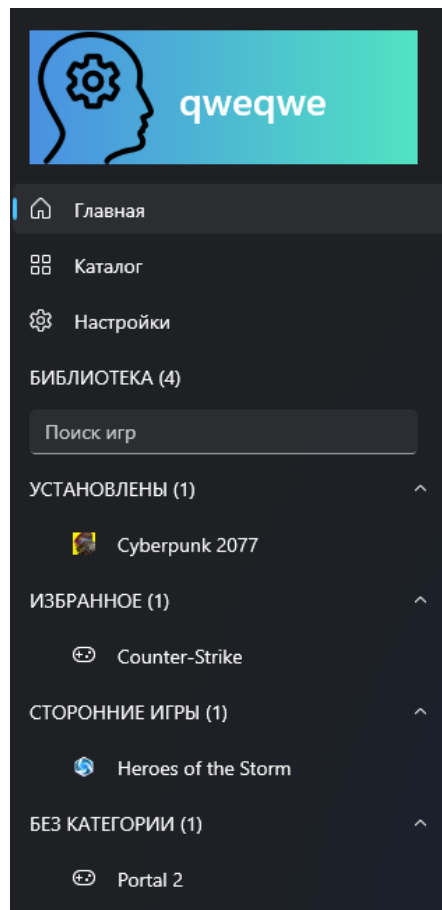


Рисунок 31 – Навигация с играми в библиотеке

В листинге 49 описан сервис NotificationService, реализующий систему уведомлений в приложении. Сервис выполняет следующие функции:

1. Инициализация компонентов уведомлений через метод Initialize(), принимающий элементы интерфейса InfoBar и Grid.
2. Показ уведомлений с гибкой настройкой параметров:
 - Текст сообщения и заголовок
 - Тип уведомления (информационное, успех, ошибка, предупреждение)
 - Автоматическое скрывание с настраиваемым таймаутом
3. Управление временем показа через встроенный таймер DispatcherQueueTimer.
4. Автоматическая остановка предыдущих уведомлений перед показом новых.
5. Готовые методы для стандартных сценариев:
 - ShowSuccess() для успешных операций
 - ShowError() для ошибок
 - ShowWarning() для предупреждений
 - ShowInfo() для информационных сообщений
6. Потокбезопасная реализация через DispatcherQueue.TryEnqueue.
7. Поддержка как автоматического скрывания (для информационных сообщений и успехов), так и постоянного отображения (для ошибок и предупреждений).

Листинг 49 – Сервис уведомлений NotificationService

```
public static class NotificationService
{
    private static InfoBar? _infoBar;
    private static Grid? _rootGrid;
    private static DispatcherQueueTimer? _autoHideTimer;
```

```

public static void Initialize(InfoBar infoBar, Grid rootGrid)
{
    _infoBar = infoBar;
    _rootGrid = rootGrid;
    // Инициализируем таймер
    _autoHideTimer = _infoBar.DispatcherQueue.CreateTimer();
    _autoHideTimer.Tick += (s, e) => HideNotification();
}

public static void ShowNotification(
    string message,
    string title = "",
    InfoBarSeverity severity = InfoBarSeverity.Informational,
    int? timeoutMs = null)
{
    if (_infoBar == null) return;

    _infoBar.DispatcherQueue.TryEnqueue(() =>
    {
        // Останавливаем предыдущий таймер (если был)
        _autoHideTimer!.Stop();
        _infoBar.Title = title;
        _infoBar.Message = message;
        _infoBar.Severity = severity;
        _infoBar.IsOpen = true;

        // Устанавливаем таймаут только для Informational и Success
        if (timeoutMs.HasValue ||
            severity == InfoBarSeverity.Informational ||
            severity == InfoBarSeverity.Success)
        {
            _autoHideTimer.Interval = TimeSpan.FromMilliseconds(timeoutMs ?? 5000);
            _autoHideTimer.Start();
        }
    });
}

public static void HideNotification()
{
    if (_infoBar == null) return;
    _infoBar.DispatcherQueue.TryEnqueue(() =>
    {
        _autoHideTimer!.Stop();
        _infoBar.IsOpen = false;
    });
}

// Удобные методы-обёртки для разных типов уведомлений
public static void ShowSuccess(string message, string title = "Успех")
    => ShowNotification(message, title, InfoBarSeverity.Success);
public static void ShowError(string message, string title = "Ошибка")
    => ShowNotification(message, title, InfoBarSeverity.Error);
public static void ShowWarning(string message, string title = "Внимание")
    => ShowNotification(message, title, InfoBarSeverity.Warning);
public static void ShowInfo(string message, string title = "Информация")
    => ShowNotification(message, title, InfoBarSeverity.Informational);
}

```

На рисунке 32 представлено как выглядит уведомление, которое вызывается через NotificationService.

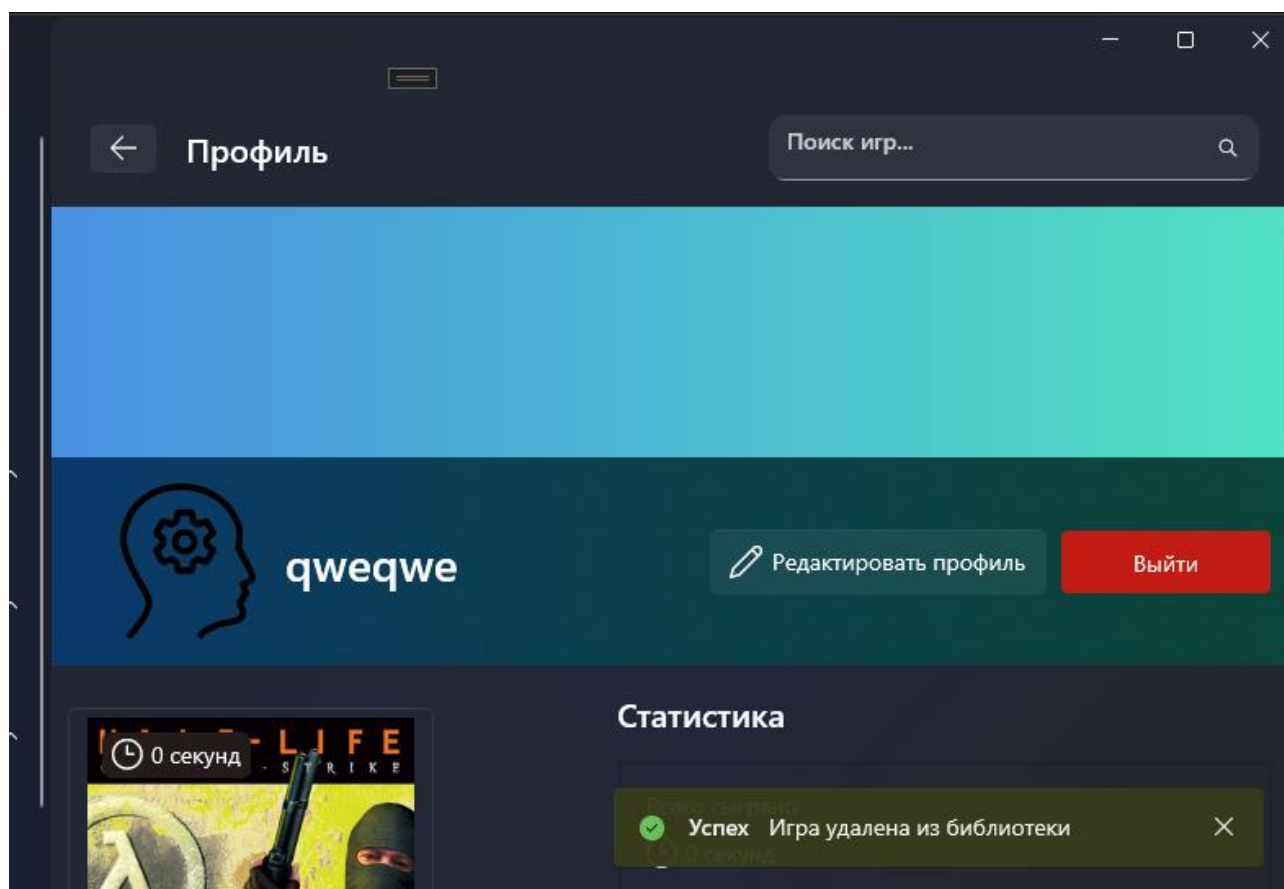


Рисунок 32 – Пример уведомления

Разработка страниц

При первом запуске приложения пользователь попадает на стартовое окно авторизации, где ему предлагаются четыре варианта действий: войти в существующий аккаунт (с помощью логина/почты и пароля), создать новый аккаунт, восстановить доступ через уникальный ключ (выданный при регистрации) или восстановить пароль через привязанную электронную почту (см. рис. 33). Этот интерфейс обеспечивает быстрый доступ ко всем основным сценариям работы с учётной записью, включая аварийное восстановление доступа, и служит точкой входа в систему для новых и существующих пользователей.

Страница регистрации предоставляет пользователю форму для ввода необходимых данных (логин, email, пароль) с возможностью создания нового аккаунта после заполнения всех полей; также на странице присутствует кнопка перехода на страницу входа для пользователей, у которых уже есть учётная запись (см. рис. 34), что обеспечивает удобное переключение между процессами регистрации и авторизации.

Рисунок 33 – Стартовое окно авторизации

Рисунок 34 – Окно регистрации аккаунта

Окно восстановления пароля с помощью ключа, выданного при регистрации, представлено на рисунке 35. Окно восстановления пароля через электронную почту представлено на рисунке 36.

Рисунок 35 – Окно восстановления пароля с помощью ключа

Рисунок 36 – Окно восстановления пароля через электронную почту

На рисунке 37 изображена главная страница приложения, которая отображает локальные сохранения пользователя, ожидающие синхронизации с облаком, для удобства

навигации рядом с каждой игрой размещена кнопка «Перейти к игре», позволяющая мгновенно открыть страницу выбранной игры без необходимости поиска через меню.

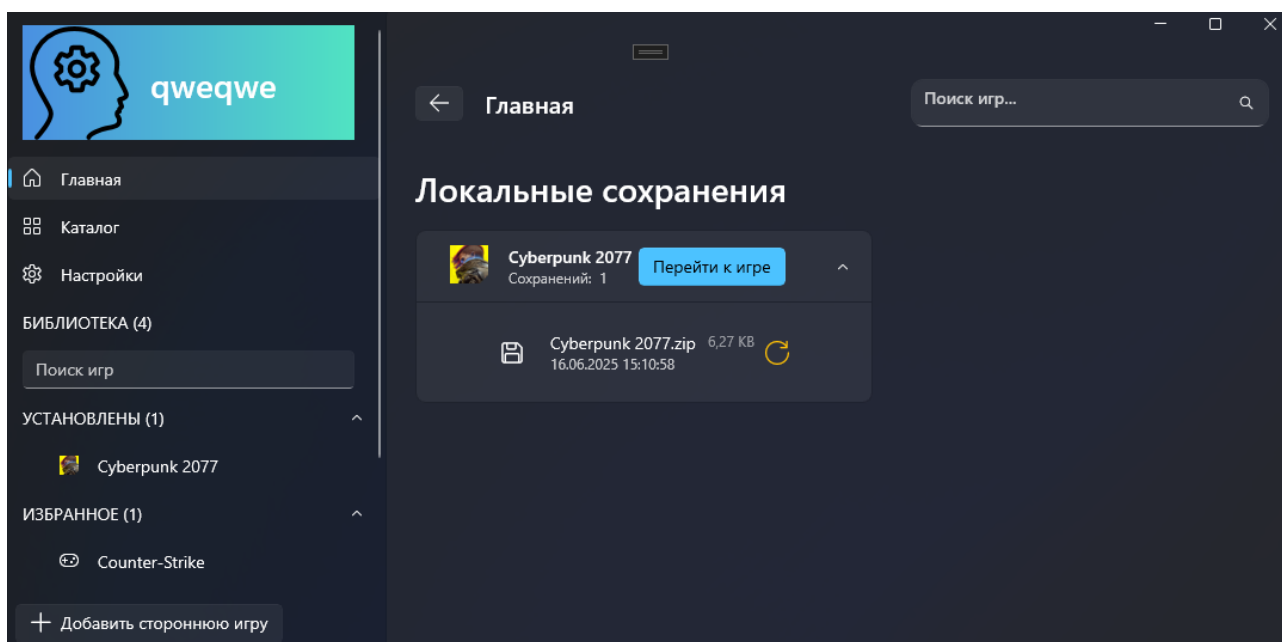


Рисунок 37 – Главная страница с локальными сохранениями

На рисунке 38 изображена главная страница с отсутствующими локальными сохранениями.

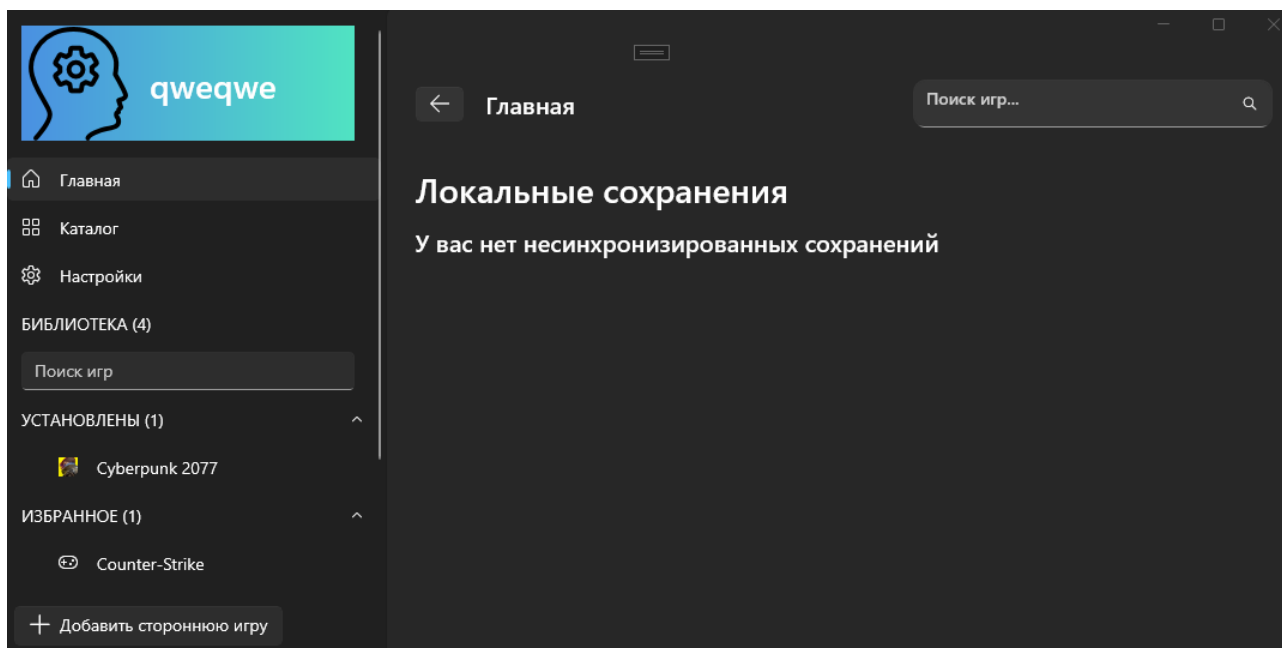


Рисунок 38 – Главная страница без локальных сохранений

На странице «Настройки» реализован многоуровневый интерфейс с двумя вкладками – «Главная» и «Аккаунт». В разделе «Главная» пользователь может выбрать путь сохранения файлов через диалоговое окно выбора директории, а также управлять автоматической очисткой локальных данных после синхронизации с помощью чекбокса (см. рис. 39).

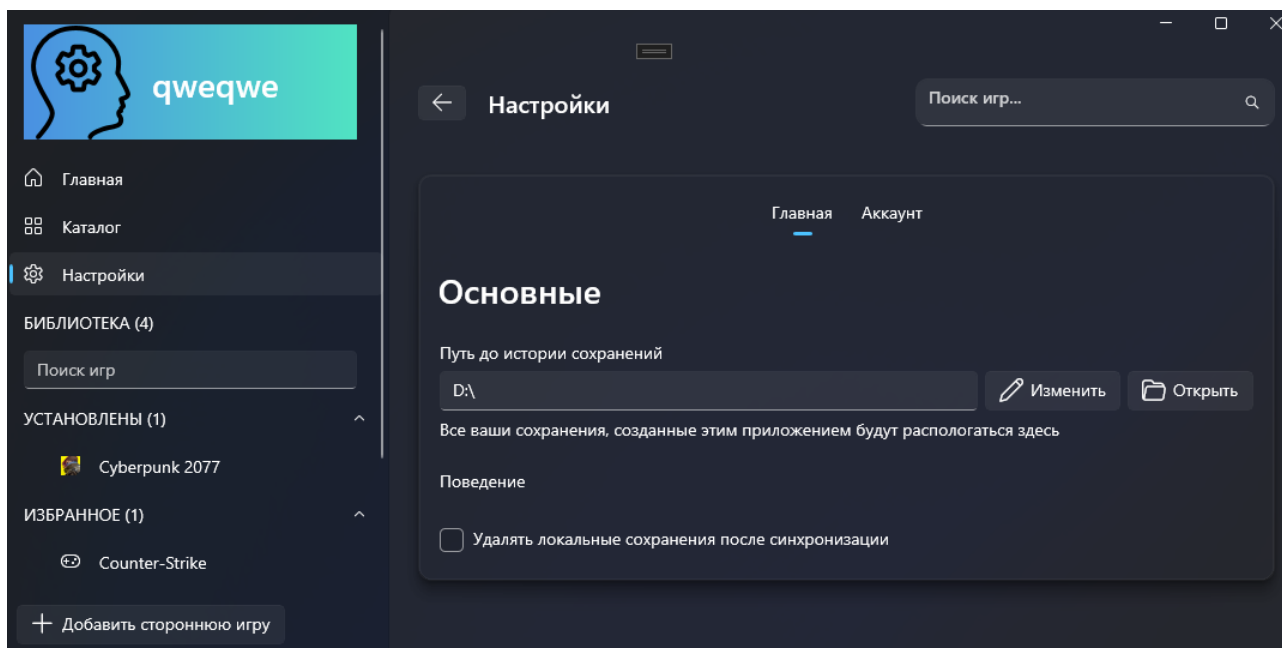


Рисунок 39 – Страница настройки-главная

В разделе «Аккаунт» располагается информация о том, привязана ли электронная почта, облачный сервис, а также кнопки для редактирования электронной почты и обновления пароля (см. рис. 40). Если нажать на кнопку «Обновить электронную почту», то открывается модальное окно, в котором показывается информация о пользователе и поле для ввода электронной почты (см. рис. 41). Пользователь может, не переключаясь, нажать на «Хотите обновить пароль» и модальное окно заменится на «Обновление пароля» (см. рис. 42).

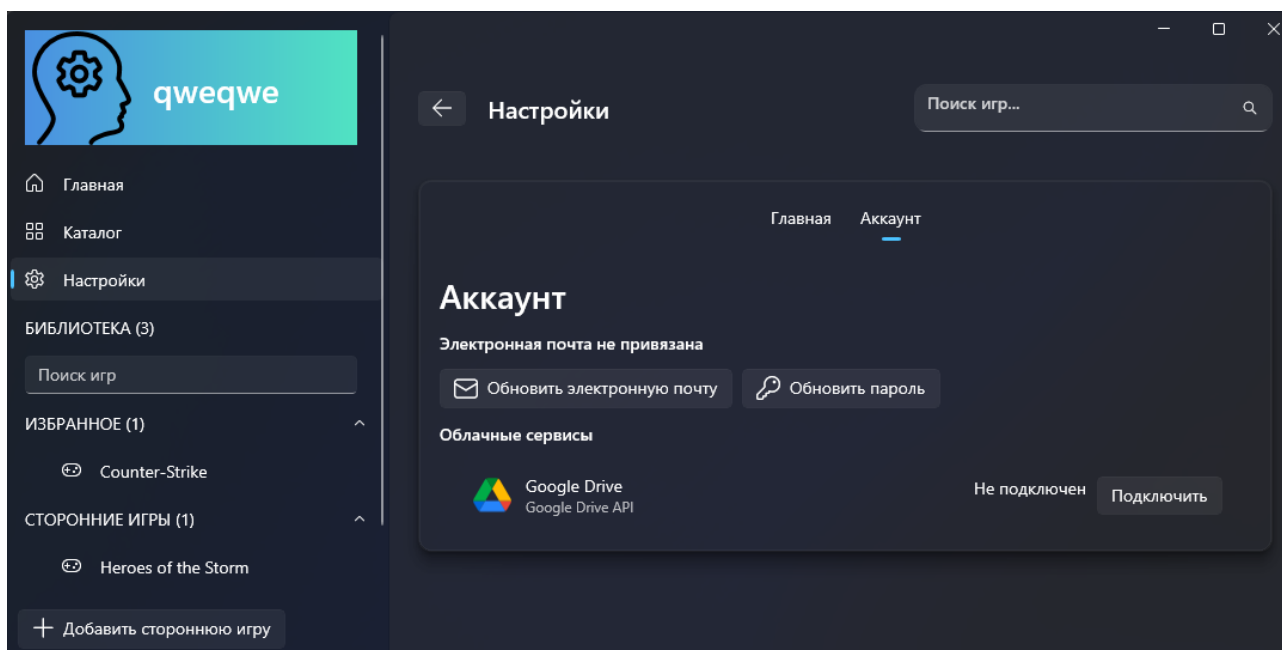


Рисунок 40 – Страница настройки-аккаунт

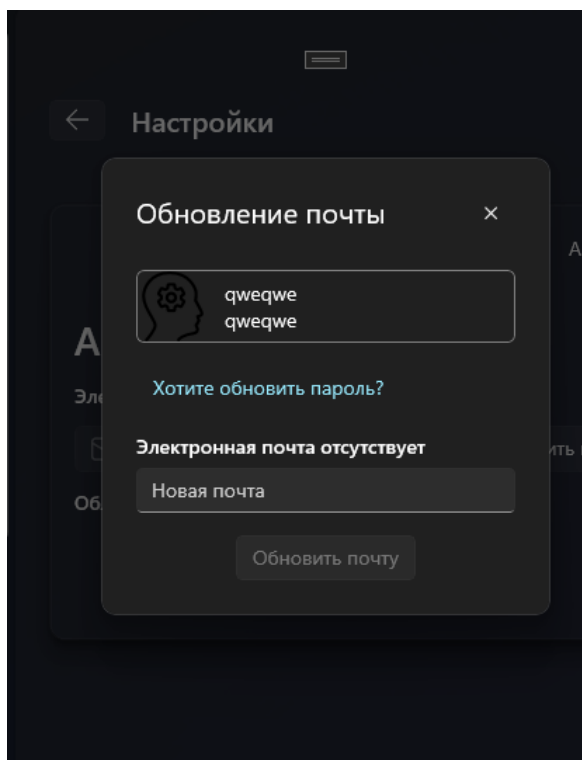


Рисунок 41 – Модальное окно обновления почты

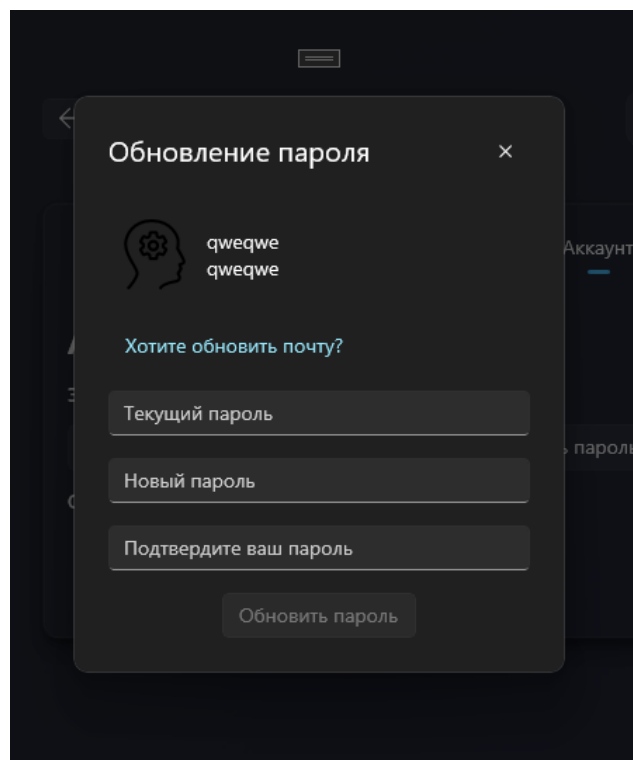


Рисунок 42 – Модальное окно обновления пароля

На рисунке 43 представлена страница «Каталог», которая предоставляет пользователю удобный доступ к списку доступных игр, где каждая игра представлена в виде интерактивного элемента — при нажатии на неё происходит переход на соответствующую страницу с подробной информацией. Для быстрого поиска нужной игры в интерфейсе реализовано поисковое поле с функцией автодополнения: при вводе даже нескольких букв система мгновенно фильтрует содержимое каталога и отображает релевантные результаты, что значительно ускоряет навигацию.

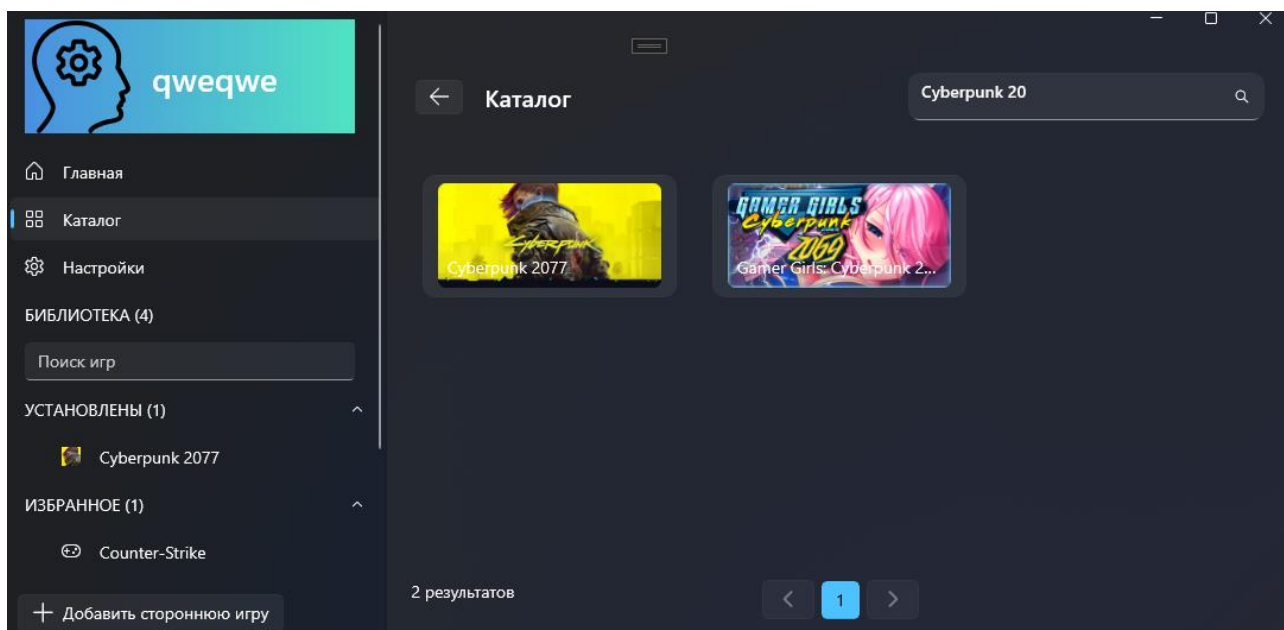


Рисунок 43 – Страница «Каталог»

На странице с профилем вверху располагается хедер, шапка и никнейм пользователя, а также кнопки выхода из аккаунта и редактирования профиля. В основном контенте располагается статистика, недавняя активность и полностью вся библиотека пользователя, на все карточки с играми можно нажать и перейти на страницу с игрой.

На рисунке 44 представлена страница профиля пользователя, которая имеет четкую структуру: в верхней части расположены хедер с шапкой-обложкой, отображающий никнейм пользователя, а также функциональные кнопки «Выйти» и «Редактировать профиль» для управления учетной записью. Основной раздел страницы содержит три ключевых блока: статистику игровой активности (общее время), ленту недавних действий (последние сыгранные игры) и полную библиотеку игр пользователя, где каждая игра представлена в виде интерактивной карточки с возможностью перехода на её страницу по клику.

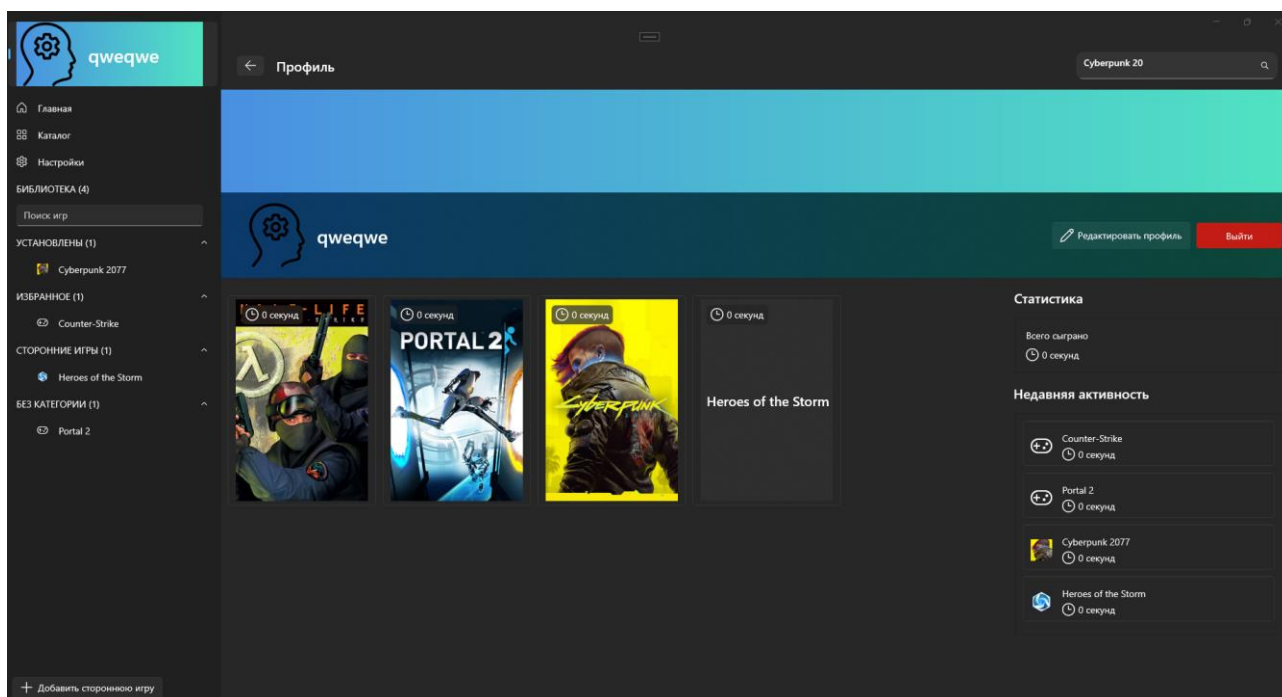


Рисунок 44 – Страница «Профиль»

При нажатии на «Редактировать профиль» открывается модальное окно, где пользователь может изменить аватар, хедер и отображаемое имя. Пользователь может выбрать как он хочет загрузить изображение: загрузить картинку со своего компьютера или вставить ссылку на изображение из интернета (см. рис. 45).

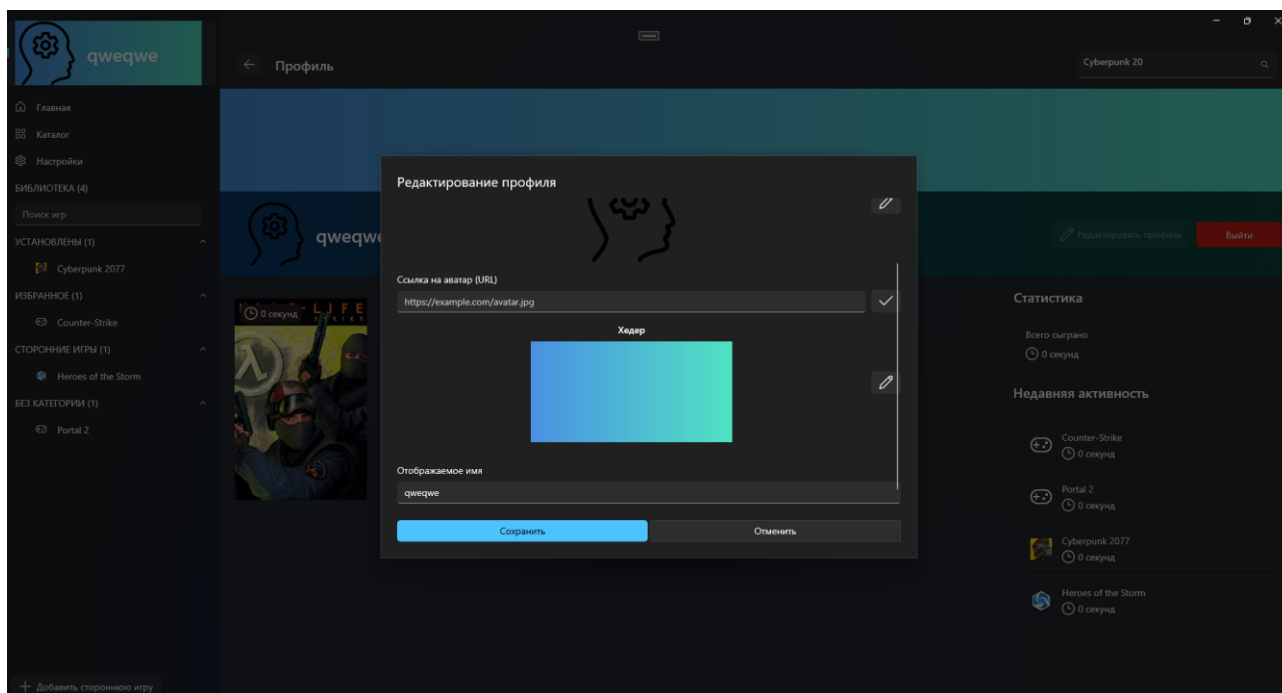


Рисунок 45 – Модальное окно редактирования профиля

Страница игры представляет собой детализированную панель управления, где в шапке отображаются основные визуальные элементы: название игры, хедер и логотип, кнопки «Играть», «Избранное», «Настройки» и информация об общем времени игры, дата последнего запуска (см. рис. 46).

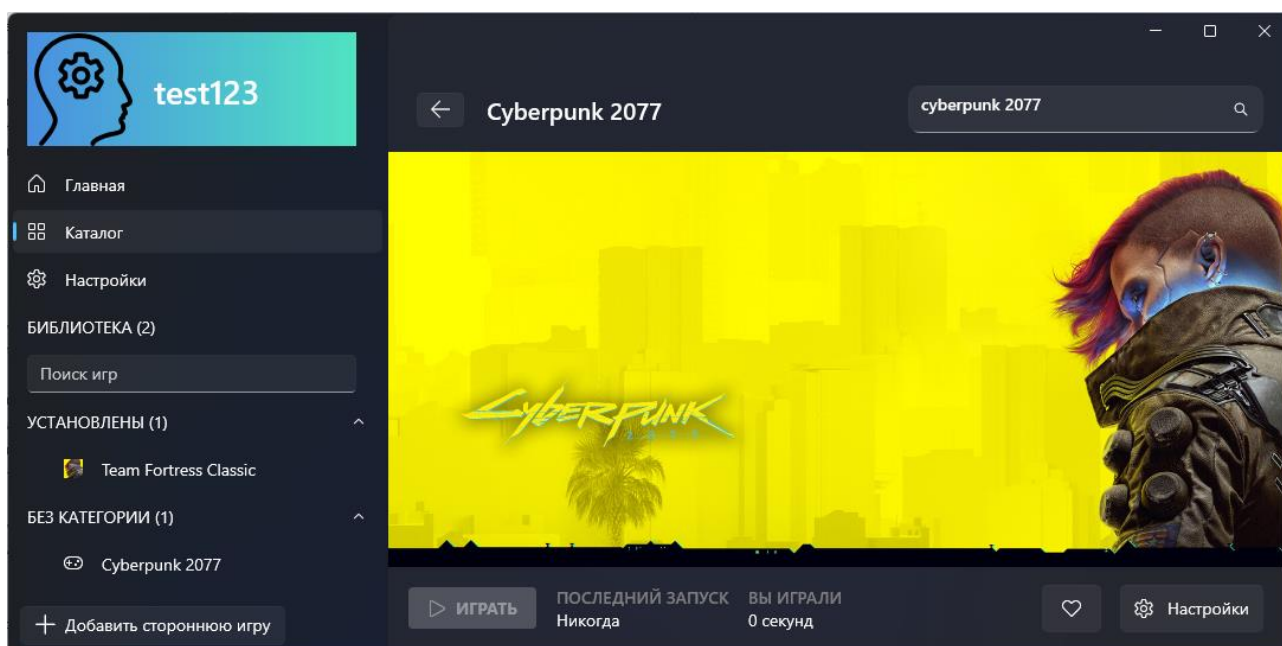


Рисунок 46 – Страница игры

При нажатии на кнопку «Настройки» появляется диалоговое окно, в котором пользователь может выбрать путь к исполняемому файлу игры, путь до папки сохранений игры, если она автоматически не была найдена (см. рис. 47). Есть кнопка удаления из библиотеки, при нажатии на которую игра удалится из библиотеки, но сохранения к игре останутся.

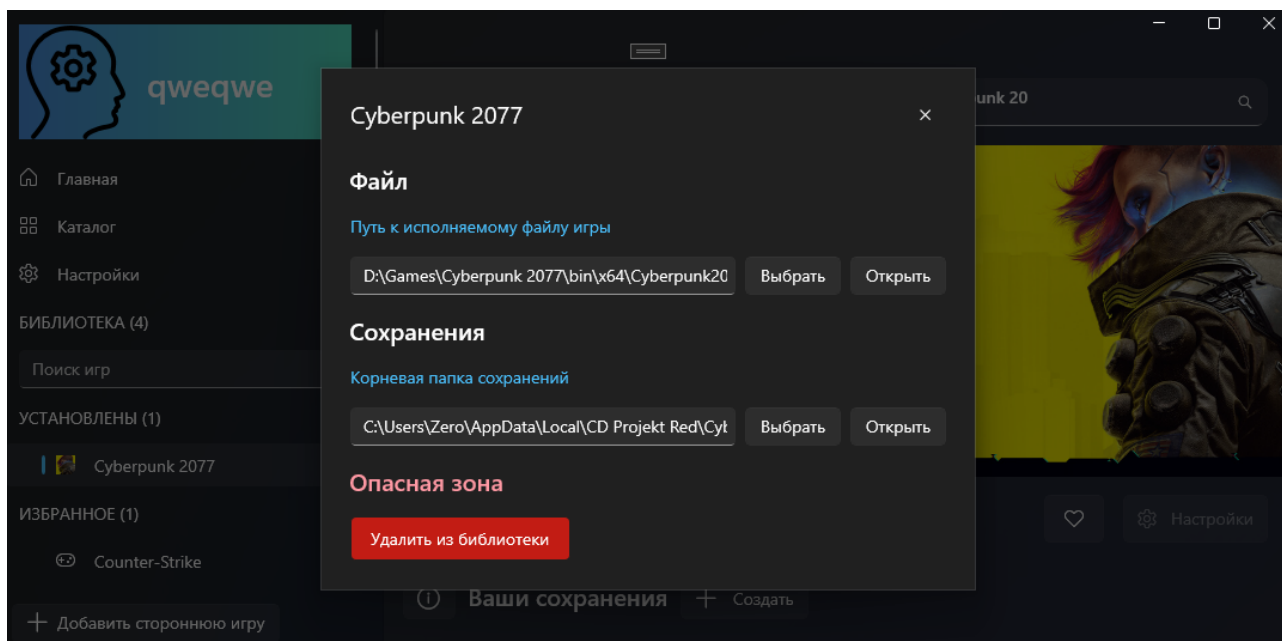


Рисунок 47 – Модальное окно настроек игры

Также на странице с игрой пользователь может создать архив папки сохранений, если он указал эту папку в настройках, при создании пользователю предлагается написать версию и описание сохранения (см. рис. 48).

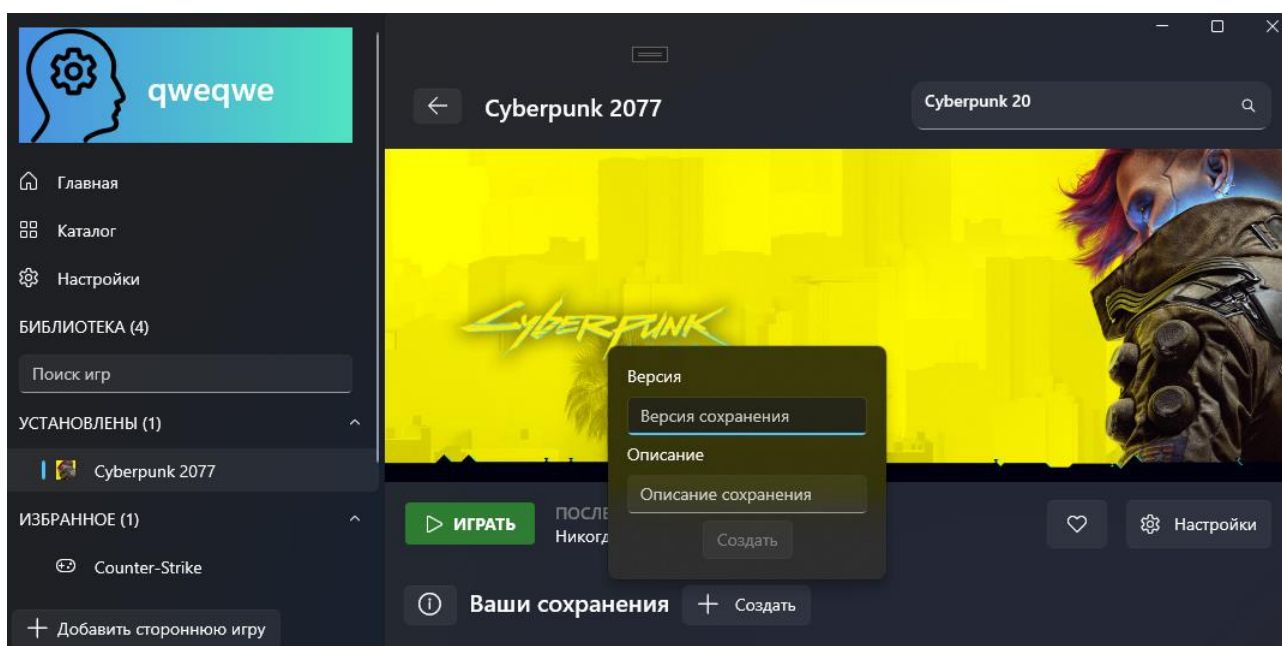


Рисунок 48 – Создание сохранения

На рисунке 49 показано как выглядит сохранение и какие функции предоставлены пользователю для взаимодействия с сохранениями.

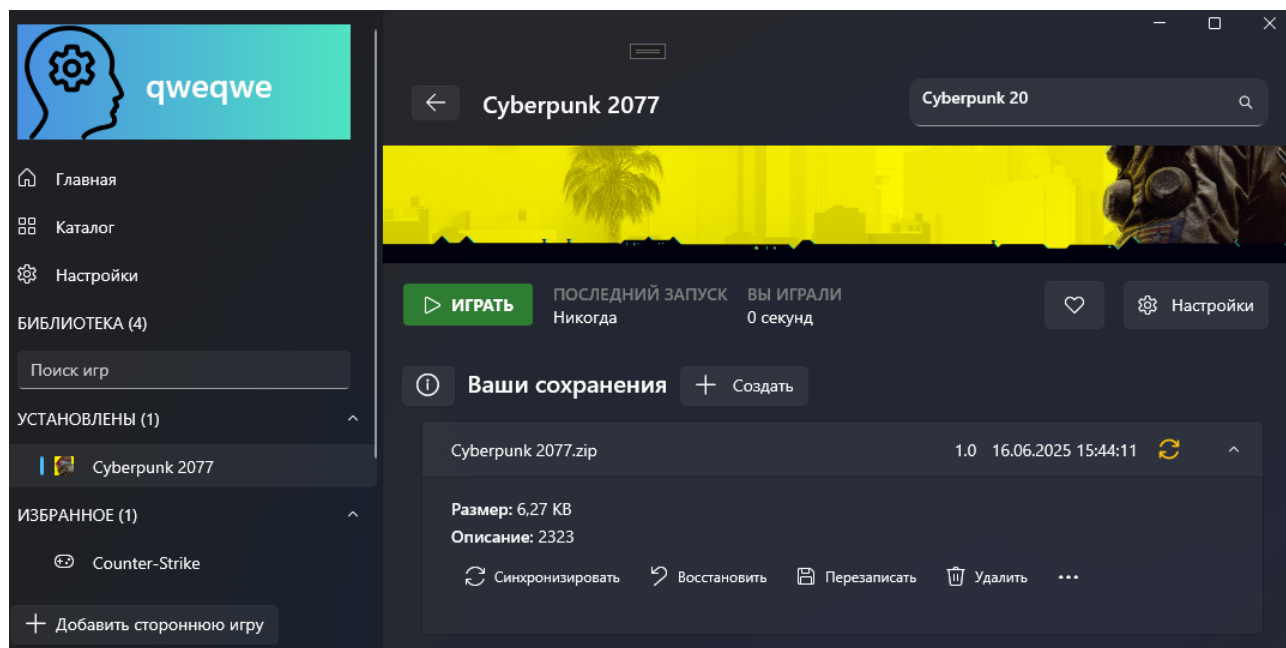


Рисунок 49 – Действия с сохранением

ГЛАВА 5. ТЕСТИРОВАНИЕ

Тестирование серверной части

В ходе разработки системы было проведено автоматизированное тестирование API с использованием Postman. Для проверки корректности работы эндпоинтов были созданы тестовые сценарии, охватывающие основные аспекты взаимодействия с сервером.

Тестирование включало в себя:

- Проверку корректности обработки запросов (GET, POST, DELETE).
- Оценку соответствия возвращаемых данных ожидаемому формату.
- Тестирование обработки ошибок и валидации данных.
- Измерение времени отклика API.

Разработанные тесты позволили выявить и устранить возможные ошибки на ранних этапах разработки, обеспечив стабильность и предсказуемость работы сервиса.

200 OK

В листинге 50 представлен тест, проверяющий успешную обработку запроса сервером, где ожидается код состояния 200 (OK), подтверждающий корректное выполнение операции. Дополнительно тест включает проверку времени отклика сервера, которое должно быть не более 500 мс для соблюдения требований к производительности.

Листинг 50 – Тест 200 OK

```
pm.test("Проверка кода ответа", function () {  
    pm.response.to.have.status(200);  
});  
pm.test("Проверка времени ответа", function () {  
    pm.expect(pm.response.responseTime).to.be.below(500);  
});
```

Ресурсы:

Метод	Эндпоинт
GET	/profile
POST	/profile
GET	/profile/statistic
GET	/profile/services
GET	/library
GET	/games
GET	/games/{game}

GET	/games/{game}/path
GET	/sidegames/{sideGame}
GET	/saves/game/{game}/my
GET	/saves/sidegame/{sideGame}/my
PATCH	/saves/{save}
POST	/saves/google-drive/generate-upload-url
POST	/saves/{save}/google-drive/confirm-upload
GET	/saves/{save}/google-drive/download
GET	/saves/{save}/google-drive/share
POST	/saves/{save}/google-drive/generate-overwrite-url
GET	/google-drive/auth-url

201 Created

В листинге 51 представлен тест, проверяющий успешное создание объекта на сервере, о чем свидетельствует ожидаемый код состояния 201 (Created). Тест также включает проверку времени отклика сервера, которое не должно превышать 500 мс для соответствия требованиям производительности.

Листинг 51 – Тест 201 Created

```
pm.test("Проверка кода ответа", function () {
    pm.response.to.have.status(201);
});

pm.test("Проверка времени ответа", function () {
    pm.expect(pm.response.responseTime).to.be.below(500);
});
```

Ресурсы:

Метод	Эндпоинт
POST	/register
POST	/library/game/{game}
POST	/sidegames

204 No Content

В листинге 52 представлен тест, проверяющий корректную обработку запроса сервером с возвратом статуса 204 (No Content), что указывает на успешное выполнение операции без возвращаемого тела ответа. Тест включает три ключевые проверки: соответствие кода состояния 204, время отклика сервера не более 500 мс (проверка производительности), а также обязательное отсутствие содержимого в теле ответа (пустая строка).

Листинг 52 – Тест 204 No Content

```
pm.test("Проверка кода ответа", function () {  
    pm.response.to.have.status(204);  
});  
pm.test("Проверка времени ответа", function () {  
    pm.expect(pm.response.responseTime).to.be.below(500);  
});  
pm.test("Проверка, что тело ответа пустое", function () {  
    pm.expect(pm.response.text()).to.equal("");  
});
```

Ресурсы:

Метод	Эндпоинт
POST	/logout
POST	/restore-from-key
POST	/mail/send
PATCH	/library/game/{game}
PATCH	/library/game/{game}/update
DELETE	/library/game/{game}
PATCH	/library/sidegame/{sideGame}
PATCH	/library/sidegame/{sideGame}/update
DELETE	/sidegames/{sideGame}
DELETE	/saves/{save}/google-drive/delete
DELETE	/google-drive/disconnect/{userCloudService}

401 Unauthorized

В листинге 53 представлен тест, проверяющий обработку сервером запроса без авторизации или с некорректным Bearer Token, где ожидается код состояния 401 (Unauthorized), сигнализирующий об отсутствии аутентификации. Тест включает три проверки: соответствие статус-кода 401, время отклика сервера не более 500 мс (для оценки производительности), а также наличие в ответе ключа «message» с описанием ошибки (для информативности).

Листинг 53 – Тест 401 Unauthorized

```
pm.test("Проверка кода ответа", function () {  
    pm.response.to.have.status(401);
```

```

});
pm.test("Проверка времени ответа", function () {
    pm.expect(pm.response.responseTime).to.be.below(500);
});
pm.test("Проверка наличия ключа 'message'", function () {
    pm.expect(pm.response.json()).to.have.property("message");
});

```

Ресурсы:

Метод	Эндпоинт
POST	/logout
GET	/profile
POST	/profile
GET	/profile/statistic
GET	/profile/services
GET	/library
POST	/library/game/{game}
PATCH	/library/game/{game}
PATCH	/library/game/{game}/update
DELETE	/library/game/{game}
PATCH	/library/sidegame/{sideGame}
PATCH	/library/sidegame/{sideGame}/update
GET	/games
GET	/games/{game}
GET	/games/{game}/path
POST	/sidegames
GET	/sidegames/{sideGame}
DELETE	/sidegames/{sideGame}
GET	/saves/game/{game}/my
GET	/saves/sidegame/{sideGame}/my
PATCH	/saves/{save}
POST	/saves/google-drive/generate-upload-url
POST	/saves/{save}/google-drive/confirm-upload

GET	/saves/{save}/google-drive/download
GET	/saves/{save}/google-drive/share
DELETE	/saves/{save}/google-drive/delete
POST	/saves/{save}/google-drive/generate-overwrite-url
GET	/google-drive/auth-url
DELETE	/google-drive/disconnect/{userCloudService}

403 Forbidden

В листинге 54 представлен тест для проверки обработки сервером запроса с недостаточными правами доступа (403 Forbidden), который включает три ключевые проверки: соответствие кода состояния 403, подтверждающего запрет доступа к ресурсу; контроль времени отклика (не более 500 мс) для соблюдения требований производительности; а также наличие в ответе сервера ключа «message», содержащего описание причины отказа.

Листинг 54 – Тест 403 Forbidden

```
pm.test("Проверка кода ответа", function () {
  pm.response.to.have.status(403);
});
pm.test("Проверка времени ответа", function () {
  pm.expect(pm.response.responseTime).to.be.below(500);
});
pm.test("Проверка наличия ключа 'message'", function () {
  pm.expect(pm.response.json()).to.have.property("message");
});
```

Ресурсы:

Метод	Эндпоинт
PATCH	/saves/{save}
POST	/saves/{save}/google-drive/confirm-upload
GET	/saves/{save}/google-drive/download
GET	/saves/{save}/google-drive/share
DELETE	/saves/{save}/google-drive/delete
DELETE	/google-drive/disconnect/{userCloudService}

404 Not Found

В листинге 55 представлен тест для проверки обработки запроса на несуществующий ресурс (404 Not Found), который включает три основные проверки: первая проверяет, что

сервер возвращает код состояния 404, указывающий на отсутствие запрашиваемого ресурса; вторая контролирует время отклика сервера, которое должно быть не более 500 мс для обеспечения приемлемой производительности; третья проверяет наличие в ответе сервера ключа «message», который обычно содержит описание ошибки, что важно для информирования пользователя о причине недоступности ресурса.

Листинг 55 – Тест 404 Not Found

```
pm.test("Проверка кода ответа", function () {
  pm.response.to.have.status(404);
});

pm.test("Проверка времени ответа", function () {
  pm.expect(pm.response.responseTime).to.be.below(500);
});

pm.test("Проверка наличия ключа 'message'", function () {
  pm.expect(pm.response.json()).to.have.property("message");
});
```

Ресурсы:

Метод	Эндпоинт
POST	/library/game/{game}
PATCH	/library/game/{game}
PATCH	/library/game/{game}/update
DELETE	/library/game/{game}
PATCH	/library/sidegame/{sideGame}
PATCH	/library/sidegame/{sideGame}/update
GET	/games/{game}
GET	/games/{game}/path
GET	/sidegames/{sideGame}
DELETE	/sidegames/{sideGame}
GET	/saves/game/{game}/my
GET	/saves/sidegame/{sideGame}/my
PATCH	/saves/{save}
POST	/saves/{save}/google-drive/confirm-upload
GET	/saves/{save}/google-drive/download
GET	/saves/{save}/google-drive/share
DELETE	/saves/{save}/google-drive/delete

POST	/saves/{save}/google-drive/generate-overwrite-url
DELETE	/google-drive/disconnect/{userCloudService}

422 Unprocessable Content

В листинге 56 представлен тест для проверки обработки сервером некорректных данных (422 Unprocessable Content), который включает четыре основные проверки: соответствие кода состояния 422, указывающего на ошибки валидации введенных данных; контроль времени отклика сервера (не более 500 мс); наличие в ответе ключа «message» с описанием ошибки; а также проверку наличия ключа «errors», содержащего детализированную информацию о полях с ошибками валидации.

Листинг 56 – Тест 422 Unprocessable Content

```
pm.test("Проверка кода ответа", function () {
  pm.response.to.have.status(422);
});
pm.test("Проверка времени ответа", function () {
  pm.expect(pm.response.responseTime).to.be.below(500);
});
pm.test("Проверка наличия ключа 'message'", function () {
  pm.expect(pm.response.json()).to.have.property("message");
});
pm.test("Проверка наличия ключа 'errors'", function () {
  pm.expect(pm.response.json()).to.have.property("errors");
});
```

Ресурсы:

Метод	Эндпоинт
POST	/register
POST	/login
POST	/restore-from-key
POST	/mail/send
POST	/mail/verify
POST	/mail/restore
POST	/profile
POST	/sidegames
PATCH	/library/game/{game}/update

PATCH	/library/sidegame/{sideGame}/update
PATCH	/saves/{save}
POST	/saves/google-drive/generate-upload-url
POST	/saves/{save}/google-drive/confirm-upload
POST	/saves/{save}/google-drive/generate-overwrite-url

Результаты тестирования серверной части

В ходе тестирования API с помощью Postman большинство тестов были успешно пройдены (см. рис. 50).

Tests - Run results Run Again Automate Run + New Run Export Results

Ran today at 09:33:25 · [View all runs](#)

Source	Environment	Iterations	Duration	All tests	Avg. Resp. Time
Runner	My	1	48s 663ms	249	285 ms

All Tests Passed (235) Failed (14) Skipped (0) View Summary

DELETE 404
<https://savayaqu.duckdns.org/playsaveback/api/sidegames/1> 404 • 253 ms • 318 B

- PASS Проверка кода ответа
- PASS Проверка времени ответа
- PASS Проверка наличия ключа 'message'

POST 201
<https://savayaqu.duckdns.org/playsaveback/api/library/game/2> 201 • 260 ms • 812 B

- PASS Проверка кода ответа
- PASS Проверка времени ответа

POST 401
<https://savayaqu.duckdns.org/playsaveback/api/library/game/2> 401 • 224 ms • 303 B

- PASS Проверка кода ответа
- PASS Проверка времени ответа
- PASS Проверка наличия ключа 'message'

Рисунок 50 – Результаты тестирования API в Postman

Тестирование десктопного приложения

После завершения разработки десктопного приложения «PlaySaveBack» было проведено ручное тестирование, целью которого являлась проверка корректности отображения контента, логики работы и взаимодействия с серверной частью через RESTful API, благодаря разработанным тест-кейсам. В таблице 2 приведены разработанные тест-кейсы для проверки функциональности системы.

Таблица 2 – Тест-кейсы

№	Сценарий	Входные данные	Этапы тестирования	Ожидаемый результат	Фактический результат
1	Регистрация с валидными данными	Логин: qweqwe123 Email: admin@mail.ru Пароль: Admin123! Подтверждение пароля: Admin123!	Заполнить форму регистрации, нажать кнопку «Зарегистрироваться»	Показ модального окна с ключом восстановления	Показ модального окна с ключом восстановления
2	Регистрация с не валидными данными	Логин: qweqwe123 Email: 123 Пароль: Admin123! Подтверждение пароля: Admin123!	Заполнить форму регистрации, нажать кнопку «Зарегистрироваться»	Показ под полем Email ошибки «The email field must be a valid email address.»	Показ под полем Email ошибки «The email field must be a valid email address.»
3	Авторизация с верными данными	Логин: qweqwe123 Пароль: Admin123!	Заполнить форму авторизацию, нажать кнопку «Войти»	Редирект на главную страницу приложения	Редирект на главную страницу приложения
4	Авторизация с неверными данными	Логин: qweqwe123 Пароль: Password111	Заполнить форму авторизацию, нажать кнопку «Войти»	Показ ошибки валидации данных «Invalid credentials»	Показ ошибки валидации данных «Invalid credentials»
5	Переход на страницу профиля		Нажать на элемент со своим никнеймом	Показ страницы профиля	Показ страницы профиля
6	Переход на страницу каталога		Нажать на элемент «Каталог»	Показ страницы «Каталог»	Показ страницы «Каталог»
7	Переход на страницу каталога через «Поиск игр...»	Cyberpunk 2077	Заполнить поле входными данными	Редирект на каталог и показ найденной игры	Редирект на каталог и показ найденной игры
8	Открытие модального окна редактирования профиля		Нажать на кнопку «Редактировать профиль»	Показ модального окна редактирования профиля с отображением текущего хедера, аватара и никнейма.	Показ модального окна редактирования профиля с отображением текущего хедера, аватара и никнейма.
9	Редактирование профиля	Отображаемое имя: Blessed	Заполнить форму, нажать «Сохранить»	Заккрытие модального окна, изменение отображения	Заккрытие модального окна, изменение отображения данных

				данных пользователя на странице профиля	пользователя на странице профиля
10	Выход из аккаунта		На странице профиля нажать кнопку «Выйти»	Закрытие текущего окна и открытие окна авторизации	Закрытие текущего окна и открытие окна авторизации
11	Редактирование пути до истории сохранений		На странице «Настройки» в разделе «Главная» нажать кнопку «Изменить»	Открытие модального окна с выбором папок, изменения отображения пути	Открытие модального окна с выбором папок, изменения отображения пути
12	Открытие модального окна обновления электронной почты		На странице «Настройки» в разделе «Аккаунт» нажать кнопку «Обновить электронную почту»	Открытие модального окна «Обновление почты»	Открытие модального окна «Обновление почты»
13	Переход из модального окна обновления электронной почты в модальное окно изменения пароля		В модальном окне нажать «Хотите обновить пароль?»	Замена текущего модального окна на модальное окно изменения пароля	Замена текущего модального окна на модальное окно изменения пароля
14	Подключение облачного сервиса		На странице «Настройки» в разделе «Аккаунт» нажать кнопку напротив Google Drive «Подключить»	Открытие ссылки в браузере на авторизацию в Google, после авторизации изменение статуса и кнопки	Открытие ссылки в браузере на авторизацию в Google, после авторизации изменение статуса и кнопки
15	Отключение облачного сервиса		Напротив подключенного облачного сервиса нажать кнопку «Отключить»	Статус и кнопка меняются на «Не подключен» и «Подключить»	Статус и кнопка меняются на «Не подключен» и «Подключить»
16	Добавление сторонней игры		В левом меню навигации нажать кнопку «Добавить стороннюю игру»	Добавление игры в левое меню с названием выбранного файла	Добавление игры в левое меню с названием выбранного файла
17	Добавление сторонней игры, которая уже находится в библиотеке		В левом меню навигации нажать кнопку «Добавить стороннюю игру»	Показывается уведомление об ошибке красного цвета в правом нижнем углу приложения	Показывается уведомление об ошибке красного цвета в правом нижнем углу приложения
18	Добавление игры в библиотеку		На странице с игрой, которой нет в библиотеке, нажать кнопку «Добавить в библиотеку»	Показывается уведомление зеленого цвета об успехе, добавляется игра в левое меню навигации.	Показывается уведомление зеленого цвета об успехе, добавляется игра в левое меню навигации.

19	Запуск игры без указания исполняемого файла		На странице с игрой, которая есть в библиотеке, нажать кнопку «Играть»	Кнопка серого цвета не нажимается и ничего не происходит	Кнопка серого цвета не нажимается и ничего не происходит
20	Добавление игры в избранное		На странице с игрой, которая есть в библиотеке, нажать кнопку с символом пустого сердца	Символ у кнопки меняется на заполненное сердце, меняется тип игры в левом навигационном меню	Символ у кнопки меняется на заполненное сердце, меняется тип игры в левом навигационном меню
21	Удаление игры из избранного		На странице с игрой, которая есть в библиотеке, нажать кнопку с символом заполненного сердца	Символ у кнопки меняется на пустое сердце, меняется тип игры в левом навигационном меню	Символ у кнопки меняется на пустое сердце, меняется тип игры в левом навигационном меню
22	Открытие модального окна настроек игры		На странице с игрой, которая есть в библиотеке, нажать кнопку «Настройки»	Открытие модального окна с названием игры, выбором файла и пути до папки сохранений.	Открытие модального окна с названием игры, выбором файла и пути до папки сохранений.
23	Выбор исполняемого файла игры		В модальном окне настроек игры напротив поля «Файл» нажать кнопку «Выбрать» и выбрать файл с расширением .exe	Путь до файла изменился на указанный	Путь до файла изменился на указанный
24	Выбор пути до папки сохранения игры		В модальном окне настроек игры напротив поля «Сохранения» нажать кнопку «Выбрать» и выбрать папку	Путь до папки изменился на указанный	Путь до папки изменился на указанный
25	Удаление игры из библиотеки		В модальном окне настроек игры нажать кнопку «Удалить из библиотеки», подтвердить действие	Модальное окно закрывается, редирект на страницу профиля, показ уведомления зеленого цвета об успехе	Модальное окно закрывается, редирект на страницу профиля, показ уведомления зеленого цвета об успехе
26	Запуск игры с указанным исполняемым файлом		Нажать зеленую кнопку «Играть»	Открывается указанный исполняемый файл, кнопка «Играть» становится серого цвета и перестает нажиматься	Открывается указанный исполняемый файл, кнопка «Играть» становится серого цвета и перестает нажиматься

27	Заккрытие исполняемого файла, запущенного через приложение		Заккрыть исполняемый файл, запущенный через приложение	Меняются данные полей «Последний запуск» и «Вы играли»	Меняются данные полей «Последний запуск» и «Вы играли»
28	Показ модального окна при создании сохранения		На странице с игрой нажать кнопку «Создать»	Открывается модальное окно с полями «Версия» и «Описание» с кнопкой «Создать»	Открывается модальное окно с полями «Версия» и «Описание» с кнопкой «Создать»
29	Создание сохранения к игре без указания папки до сохранения	Версия: 1 Описание:	На странице с игрой нажать кнопку «Создать», в появившемся окне ввести данные	Кнопка «Создать» в появившемся окне неактивна	Кнопка «Создать» в появившемся окне неактивна
30	Создание сохранения к игре с указанием папки до сохранения	Версия: 1 Описание:	На странице с игрой нажать кнопку «Создать», в появившемся окне ввести данные и нажать кнопку «Создать»	На странице с игрой появилось сохранение	На странице с игрой появилось сохранение
31	Синхронизация сохранения		На странице с игрой в раскрытом сохранении нажать кнопку «Синхронизировать»	Появилось окно с выбором сервиса для синхронизации	Появилось окно с выбором сервиса для синхронизации
32	Удаление сохранения		На странице с игрой в раскрытом сохранении нажать кнопку «Удалить»	Сохранение исчезло, появилось уведомление зеленого цвета об успехе	Сохранение исчезло, появилось уведомление зеленого цвета об успехе
33	Поделиться синхронизированным сохранением		На странице с игрой в раскрытом синхронизированном сохранении нажать кнопку «Поделиться»	Появилось уведомление зеленого цвета об успехе, ссылка скопировалась в буфер обмена	Появилось уведомление зеленого цвета об успехе, ссылка скопировалась в буфер обмена

ГЛАВА 6. РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ ИНФОРМАЦИОННОГО ПРОДУКТА

Регистрация

1. Перейдите на страницу регистрации (см. рис. 34).
2. Введите корректные данные.
3. Нажмите зарегистрироваться.
4. Запишите выданный ваш ключ (KEY) в надежное место (он понадобится для восстановления аккаунта).

Авторизация

1. Перейдите на страницу входа (см. рис. 33).
2. Введите в поля свои данные, которые вы использовали для регистрации.
3. Нажмите кнопку «Войти».

Восстановление аккаунта через ключ, выданный при регистрации

1. На странице входа в аккаунт нажмите кнопку «Забыли пароль?» в выпадающем списке нажмите кнопку «Восстановить с помощью ключа» (см. рис. 33).
2. Заполните форму своими данными (см. рис. 35).
3. Нажмите кнопку «Восстановить пароль».

Восстановление аккаунта через электронную почту

1. На странице входа в аккаунт нажмите кнопку «Забыли пароль?» в выпадающем списке нажмите кнопку «Восстановить через email» (см. рис. 33).
2. Введите электронную почту, которую указали для аккаунта (см. рис. 36).
3. Нажмите кнопку «Отправить код».
4. Проверьте указанную почту, на неё должно прийти сообщение от «PlaySaveBack» с кодом восстановления.
5. Скопируйте код и введите его в приложении.
6. Нажмите кнопку «Подтвердить».
7. Заполните форму нового пароля.
8. Нажмите кнопку «Восстановить пароль».

Добавление игры в библиотеку

1. Перейдите на страницу с игрой.
2. Нажмите кнопку «Добавить в библиотеку» (см. рис. 51).

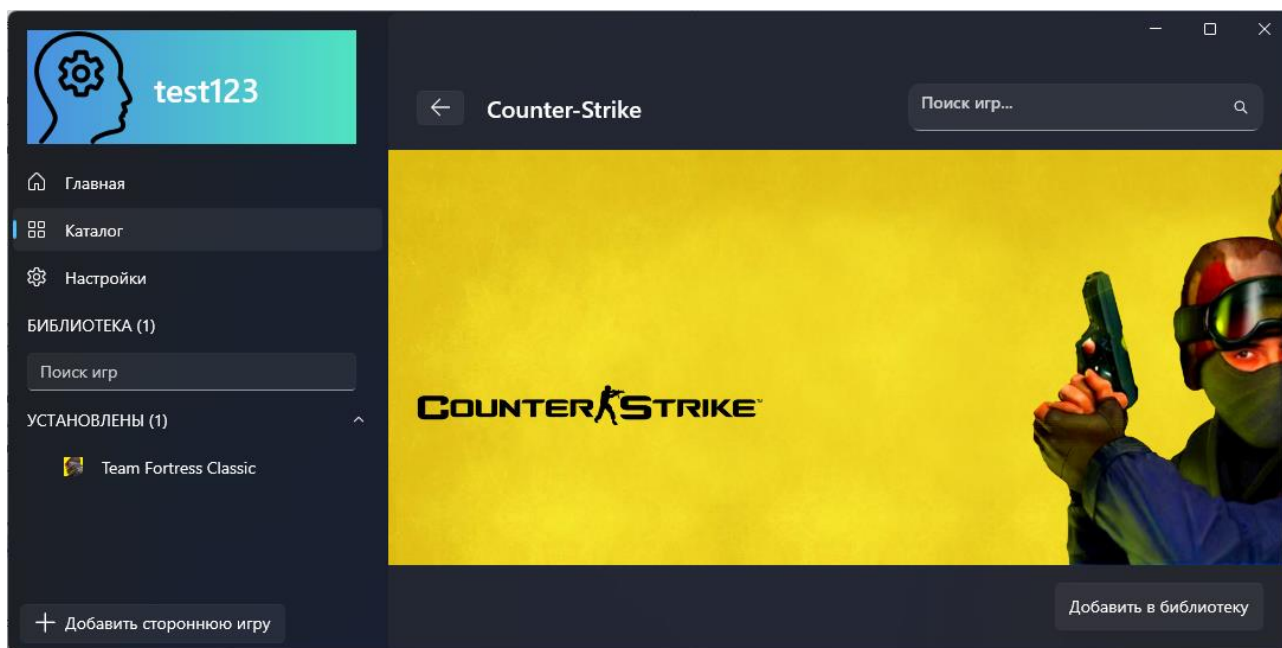


Рисунок 51 – Добавление игры в библиотеку

Добавление сторонней игры в библиотеку

1. В навигационном меню слева нажмите кнопку «Добавить стороннюю игру» (см. рис. 30).
2. В открывшемся окне выберите исполняемый файл игры с расширением «.exe».

Удаление игры из библиотеки

1. Перейдите на страницу игры.
2. Нажмите кнопку «Настройки» с иконкой шестерёнок.
3. Нажмите красную кнопку «Удалить из библиотеки».
4. Подтвердите, что хотите удалить игру, в появившемся окне нажмите «Да».

Просмотр профиля

В левом навигационном меню сверху нажмите на свой никнейм (см. рис. 30).

Просмотр статистики

Перейдите на страницу профиля (см. рис. 44).

Просмотр недавней активности

Перейдите на страницу профиля (см. рис. 44).

Просмотр несинхронизированных сохранений

Перейти на страницу «Главная» (см. рис. 37), которая располагается в левом навигационном меню приложения (см. рис. 30).

Редактирование внешнего вида профиля

1. Перейдите на страницу профиля.
2. Нажмите кнопку «Редактировать профиль» (см. рис. 44).
3. Нажмите на кнопку в виде «карандаша» напротив поля «аватар» или «хедер» (см. рис. 45).
4. В выпадающем списке выберите ваш вариант.
5. При выборе «Выбрать файл» выберите файл с расширением .jpg, .jpeg, .png, .gif, .svg.
6. При выборе «Вставить ссылку (URL)» в появившемся поле вставьте ссылку формата URL.
7. Нажмите кнопку «Сохранить».

Выход из аккаунта

1. Перейдите на страницу профиля (см. рис. 44).
2. Нажмите кнопку «Выйти».

Изменение расположения создаваемых сохранений

1. В левом навигационном меню выберите «Настройки».
2. Выберите вкладку «Главная» (см. рис. 39).
3. Нажмите кнопку «Изменить».
4. Выберите папку.

Автоматическое удаление локальных сохранений после синхронизации

1. В левом навигационном меню выберите «Настройки».
2. Выберите вкладку «Главная» (см. рис. 39).
3. Напротив текста «Удалять локальные сохранения после синхронизации» нажмите на чекбокс.
4. В чекбоксе находится галочка – опция включена.
5. В чекбоксе нет галочки – опция отключена.

Обновление пароля

1. В левом навигационном меню выберите «Настройки».
2. Выберите вкладку «Аккаунт» (см. рис. 40).
3. Нажмите кнопку «Обновить пароль».
4. В открывшемся модальном окне введите данные (см. рис. 42).
5. Нажмите кнопку «Обновить пароль».

Обновление электронной почты

1. В левом навигационном меню выберите «Настройки».
2. Выберите вкладку «Аккаунт» (см. рис. 40).
3. Нажмите кнопку «Обновить электронную почту».
4. В открывшемся модальном окне введите данные (см. рис. 41).
5. Нажмите кнопку «Обновить почту».

Подключение облачного сервиса

1. В левом навигационном меню выберите «Настройки».
2. Выберите вкладку «Аккаунт» (см. рис. 40).

3. Напротив нужного вам облачного сервиса нажмите кнопку «Подключить».
4. В открывшейся ссылке браузера войдите в свой аккаунт Google и следуйте дальнейшей инструкции от Google.

Отключение облачного сервиса

1. В левом навигационном меню выберите «Настройки».
2. Выберите вкладку «Аккаунт» (см. рис. 40).
3. Напротив нужного вам облачного сервиса нажмите кнопку «Отключить».
4. Все синхронизированные сохранения с помощью данного облачного сервиса будут скрыты, чтобы они появились – снова подключите облачный сервис.

Добавление игры в избранное

1. Перейдите на страницу игры, которая имеется у вас в библиотеке (см. рис. 31).
2. Нажмите на кнопку в виде пустого сердца (см. рис. 46).

Удаление игры из избранного

1. Перейдите на страницу игры, которая имеется у вас в библиотеке (см. рис. 31).
2. Нажмите на кнопку в виде заполненного сердца.

Настройки игры

1. Перейдите на страницу игры, которая имеется у вас в библиотеке (см. рис. 31).
2. Нажмите кнопку «Настройки» с иконкой шестерёнок (см. рис. 46).
3. В открывшемся окне (см. рис. 47) укажите путь к исполняемому файлу игры формата .exe, нажав на кнопку «Выбрать» напротив поля «Файл».
4. Если папка сохранений не была найдена автоматически – нажмите кнопку «Выбрать» напротив поля «Сохранения» и выберите корневую папку сохранений игры.

Создание сохранения

1. Перейдите на страницу игры, которая имеется у вас в библиотеке (см. рис. 31).
2. Напротив поля «Ваши сохранения» нажмите кнопку «Создать» (см. рис. 48).
3. Заполните поля в появившемся окне.
4. Нажмите кнопку «Создать» в появившемся окне.

Синхронизация сохранения

1. Выберите ваше несинхронизированное сохранения.
2. Нажмите кнопку «Синхронизировать» (см. рис. 49).
3. В появившемся окне выберите сервис.

Восстановление сохранения

1. Выберите ваше сохранение.
2. Нажмите кнопку «Восстановить» (см. рис. 49).

Перезапись сохранения

1. Выберите ваше сохранение.
2. Нажмите кнопку «Перезапись» (см. рис. 49).

Удаление сохранения

1. Выберите ваше сохранение.
2. Нажмите кнопку «Удалить» (см. рис. 49).

Поделиться сохранением

1. Выберите ваше синхронизированное сохранение.
2. Нажмите на кнопку «...» (см. рис. 49).
3. Выберите появившуюся кнопку «Поделиться» (см. рис. 52).
4. Ссылка на сохранение будет скопировано в ваш буфер обмена.

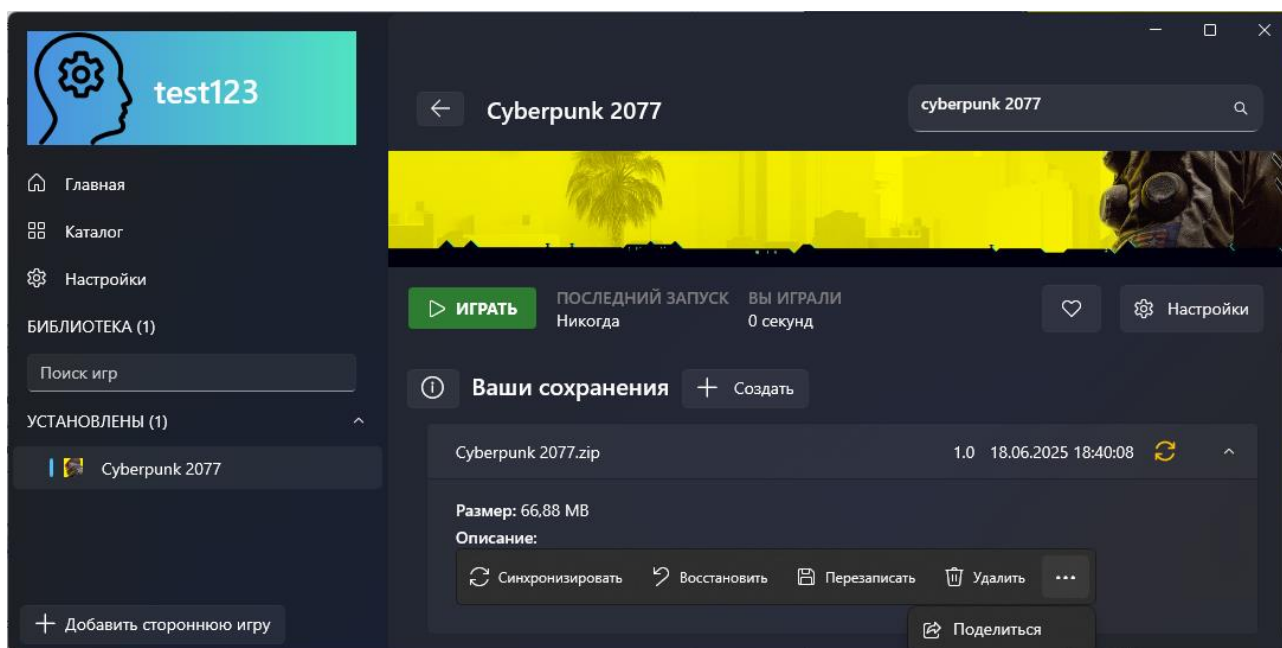


Рисунок 52 – Появившееся кнопка «Поделиться»

ЗАКЛЮЧЕНИЕ

В ходе выполнения дипломного проекта был успешно разработан сервис управления игровыми сохранениями «PlaySaveBack», полностью соответствующий поставленной цели. В рамках работы поэтапно решены все ключевые задачи:

1. Реализована эффективная структура базы данных на SQLite, обеспечивающая надежное хранение пользовательских данных и истории сохранений.
2. Разработан RESTful API на Laravel 12.
3. Создано десктопное приложение на WinUI 3 с интуитивным интерфейсом.
4. Проведено автоматизированное тестирование серверной части.
5. Разработаны сценарии ручного тестирования и проведено тестирование десктопного приложения вручную.
6. Разработано руководство пользователя для взаимодействия с десктопным приложением.

Полученные результаты имеют значительную практическую ценность, решая актуальную проблему потери игрового прогресса. Система демонстрирует потенциал для дальнейшего развития, включая возможную интеграцию с популярными игровыми платформами и различными облачными сервисами.

Таким образом, дипломный проект успешно завершен, все поставленные задачи выполнены в полном объеме, а разработанное решение готово к практическому внедрению.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Официальная документация SQLite [Электронный ресурс]. — URL: <https://www.sqlite.org/docs.html> (дата обращения: 29.03.2025).
2. Официальная документация PHP 8.3 [Электронный ресурс]. — URL: <https://www.php.net/manual/ru/migration83.new-features.php> (дата обращения: 23.01.2025).
3. Официальная документация Laravel 12 [Электронный ресурс]. — URL: <https://laravel.su/docs/12.x/database> (дата обращения: 05.03.2025).
4. Учебник по WPF и C# [Электронный ресурс]. — URL: <https://metanit.com/sharp/wpf/2.php> (дата обращения: 29.03.2025).
5. Эшкрафт Э. Изучаем WinUI 3.0. — СПб.: Питер, 2021. — 352 с.
6. Зандстра М. PHP. Объекты, шаблоны и методики программирования. — СПб.: Питер, 2023. — 576 с.